

31.2. Notación adicional de los paquetes en UML

Por último, a propósito de los paquetes, UML proporciona una notación alternativa para ilustrar paquetes internos y externos. Algunas veces es difícil dibujar una caja de paquete externo alrededor de paquetes internos. Las alternativas se muestran en la Figura 31.6.

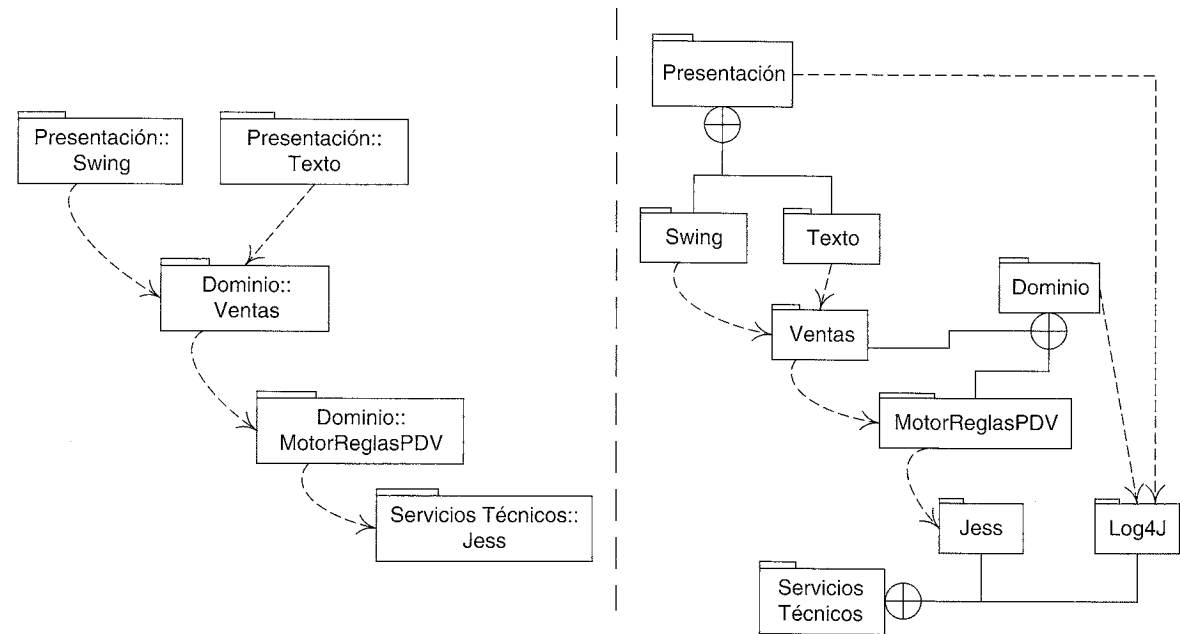


Figura 31.6. Enfoques UML alternativos para mostrar la estructura de paquetes, utilizando nombres de camino UML, o el símbolo de un círculo con una cruz.

31.3. Lecturas adicionales

La mayoría del trabajo detallado —no es sorprendente— sobre las mejoras en el diseño de paquetes para reducir el impacto de las dependencias procede de la comunidad de C++, aunque los principios se pueden aplicar a otros lenguajes. *Designing Object-Oriented C++ Applications Using the Booch Method* de Martin [Martin95] cubre bien el tema, al igual que *Large Scale C++ Software Design* [Lakos96]. El tema también se introduce en *Java 2 Performance and Idiom Guide* [GL99].

INTRODUCCIÓN AL ANÁLISIS ARQUITECTURAL Y EL SAD

Error, no hay teclado – presione F1 para continuar.
mensaje inicial de la BIOS del PC

Objetivos

- Crear tablas de factores de la arquitectura.
- Crear memorándums técnicos que recojan las decisiones acerca de la arquitectura.
- Conocer los principios básicos del diseño arquitectural.
- Conocer recursos para aprender los patrones de arquitectura.

Introducción

La esencia del análisis arquitectural es identificar los factores que deberían influir en la arquitectura, entender su variabilidad y prioridad, y resolverlos. La parte difícil es conocer qué hay que preguntar, valorando los compromisos y conociendo las muchas formas de resolver un factor significativo desde el punto de vista de la arquitectura, que se extienden desde omisiones benignas, a diseños extravagantes, o a productos de terceras partes.

En el UP, los factores de la arquitectura se recogen en la Especificación Complementaria, y las decisiones de diseño que los resuelven se recogen en el **Documento de la Arquitectura del Software** (SAD, *Software Architecture Document*, descrito con más detalle casi al final de este capítulo).

El análisis arquitectural comienza pronto durante la fase de inicio, y es uno de los puntos de interés de la fase de elaboración; es una actividad de alta prioridad y que tiene mucha influencia en el desarrollo de software. Este tema se ha aplazado hasta este

punto del libro de manera que se pudieran presentar en primer lugar los fundamentos del A/DOO. Es una actividad útil para:

- reducir el riesgo de olvidar algo esencial en el diseño de los sistemas
- evitar dedicar excesivo esfuerzo a cuestiones de poca prioridad
- ayudar a alinear el producto con los objetivos del negocio

Este capítulo es una introducción a los pasos e ideas básicas del análisis arquitectural desde la perspectiva del UP; es decir, al método, en lugar de a los consejos y astucias de los arquitectos expertos. De este modo, no es un libro de recetas de cocina de soluciones de arquitectura —un tema amplio y dependiente del contexto que va más allá del alcance de este libro introductorio—. No obstante, el caso de estudio del PDV NuevaEra que se comenta en el capítulo proporciona ejemplos concretos de soluciones relacionadas con la arquitectura.

32.1. Análisis arquitectural

El **análisis arquitectural** trata de la identificación y resolución de los requisitos no funcionales del sistema (por ejemplo, calidad), en el contexto de los requisitos funcionales.

En el UP, el término comprende tanto la investigación arquitectural (identificación) como el diseño arquitectural (resolución). A continuación se presentan algunos ejemplos de muchas de las cuestiones que se tienen que identificar y resolver al nivel de la arquitectura:

- ¿Cómo afectan en el diseño los requisitos de fiabilidad y tolerancia a fallos?
 - Por ejemplo, en el PDV NuevaEra, ¿para qué servicios remotos (ej. calculador de impuestos) se les podrá mantener el servicio ante fallos mediante servicios locales? ¿Por qué? ¿Proporcionan exactamente los mismos servicios localmente que de manera remota, o existen diferencias?
- ¿Cómo afecta en la rentabilidad el coste de las licencias de los subcomponentes comprados?
 - Por ejemplo, el fabricante del excelente servidor de bases de datos, *SinPistas*, quiere el 2% de cada PDV NuevaEra que se venda, si se utiliza su producto como subcomponente. La utilización de su producto acelerará el desarrollo (y su salida al mercado) porque es robusto y proporciona muchos servicios, y lo conocen muchos desarrolladores, pero esto tiene un precio. ¿En lugar de eso debería utilizar el equipo el servidor de bases de datos *SuSQL* de libre distribución, menos robusto? ¿Con qué riesgo? ¿Cómo limita la facultad de cobrar por el producto NuevaEra?
- ¿Cómo afecta la distribución de los servicios en los requisitos de calidad y los requisitos funcionales?
 - Por ejemplo, utilizando un sistema calculador de impuestos remoto (único y centralizado) reduce la huella de cada cliente NuevaEra, reduce los costes de li-

cencia (sólo se necesita una copia), y minimiza el esfuerzo de configuración para un cliente determinado (cada instalación requiere ajustes semanales debido a cambios en las políticas del negocio y en las de legislación). Sin embargo, el servicio remoto reduce el tiempo de respuesta lo suficiente para que los impuestos sólo se puedan calcular una vez, después de introducir todos los artículos; uno no puede ver un precio parcial con impuestos tras cada línea de venta; y la llamada remota tarda mucho. También crea un único punto de fallo.

- ¿Cómo afectan al diseño los requisitos de adaptabilidad y de configuración?
 - Por ejemplo, la mayoría de las tiendas difieren en las reglas del negocio que quieren representar en sus aplicaciones de PDV. ¿Cuáles son las variaciones? ¿Cuál es la “mejor” forma de diseñarlas? ¿Cuáles son los criterios para la mejor? ¿Puede NuevaEra ganar más dinero exigiendo que se adapte la programación a cada cliente (y, ¿cuánto esfuerzo supondrá?), o con una solución que permita a los clientes que las adapten ellos mismos fácilmente? ¿“Más dinero” debería ser el objetivo a corto plazo?

Pasos comunes en el análisis arquitectural

Existen varios métodos para el análisis arquitectural. Comunes a la mayoría de ellos son algunas variaciones de los siguientes pasos:

1. Identificar y analizar los requisitos no funcionales que influyen en la arquitectura. Los requisitos funcionales también son relevantes (especialmente en términos de variabilidad o cambio), pero se les presta una atención completa a los no funcionales. En general, todos éstos podrían llamarse **factores de la arquitectura** (también conocidos como **controladores de la arquitectura**, *architectural drivers*).
 - Este paso podría caracterizarse como un análisis de requisitos ordinario, pero puesto que se lleva a cabo en el contexto de la identificación del impacto de la arquitectura y en la toma de decisiones sobre las soluciones de la arquitectura de alto nivel, se considera como parte del análisis arquitectural en el UP.
 - Por lo que se refiere al UP, algunos de estos requisitos se identificarán y recopilarán en líneas generales en la Especificación Complementaria o en los casos de uso durante la fase de inicio. Durante el análisis arquitectural, que tiene lugar al principio de la elaboración, el equipo investiga estos requisitos más detenidamente.
2. Para aquellos requisitos que influyen de manera significativa en la arquitectura, analizar las alternativas y crear soluciones que resuelvan el impacto. Éstas son las **decisiones sobre la arquitectura**.
 - Las decisiones varían desde “eliminar el requisito”, a una solución a medida, a “detener el proyecto”, o a “contratar un experto”.

Esta presentación introduce estos pasos básicos en el contexto del caso de estudio del PDV NuevaEra. Por simplicidad, evita cuestiones del despliegue de la arquitectura tales

como la configuración del hardware y del sistema operativo, que son muy sensibles al contexto y al momento en que se hace.

32.2. Tipos y vistas de la arquitectura

Algunas de las descripciones de arquitectura definen tipos diferentes, como la “arquitectura de aplicación” (asignación de características a componentes) o “arquitectura del sistema” (configuración del hardware y del sistema operativo).

En el UP, existe una especialización de la información parecida, pero se describen en “vistas” de la arquitectura, que resumen y destacan una perspectiva particular. Por ejemplo, la **vista lógica** de la arquitectura, que se introdujo en el Capítulo 30, resume la organización y la funcionalidad de los elementos del software importantes (como las capas) —es similar al término arquitectura de la aplicación—. La **vista de despliegue** resume la topología del sistema, las comunicaciones y la correspondencia entre los elementos ejecutables y los nodos de proceso —es análogo al término arquitectura del sistema.

El UP define seis vistas de la arquitectura, que se describen en detalle casi al final de este capítulo. Concretamente, las vistas combinan texto y diagramas, y —si se describen del todo— se recogen en el SAD.

El análisis arquitectural se relaciona con las vistas de la arquitectura porque las decisiones sobre la arquitectura se reflejan y describen en una o más vistas de la arquitectura.

32.3. La ciencia: identificación y análisis de los factores de la arquitectura

Factores de la arquitectura

Cualquiera y todos los requisitos FURPS+ pueden influir de manera significativa en la arquitectura de un sistema, variando desde la fiabilidad, a la planificación, a las habilidades y a las restricciones de coste. Por ejemplo, un caso de planificación ajustada, habilidades limitadas y dinero suficiente probablemente favorece contratar o alquilar especialistas externos, en lugar de construir todos los componentes dentro de la compañía.

Sin embargo, los factores que influyen con más fuerza en la arquitectura tienden a encontrarse en las categorías FURPS+ de alto nivel de funcionalidad, fiabilidad, rendimiento, soporte, implementación e interfaz (véase el Capítulo 5 para un desglose detallado). Curiosamente, son los atributos de calidad no funcionales (como la fiabilidad o el rendimiento) los que dan a una arquitectura concreta su sabor único, en lugar de sus requisitos funcionales. Por ejemplo, el diseño en el sistema NuevaEra para dar soporte a diferentes componentes de terceras partes con interfaces únicas, y el diseño para dar soporte a la conexión fácil de diferentes conjuntos de reglas del negocio.

En el UP, estos factores con implicaciones en la arquitectura se denominan **requisitos significativos para la arquitectura**. Utilizamos aquí “factores” para simplificar.

Se pueden caracterizar muchos factores técnicos y organizativos como *restricciones* que limitan la solución de alguna manera (como, debe ejecutarse en Linux, o el presupuesto para comprar componentes de terceras partes es X).

Escenarios de calidad

Cuando se definen los requisitos de calidad durante el análisis de los factores de la arquitectura, se recomienda el uso de los **escenarios de calidad**¹, ya que definen respuestas cuantificables (o por lo menos observables) y, por tanto, se pueden verificar. No sirve de mucho establecer de manera vaga “el sistema será fácil de modificar” sin ninguna medida de lo que eso significa.

Cuantificar algunas cosas, como los objetivos de rendimiento y el tiempo entre fallos, son prácticas bien conocidas, pero los escenarios de calidad amplían esta idea y promueve la recopilación de todos (o al menos, la mayoría) los factores como sentencias cuantificables.

Los escenarios de calidad son sentencias cortas de la forma <estímulo> <respuesta cuantificable>; por ejemplo:

- Cuando se envía la venta completa al calculador de impuestos remoto para añadir los impuestos, el resultado se devolverá en 2 segundos “la mayoría” de las veces, medido en un entorno de producción bajo condiciones de carga “media”.
- Cuando llega un informe de errores de un voluntario de una prueba de una versión beta de NuevaEra, responda con una llamada de teléfono en un día laborable.

Nótese que será necesario que el arquitecto de NuevaEra realice un estudio adicional y defina “la mayoría” y “media”; un escenario de calidad no es realmente válido hasta que no se puede probar, lo que implica que se ha especificado completamente. También, observe la calificación en el primer escenario de calidad en función del entorno en el que se aplica. Esto mejora algo la especificación de un escenario de calidad, verifica que lo pasa en un entorno de desarrollo ligeramente cargado, pero falla al evaluarlo en un entorno de producción realista.

Escoja sus Batallas

Una advertencia: la escritura de estos escenarios de calidad puede ser un espejismo de utilidad. Es fácil *escribir* estas especificaciones detalladas, pero no materializarlas. ¿Alguien las probará realmente alguna vez? ¿Cómo y por quién? Se requiere una fuerte dosis de realismo cuando se escriben; no tiene sentido listar muchos objetivos sofisticados si realmente nunca nadie acabará comprobándolos.

Esto está relacionado con la discusión acerca de “escoja sus batallas” que se presentó en un capítulo anterior sobre el patrón Variaciones Protegidas. ¿Cuáles son realmente los escenarios de calidad críticos que causan el éxito o el fracaso? Por ejemplo, en un sistema de reservas de vuelos, es realmente crítico para el éxito del sistema que se completen las

¹ Un término utilizado en varios métodos de la arquitectura promovido por el Instituto de Ingeniería del Software (SEI, *Software Engineering Institute*); por ejemplo, en el método de *Diseño Basado en la Arquitectura*.

transacciones de manera rápida y consistente bajo condiciones de carga alta —debe comprobarse sin lugar a dudas—. En el sistema NuevaEra, la aplicación realmente deber ser tolerante a fallos y mantener el servicio ante los fallos mediante copias locales de los servicios cuando fallan los remotos —sin duda se debe probar y validar debidamente—. Por tanto, céntrese en escribir los escenarios de calidad para las batallas importantes, y siga desde el principio hasta el final un plan para evaluarlos.

Descripción de los factores

Un objetivo importante del análisis arquitectural es entender la influencia de los factores, sus prioridades, y la manera en que varían (necesidad inmediata de flexibilidad y evolución futura). Por tanto, la mayoría de los métodos de la arquitectura (por ejemplo, véase [HNS00]) abogan por la creación de una tabla o árbol con variaciones de la siguiente información (el formato varía dependiendo del método). El siguiente estilo que se muestra en la Tabla 32.1 se denomina **tabla de factores**, que en el UP forma parte de la Especificación Complementaria.

Tabla 32.1. Ejemplo de tabla de factores.

Factor	Medidas y Escenarios de Calidad	Variabilidad (flexibilidad actual y futura evolución)	Impacto del factor (y su variabilidad) en las personas involucradas, arquitectura y otros factores	Prioridad para el éxito	Dificultad o Riesgo
Fiabilidad-Capacidad de recuperación					
Recuperación de los fallos en los servicios remotos	Cuando falla un servicio remoto, restablecer la conexión con él en 1 minuto, bajo una carga normal de la tienda en un entorno de producción.	Flexibilidad actual: nuestro EME dice que son aceptables (y convenientes) los servicios simplificados locales del lado del cliente hasta que sea posible la re-conexión. Evolución: en 2 años, algunas tiendas podrían tener la intención de pagar por una copia local completa de los servicios remotos (como el calculador de impuestos). ¿Probabilidad? Alta.	Impacto alto en el diseño a gran escala. A las tiendas realmente les disgusta cuando fallan los servicios remotos, ya que les impide o restringe el uso de un PDV para realizar las ventas.	A	M
...	...	...	...		

Leyenda: A: Alta. M: Media. EME: Experto en la Materia de Estudio.

Obsérvese el esquema de clasificación: *Fiabilidad—Capacidad de recuperación* (a partir de las categorías del FURPS+). Éste no se presenta como el mejor o el único esquema, pero resulta útil para agrupar los factores de la arquitectura en categorías. Por ejemplo,

ciertas categorías (como fiabilidad y rendimiento) están fuertemente relacionadas con la identificación y definición de los planes de pruebas y, por tanto, es conveniente agruparlas.

Los valores de los códigos básicos para la prioridad y el riesgo de A/M/B simplemente insinúan que se utilicen algunos códigos que el equipo encuentre útiles; existe una variedad de esquemas de codificación (numéricos y cualitativos) procedentes de diferentes métodos y estándares (como el ISO 9126) de arquitectura. Una advertencia: Si el esfuerzo extra de utilizar un esquema más complejo no le conduce a ninguna acción práctica, no merece la pena.

Factores y los artefactos del UP

El repositorio central de requisitos funcionales en el UP son los casos de uso, y ellos, junto con la Visión y la Especificación Complementaria, son una fuente de inspiración importante cuando se está creando una tabla de factores. En los casos de uso, se deberían revisar los *Requisitos especiales*, las *Variaciones de la tecnología* y los *Temas abiertos*, y reunir sus factores de la arquitectura implícitos o explícitos en la Especificación Complementaria.

Es razonable recoger al principio los factores relacionados con casos de uso en los casos de uso mientras se están creando, debido a que existe una relación obvia, pero al final es más conveniente (en cuanto a la gestión del contenido, traza y legibilidad) reunir todos los factores de la arquitectura en un sitio —en la tabla de factores en la Especificación Complementaria—.

Caso de Uso UC1: Procesar Venta

Escenario principal de éxito:
1. ...

Requisitos especiales:

- El tiempo de respuesta para la autorización de crédito será de 30 segundos el 90% de las veces
- De algún modo, queremos que el sistema se recupere de manera robusta cuando falla el acceso a servicios remotos, como el sistema de inventario.
- ...

Lista de tecnología y variaciones de datos:
2a. El identificador del artículo se introduce por medio de un escáner láser de código de barras (si está presente el código de barras) o del teclado.
...

Temas abiertos:

- ¿Cuáles son las variaciones en la ley de impuestos?
- Estudiar las cuestiones sobre la recuperación de los servicios remotos.

32.4. Ejemplo: tabla de factores parcial de la arquitectura del PDV NuevaEra

La tabla de factores parcial de la Tabla 32.2 muestra algunos factores relacionados con la discusión posterior.

Tabla 32.2. Tabla de factores parcial para el análisis arquitectural de NuevaEra.

Factor	Medidas y Escenarios de Calidad	Variabilidad (flexibilidad actual y futura evolución)	Impacto del factor (y su variabilidad) en las personas involucradas, arquitectura y otros factores	Prioridad para el éxito	Dificultad o Riesgo
Fiabilidad—Capacidad de recuperación					
Recuperación de los fallos en los servicios remotos.	Cuando falla un servicio remoto, restablecer la conexión con él en 1 minuto, bajo una carga normal de la tienda en un entorno de producción.	Flexibilidad actual: nuestro EME dice que son aceptables (y convenientes) los servicios simplificados locales del lado del cliente hasta que sea posible la reconexión. Evolución: en 2 años, algunas tiendas podrían tener la intención de pagar por una copia local completa de los servicios remotos (como el calculador de impuestos). ¿Probabilidad? Alta.	Impacto alto en el diseño a gran escala. A las tiendas realmente les disgusta cuando fallan los servicios remotos, ya que les impide o restringe el uso de un PDV para realizar las ventas.	A	M
Recuperación de los fallos en bases de datos de productos remotas.	Como arriba.	Flexibilidad actual: nuestro EME dice que es aceptable (y conveniente) que se almacene en el lado del cliente información acerca de los productos “más comunes” hasta que sea posible la reconexión. Evolución: en 3 años, las soluciones de almacenamiento masivo y replicación en el lado del cliente, serán baratas y efectivas, permitiendo que se mantenga una copia completa permanente y así el uso local. ¿Probabilidad? Alta.	Como arriba.	A	M
Soporte—Adaptabilidad					
Dar soporte a muchos servicios de terceras partes (calculador de impuestos, inventario, RRHH, contabilidad). Variarán en cada instalación.	Cuando se deba integrar un nuevo servicio de terceras partes, se puede hacer con un esfuerzo de 10 días persona.	Flexibilidad actual: como se describe en el factor. Evolución: ninguna.	Se requiere para la aprobación del producto. Poco impacto en el diseño.	A	B

(Continúa)

Tabla 32.2. Tabla de factores parcial para el análisis arquitectural de NuevaEra. (Continuación)

Factor	Medidas y Escenarios de Calidad	Variabilidad (flexibilidad actual y futura evolución)	Impacto del factor (y su variabilidad) en las personas involucradas, arquitectura y otros factores	Prioridad para el éxito	Dificultad o Riesgo
Soporte—Adaptabilidad (continuación)					
¿Dar soporte a terminales PDA inalámbricos para los clientes del PDV?	Cuando se añade el soporte a estos terminales, no requiere que se cambie el diseño de las capas de la arquitectura que no tienen que ver con el UI.	Flexibilidad actual: no se requiere en este momento. Evolución: en 3 años, pensamos que es muy probable que el mercado demande “PDAs” inalámbricos para los clientes del PDV.	Impacto alto en el diseño en cuanto a las variaciones protegidas de muchos elementos. Por ejemplo, los sistemas operativos y las UIs son diferentes en dispositivos pequeños.	B	A
Otros—Legal					
Se deben aplicar la legislación de impuestos actual.	Cuando el auditor evalúe si se ajusta, encontrará que se ajusta al 100%. Cuando cambia la legislación de los impuestos, estarán operativas dentro del plazo marcado por el gobierno.	Flexibilidad actual: la conformidad es inflexible, pero la legislación de los impuestos pueden cambiar casi semanalmente debido a que hay muchas leyes y niveles de impuestos del gobierno (nacional, regional,...). Evolución: ninguna.	Fallar en el cumplimiento es un delito. Influye en los servicios de cálculo de impuestos. Es difícil escribir nuestro propio servicio —leyes complejas, cambios constantes, la necesidad de seguir la pista a todos los niveles de gobierno—. Aunque, riesgo fácil/bajo si se compra un paquete.	A	B

Leyenda: A: Alta; M: Media; B: Baja; EME: Experto en la Materia de Estudio.

32.5. El arte: resolución de los factores de la arquitectura

Uno podría decir que la *ciencia* de la arquitectura es la recolección y organización de la información sobre los factores de la arquitectura, como en la tabla de factores. El *arte* de la arquitectura es tomar las decisiones acertadas para resolver estos factores, teniendo en cuenta los compromisos, interdependencias y prioridades.

Los arquitectos expertos conocen una variedad de áreas (por ejemplo, estilos y patrones de arquitectura, tecnologías, productos, peligros y tendencias) y las aplican a sus decisiones.

Registro de alternativas, decisiones y la motivación de la arquitectura

Ignorando por ahora los principios de la toma de decisiones sobre la arquitectura, casi todos los métodos de arquitectura recomiendan mantener un registro de las soluciones alternativas, decisiones, factores que influyen, y el motivo de las cuestiones y decisiones relevantes.

A tales registros se les ha llamado **memorándums técnicos** [Cunningham96], **tarjetas de cuestiones** [HNS00], y **documentos de técnicas de arquitectura** (propuestas de arquitectura del SEI), con diversos grados de formalidad y sofisticación. En algunos métodos, estos memorándums son la base para todavía otro paso de revisión y refinamiento.

En el UP, los memorándums se deben recoger en el SAD.

Un aspecto importante de los memorándums técnicos es la *motivación* o los fundamentos. Cuando un futuro desarrollador o arquitecto necesita modificar el sistema², es enormemente útil entender los motivos que están detrás del diseño, como *por qué* se escogió un enfoque concreto para la recuperación de los fallos de los servicios remotos en el PDV NuevaEra y se rechazaron otros, con el objeto de tomar decisiones con conocimiento de causa sobre los cambios del sistema.

Es importante explicar los motivos para rechazar las alternativas, ya que durante la evolución futura del producto, un arquitecto podría reconsiderar estas alternativas, o por lo menos querer conocer qué alternativas se consideraron, y por qué se escogió una.

A continuación presentamos un memorándum técnico de ejemplo que recoge una decisión sobre la arquitectura para el PDV NuevaEra. Por supuesto, el formato exacto no es importante. Manténgalo simple y recoja únicamente la información que ayudará a los futuros lectores a tomar decisiones con conocimiento de causa cuando estén cambiando el sistema.

Memorándum Técnico

Asunto: Fiabilidad—Recuperación de fallos en los servicios remotos

Resumen de la solución: Ubicación transparente utilizando un servicio de búsqueda, mantenimiento del servicio ante los fallos pasando de remoto a local, y replicación parcial local del servicio.

Factores

- Recuperación robusta de los fallos en los servicios remotos (ej. calculador de impuestos, inventario)

² ¡O cuando han pasado cuatro semanas y el arquitecto original ha olvidado su propia motivación!

- Recuperación robusta de los fallos en la base de datos de productos (ej. descripción de productos y precios) remota

Solución

Conseguir variaciones protegidas con respecto a la ubicación de los servicios utilizando un Adaptador creado en una FactoriaDeServicios. Donde sea posible, ofrecer implementaciones locales de servicios remotos, normalmente con comportamiento simplificado o restringido. Por ejemplo, el calculador de impuestos local utilizará porcentajes de impuestos constantes. La base de datos con información de los productos local será una pequeña caché de los productos más comunes. Se almacenarán las actualizaciones del inventario y se remitirán cuando se restablezca la conexión.

Véase también el memorándum técnico de *Adaptabilidad—Servicios de terceras partes* para conocer los aspectos de adaptabilidad de esta solución, porque las implementaciones de los servicios remotos variarán en cada instalación.

Para satisfacer los escenarios de calidad de reconexión con los servicios remotos ASAP, utilizar para los servicios objetos Proxy inteligentes, que en cada llamada a un servicio comprueban si se puede reactivar el servicio remoto y lo redireccionan a éste cuando es posible.

Motivación

¡Las tiendas realmente no quieren dejar de vender! Por tanto, si el PDV NuevaEra ofrece este nivel de fiabilidad y recuperación, será un producto muy atractivo, ya que ninguno de nuestros competidores proporciona esta capacidad. La pequeña caché de productos está motivada por la gran limitación de recursos en el lado del cliente. El calculador de impuestos de terceras partes real no se duplica en el cliente fundamentalmente debido a que el coste de las licencias es más alto y a los esfuerzos de configuración (puesto que cada instalación del calculador requiere ajustes casi semanales). Este diseño también soporta los puntos de evolución de los deseos de futuros clientes y es capaz de replicar los servicios permanentemente como el calculador de impuestos en cada terminal del cliente.

Cuestiones sin resolver

Ninguna

Alternativas consideradas

Una calidad de “nivel oro” del contrato de servicio con los servicios de autorización de crédito para mejorar la fiabilidad. Estaba disponible, pero era demasiado caro.

Nótese como se ilustra en este ejemplo —y es un punto clave— que una decisión sobre la arquitectura descrita en un memorándum técnico podría resolver un grupo de factores, no sólo uno.

Prioridades

Hay una jerarquía de objetivos que guían las decisiones sobre la arquitectura:

- Restricciones inflexibles, entre las que se encuentran ajustarse a las normas legales y de seguridad.
 - El PDV NuevaEra debe aplicar correctamente las leyes de impuestos.
- Objetivos del negocio.
 - Demo de las características relevantes listo para la feria POSWorld de Hamburgo dentro de 18 meses.

- Tiene cualidades y características atractivas para los grandes almacenes de Europa (por ejemplo, que soporte diferentes monedas y reglas de negocio a medida).
3. Todos los otros objetivos.
- A menudo se les pueden seguir la pista hacia atrás hasta establecer directamente objetivos del negocio, pero son indirectos. Por ejemplo, se podría establecer la traza entre “fácilmente extensible: puede añadir <alguna unidad funcional> en 10 semanas persona” y el objetivo del negocio de “nueva versión cada seis meses”.

En el UP, muchos de estos objetivos se recogen en el artefacto de Visión. Recuerde que los valores de la *Prioridad para el éxito* en la tabla de factores debería reflejar la prioridad de estos objetivos.

Existe un aspecto distintivo de la toma de decisiones a este nivel frente al diseño de objetos a pequeña escala: uno tiene que considerar simultáneamente más objetivos (y a menudo que influyen globalmente) y sus compromisos. Además, los objetivos del negocio pasan a ser cruciales para las decisiones técnicas (o al menos deberían). Por ejemplo:

Memorándum Técnico

Asunto: Legal—Cumplimiento de la legislación de impuestos

Resumen de la solución: Comprar un componente para el cálculo de impuestos.

Factores

- Se deben aplicar, por ley, la legislación de impuestos actuales.

Solución

Comprar un calculador de impuestos con un acuerdo de licencia para recibir actualizaciones de la legislación de impuestos actuales. Nótese que se podrían utilizar diferentes calculadores en instalaciones distintas.

Motivación

Acelerar la salida al mercado, corrección, requisitos de mantenimiento bajos, y desarrolladores felices (véase las alternativas). Estos productos son costosos, lo que afecta a nuestros objetivos del negocio de política de contención de costes y de política de fijación de precios, pero la alternativa se considera inaceptable.

Cuestiones sin resolver

¿Cuáles son los productos líderes y sus cualidades?

Alternativas consideradas

¿Qué el equipo de NuevaEra construya uno? Se estima que puede llevar mucho tiempo, ser propenso a fallos, y crea una responsabilidad de mantenimiento continua, costosa y sin interés (para los desarrolladores de la compañía), que afecta al objetivo de “desarrolladores felices” (seguramente, el objetivo más importante de todos).

Prioridades y puntos de evolución: ingeniería por defecto o en exceso

Otra característica distintiva de la toma de decisiones sobre la arquitectura es establecer prioridades de acuerdo a la probabilidad de los **puntos de evolución** —puntos de variabilidad o cambio que *podrían* surgir en el futuro—. Por ejemplo, en NuevaEra, existe una posibilidad de que se desee terminales clientes portátiles e inalámbricos. El diseño para esto influye de manera significativa debido a las diferencias en los sistemas operativos, interfaces de usuario, recursos hardware, etcétera.

La compañía podría gastarse una cantidad enorme de dinero (e incrementar diversos riesgos) para conseguir esta “futura necesidad”. Si en el futuro resulta que no es relevante, hacerlo sería un ejercicio muy caro de sobre-ingeniería. Nótese también que se puede sostener que las futuras necesidades rara vez se dan, puesto que son especulaciones; incluso si ocurre el cambio previsto, es probable que se produzcan algunos cambios en el diseño supuesto.

Por otro lado, las futuras necesidades contra el problema de datos Y2K habría sido dinero bien empleado; en lugar de eso, hubo un esfuerzo de ingeniería por defecto con un resultado tremendamente costoso.

El arte del arquitecto es conocer qué batallas merece la pena pelear —dónde merece la pena invertir en diseños que protejan contra cambios evolutivos—.

Para decidir si se deberían evitar “futuras necesidades” prematuras, considere realmente el escenario de posponer el cambio para el futuro, cuando se requiera. ¿Cuánto del diseño y del código tendrá que cambiar en realidad? ¿Cuál será el esfuerzo? Quizás un estudio detallado del cambio potencial revelará que lo que al principio se consideró una cuestión gigantesca contra la que protegerse, se estima que el esfuerzo será sólo de unas pocas semanas-persona.

Esto es precisamente un problema difícil: “La predicción es muy difícil, especialmente si trata sobre el futuro” (atribuido, aunque sin comprobar, a Niels Bohr).

Principios básicos de diseño de la arquitectura

Los principios básicos de diseño que se han estudiado en gran parte de este libro que se aplicaron al diseño de objetos a pequeña escala, todavía son principios de gran influencia al nivel de la arquitectura a gran escala:

- Bajo acoplamiento.
- Alta cohesión.
- Variaciones protegidas (interfaces, indirección, servicio de búsqueda, etcétera).

Sin embargo, la granularidad de los componentes es mayor —se trata de bajo acoplamiento entre aplicaciones, subsistemas, o procesos, en lugar de entre pequeños objetos—.

Además, a esta escala más amplia, hay más o diferentes mecanismos para conseguir cualidades como el bajo acoplamiento y variaciones protegidas. Por ejemplo, considere el siguiente memorándum técnico:

Memorándum Técnico

Asunto: Adaptabilidad—Servicios de terceras partes

Resumen de la solución: Variaciones Protegidas utilizando interfaces y adaptadores.

Factores

- Dar soporte a muchos y cambiables servicios de terceras partes (calculadores de impuestos, autorización de crédito, inventario...).

Solución

Conseguir variaciones protegidas como sigue: Analizar varios productos para el cálculo de impuestos comerciales (y así sucesivamente para las otras categorías de productos) y construir interfaces comunes para el mínimo común denominador de funcionalidades. Entonces utilizar la Indirección mediante el patrón Adaptador. Es decir, crear un objeto Adaptador de recursos que implementa la interfaz y actúa como conexión y traductor con un calculador de impuestos back-end concreto.

Véase también el memorándum técnico *Fiabilidad—Recuperación de fallos en los servicios remotos* para conocer los aspectos de la ubicación transparente de esta solución.

Motivación

Simple. Comunicación más barata y más rápida que utilizando un servicio de intercambio de mensajes (ver alternativas), y en cualquier evento, un servicio de intercambio de mensajes no se puede utilizar para conectar directamente con el servicio de autorización de crédito externo.

Cuestiones sin resolver

¿Originarán las interfaces del mínimo común denominador un problema imprevisto, tal como ser demasiado limitada?

Alternativas consideradas

Aplicar indirección utilizando un servicio de intercambio de mensajes o publicar-suscribir (ej. una implementación JMS) entre el cliente y el calculador de impuestos, con adaptadores. Pero no se puede utilizar directamente con un autorizador de crédito, costoso (para los fiables), y más fiabilidad en la entrega de mensajes de lo que se necesita en la práctica.

El hecho es que al nivel de la arquitectura, existen normalmente nuevos mecanismos para conseguir variaciones protegidas (y otros objetivos), con frecuencia en colaboración con componentes de terceras partes, como utilizar el Servicio de Mensajes de Java (JMS; *Java Messaging Service*) o el servidor EJB.

Separación de intereses y localización del impacto

Otro principio básico que se aplica durante el análisis arquitectural es conseguir **separación de intereses**. Esto también es aplicable en la escala de pequeños objetos, pero alcanza relevancia durante el análisis arquitectural.

Los **intereses transversales** son aquéllos con amplia aplicación o influencia en el sistema, como la persistencia de datos o la seguridad. Uno *podría* diseñar el soporte a la persistencia en la aplicación NuevaEra de manera que cada objeto (que contiene código de la lógica de la aplicación) se comuniquen él mismo con una base de datos para almacenar sus datos. Estó entrelazaría el interés de la persistencia con el de la lógica de la

aplicación, en el código fuente de las clases —así también con la seguridad—. La cohesión disminuye y aumenta el acoplamiento.

En cambio, diseñando para conseguir una separación de intereses se separa el soporte a la persistencia y el soporte a la seguridad en “cosas” separadas (existen mecanismos muy diferentes para esta separación). Un objeto con lógica de la aplicación únicamente tiene lógica de la aplicación, no lógica de persistencia o de seguridad. Igualmente, un subsistema de persistencia se centra en las cuestiones de persistencia, no de seguridad. Un subsistema de seguridad no lleva a cabo la persistencia.

La separación de intereses es una forma de pensar a gran escala acerca del bajo acoplamiento y alta cohesión al nivel de arquitectura. También se aplica a objetos a pequeña escala, porque su ausencia da lugar a objetos sin cohesión que tienen muchas áreas de responsabilidad. Pero es una cuestión de arquitectura especialmente porque los intereses son amplios, y las soluciones conllevan elecciones de diseño importantes y fundamentales.

Existen por lo menos tres técnicas a gran escala para conseguir la separación de intereses:

1. Tratar el interés como un módulo en un componente separado (por ejemplo, un subsistema) e invocar sus servicios.
 - Éste es el enfoque más común. Por ejemplo, en el sistema NuevaEra, el soporte a la persistencia podría colocarse en un subsistema aparte denominado *servicio de persistencia*. Mediante una fachada, puede ofrecer una interfaz público de servicios a otros componentes. Las arquitecturas en capas también ilustran esta separación de intereses.
2. Utilizar decoradores.
 - Éste es el segundo enfoque más común; se dio a conocer en primer lugar en el Servicio de Transacciones de Microsoft, y posteriormente con los servidores EJB. En este enfoque, un interés (como la seguridad) decora otros objetos con un objeto Decorador que encapsula el objeto interno e interpone el servicio. Al Decorador se le denomina **contenedor** en la terminología EJB. Por ejemplo, en el sistema de PDV NuevaEra, el control de seguridad a los servicios externos como el sistema de RRHH se puede conseguir con un contenedor EJB que añade las comprobaciones de seguridad en el Decorador externo, alrededor de la lógica de la aplicación del objeto interno.
3. Utilizar post-compiladores y tecnologías orientadas a aspectos.
 - Por ejemplo, con entidades *bean* EJB uno puede añadir el soporte a la persistencia a clases como *Venta*. Uno especifica en un fichero de descripción de propiedades las características de persistencia de la clase *Venta*. Entonces, un post-compilador (que es otro compilador que se ejecuta después de un compilador “ordinario”) añadirá el soporte a la persistencia necesario en una clase *Venta* modificada (modificando únicamente los byte-codes) o una subclase. El desarrollador continúa viendo la clase original como una clase “limpia” que sólo contiene la lógica de la aplicación. Otra variación son las tecnologías **orientadas a aspectos** como AspectJ

(www.aspectj.org), que de manera similar da soporte a entrelazar post-compilación de intereses transversales en el código, de manera transparente para el desarrollador. Estos enfoques mantienen la ilusión de la separación durante el trabajo de desarrollo, y entrelazan el interés antes de la ejecución.

Promoción de patrones de arquitectura

Un estudio de los patrones de arquitectura y del modo en el que se podrían aplicar (o aplicar mal) al caso de estudio NuevaEra queda fuera del alcance de este texto introductorio. Sin embargo, a continuación presentamos algunas indicaciones:

Probablemente el mecanismo más común para conseguir bajo acoplamiento, variaciones protegidas y separación de intereses al nivel de la arquitectura es el patrón Capas, que se introdujo en un capítulo anterior. Éste es un ejemplo de la técnica más común de separación —tratar los intereses como módulos, en componentes separados o capas—.

Existe un amplio y creciente cuerpo de material escrito sobre patrones de arquitectura. Estudiarlos es la manera más rápida que conozco de aprender soluciones de la arquitectura. Por favor, vea las lecturas recomendadas.

32.6. Resumen de los temas del análisis arquitectural

Un asunto a destacar es que las cuestiones “arquitecturales” están especialmente relacionadas con los requisitos no funcionales, y conllevan la percepción del contexto del negocio o del mercado de la aplicación. Al mismo tiempo, no se pueden ignorar los requisitos funcionales (por ejemplo, procesar ventas); éstos proporcionan el contexto en el que se deben resolver estas cuestiones. Por otro lado, la identificación de su variabilidad es significativa para la arquitectura.

Un segundo tema es que las cuestiones arquitecturales implican problemas al nivel del sistema, a gran escala y amplios, cuya resolución normalmente conlleva decisiones de diseño a gran escala o fundamentales; por ejemplo, la elección de —o incluso el uso de— un servidor de aplicación.

Un tercer tema en el análisis arquitectural son las interdependencias y compromisos. Por ejemplo, la mejora de la seguridad podría afectar al rendimiento o a la facilidad de uso, y la mayoría de las opciones afectan al coste.

Un cuarto tema del análisis arquitectural es la generación y evaluación de soluciones alternativas. Un arquitecto experto puede ofrecer soluciones de diseño que impliquen la construcción de nuevo software, y también sugerir soluciones (o soluciones parciales) que utilicen software y hardware comercial o de libre distribución. Por ejemplo, la recuperación en un servidor remoto del PDV NuevaEra se puede conseguir mediante el diseño y programación de procesos “guardianes (*watchdog*)”, o quizás a través de la agrupación, duplicación y los servicios para sobreponerse a fallos que ofrecen algunos sistemas operativos y componentes hardware. Los buenos arquitectos conocen los productos hardware y software de terceras partes.

La definición inicial de cuestiones de la arquitectura proporciona el marco para el modo de pensar sobre el tema de la arquitectura: identificando los puntos con implicaciones a gran escala o a nivel del sistema, y resolviéndolos.

Un análisis arquitectural se preocupa de la identificación y resolución de los requisitos no funcionales (p.e. calidad) del sistema, en el contexto de los requisitos funcionales.

32.7. Análisis arquitectural en el UP

Advertencia: Análisis arquitectural en cascada

Con frecuencia, los métodos y libros sobre el análisis arquitectural implícitamente fomentan extensas decisiones de diseño sobre la arquitectura siguiendo un estilo en cascada antes de la implementación. En el desarrollo iterativo y el UP, aplique estas ideas en el contexto de pequeñas etapas, retroalimentación y adaptación, en lugar de pretender resolver completamente la arquitectura antes de programar. Aborde la implementación de las soluciones más arriesgadas o más difíciles en las primeras iteraciones, y ajuste las soluciones sobre la arquitectura en base a la retroalimentación y al conocimiento que adquiere.

Información sobre la arquitectura en los artefactos del UP

- Los factores de la arquitectura (por ejemplo, en una tabla de factores) se recogen en la Especificación Complementaria.
- Las decisiones sobre la arquitectura se recogen en el SAD. Esto incluye los memorándums técnicos y las descripciones de las vistas de la arquitectura.

El SAD y sus vistas de la arquitectura

Además de los diagramas UML de paquetes, clases e interacciones, el SAD es otro artefacto clave del Modelo de Diseño del UP. Éste describe las grandes ideas de la arquitectura, incluyendo las decisiones sobre el análisis arquitectural. Prácticamente, es una *ayuda de aprendizaje* para los desarrolladores que necesitan entender las ideas esenciales del sistema.

Cuando alguien se une al equipo, un jefe de proyecto puede decir, “¡Bienvenido al proyecto NuevaEra! Por favor, vaya al sitio web del proyecto y lea el SAD de 10 páginas para obtener una visión de las ideas más importantes”. Durante una versión posterior, cuando trabaja en el sistema gente nueva, el SAD constituye una ayuda clave para el aprendizaje.

Por tanto, se debe escribir teniendo en mente quien lo leerá y el objetivo: ¿qué necesito decir (y representar en UML) que rápidamente ayudará a alguien a entender las ideas principales del sistema?

La esencia del SAD es un resumen de las decisiones sobre la arquitectura (como con los memorándums técnicos) y las vistas de la arquitectura del UP.

Vistas de la arquitectura en el SAD

Tener una arquitectura es una cosa, describirla es algo más.

En [Kruchten95], se fomenta la influyente idea de describir una arquitectura con múltiples vistas. La idea esencial de una **vista de la arquitectura** es ésta:

Vista de la arquitectura
Una vista de la arquitectura del sistema desde una perspectiva dada; se centra sobre todo en la estructura y modularidad de los componentes fundamentales y en los principales flujos de control [RUP].
Un aspecto importante de la vista que se obvia en esta definición del RUP es la <i>motivación</i> . Es decir, una vista de la arquitectura debería explicar por qué la arquitectura es como es.
Una vista de la arquitectura es una ventana sobre el sistema desde una perspectiva particular que destaca la información relevante o ideas claves, e ignora el resto.

Una vista de la arquitectura es una herramienta de comunicación, enseñanza o de reflexión; se representa con texto y diagramas UML.

En el UP, se sugieren seis vistas de la arquitectura (se permiten más, como una vista de seguridad)³. Todas son opcionales, pero se recomienda documentar por lo menos las vistas lógica, de proceso, de caso de uso, y de despliegue. Las seis vistas son:

1. Lógica

- Organización conceptual del software en función de las capas, subsistemas, paquetes, frameworks, clases e interfaces más importantes. También resume la funcionalidad de los elementos del software importantes, como cada subsistema.
- Muestra los escenarios de realización de casos de uso (como diagramas de interacción) destacados que ilustran los aspectos claves del sistema.
- Una vista sobre el Modelo de Diseño del UP, visualizada con diagramas de paquetes, clases e interacción de UML.

2. Proceso

- Procesos e hilos de ejecución. Sus responsabilidades, colaboraciones y la asignación a ellos de los elementos lógicos (capas, subsistemas, clases,...).
- Una vista sobre el Modelo de Diseño del UP, visualizada con diagramas de clases e interacción de UML, utilizando la notación UML para procesos e hilos.

³ Las primeras versiones del UP describían las “4+1” vistas como se definen en [Kruchten95], que evolucionaron a las seis vistas.

3. Despliegue

- Despliegue físico de los procesos y componentes sobre los nodos de proceso, y la configuración de la red física entre los nodos.
- Una vista sobre el Modelo de Despliegue del UP, visualizada con los diagramas de despliegue de UML. Normalmente, la “vista” es simplemente el modelo completo en lugar de un subconjunto, ya que todo es relevante. Véase el Capítulo 38 para conocer la notación para el diagrama de despliegue de UML.

4. Datos

- Vista global del esquema de datos persistentes, la correspondencia del esquema de objetos a datos persistentes (normalmente en una base de datos relacional), el mecanismo de correspondencia de objetos a una base de datos, procedimientos almacenados en la base de datos y disparadores (*triggers*).
- Una vista sobre el Modelo de Datos del UP, visualizada con diagramas de clases de UML que se utilizan para describir un modelo de datos.

5. Casos de uso

- Resumen de los casos de uso más significativos para la arquitectura y sus requisitos no funcionales. Es decir, aquellos casos de uso que, mediante su implementación, cubren una parte significativa de la arquitectura o que influyen en muchos elementos de la arquitectura. Por ejemplo, el caso de uso *Procesar Venta*, cuando se implementa completamente, tiene estas cualidades.
- Una vista sobre el Modelo de Casos de Uso del UP, expresada textualmente y visualizada con los diagramas de casos de uso de UML.

6. Implementación

- En primer lugar, una definición del Modelo de Implementación: a diferencia de otros modelos del UP, que son texto y diagramas, este “modelo” es el código fuente real, ejecutables, etcétera. Tiene dos partes: 1) entregables, y 2) cosas que crean a los entregables (como código fuente y gráficos). El Modelo de Implementación está formado por todas estas cosas, incluyendo las páginas web, DLLs, ejecutables, código fuente, etcétera, y su organización —como el código fuente en los paquetes de Java y bytecodes organizados en ficheros JAR—.
- La vista de implementación es una descripción resumida de la organización relevante de los entregables y de las cosas que crean a los entregables (como el código fuente).
- Una vista sobre el Modelo de Implementación del UP, expresada textualmente y visualizada con los diagramas de paquetes y componentes de UML.

Por ejemplo, los diagramas de paquetes y de interacción de NuevaEra que se presentaron en el Capítulo 30 sobre la arquitectura en capas y lógica, muestran las grandes ideas de la estructura lógica de la arquitectura del software. En el SAD, el arquitecto creará una sección denominada *Vista Lógica*, insertará estos diagramas UML, y añadirá

algunos comentarios escritos acerca de para qué es cada paquete y capa, y el motivo que hay detrás del diseño lógico. De igual modo con las vistas de proceso y despliegue.

Una idea clave de las vistas de la arquitectura —que concretamente son texto y diagramas— es que *no* describen *todo* el sistema desde alguna perspectiva, sino sólo ideas destacadas desde esa perspectiva. Una vista es, si se quiere, la descripción “en un minuto de ascensor”: ¿Cuáles son las cosas más importantes que dirías en un minuto en un ascensor a un compañero sobre esta perspectiva?

Podrían crearse vistas de la arquitectura:

- después de que se construya el sistema, como resumen o ayuda de aprendizaje para futuros desarrolladores
- al final de ciertos hitos de la iteración (como al final de la iteración) para que sirva de ayuda para el aprendizaje para el equipo de desarrollo actual, y nuevos miembros
- de manera especulativa, durante las primeras iteraciones, como ayuda en el trabajo de diseño creativo, reconociendo que la vista original cambiará cuando prosiga el diseño y la implementación.

Estructura de ejemplo de un SAD

Documento de la Arquitectura del Software

Representación de la arquitectura

(Resumen del modo en el que se describirá la arquitectura en este documento, como utilizando memorándums técnicos y las vistas de la arquitectura. Esto es útil para alguien que no esté familiarizado con la idea de los memorándums técnicos o las vistas. Nótese que no son necesarias todas las vistas.)

Factores y decisiones de la arquitectura

(Referencia a la Especificación Complementaria para ver la Tabla de Factores. También, el conjunto de memorándums técnicos que resume las decisiones.)

Vista Lógica

(Los diagramas de paquetes de UML y los diagramas de clases de los elementos importantes. Comentarios sobre la estructura a gran escala y la funcionalidad de los componentes principales.)

Vista Proceso

(Diagramas de interacción y de clases de UML que ilustran los procesos e hilos de ejecución del sistema. Agruparlos de acuerdo a los hilos y procesos que interaccionan. Comentarios sobre el modo en que funciona la comunicación entre los procesos (ej. mediante RMI de Java).

Vista de Casos de Uso

(Resumen breve de los casos de uso más significativos desde el punto de vista de la arquitectura. Diagramas de interacción UML para algunas de las realizaciones de los casos de uso significativos para la arquitectura, o escenarios, con comentarios en los diagramas explicando cómo ilustran los elementos importantes de la arquitectura.)

Vista de Despliegue

(Los diagramas de despliegue de UML que muestra los nodos y la asignación de los procesos y componentes. Comentarios sobre la red.)

Fases

Inicio: Si no está claro si es técnicamente posible satisfacer los requisitos significativos de la arquitectura, el equipo debe implementar una **prueba de concepto de la arquitectura** (PDC) para determinar la viabilidad. En el UP, su creación y evaluación se denomina **Síntesis de la Arquitectura**. Esto es distinto de los anteriores experimentos pequeños y simples de programación de PDC para cuestiones técnicas aisladas. Una PDC de la arquitectura cubre ligeramente *muchos* de los requisitos significativos para la arquitectura para evaluar su viabilidad *combinada*.

Elaboración: Un objetivo importante de esta fase es la implementación de los elementos centrales de la arquitectura de riesgo, de esta manera la mayoría del análisis arquitectural se completa durante la elaboración. Normalmente se espera que la mayor parte del contenido de la tabla de factores, memorándums técnicos y el SAD se pueda completar al final de la elaboración.

Transición: Aunque idealmente los factores y decisiones significativas para la arquitectura se resolvieron mucho antes de la transición, el SAD necesitará que se repase y posiblemente que se revise al final de esta fase para asegurar que describe de manera precisa el sistema de despliegue final.

Ciclos de evolución siguientes: Antes de diseñar nuevas versiones, es común volver a visitar los factores de la arquitectura y las decisiones. Por ejemplo, la decisión de la versión 1.0 de crear un único servicio remoto para el cálculo de impuestos, en lugar de un duplicado en cada nodo de PDV, podría haberse motivado por el coste (para evitar múltiples licencias). Pero quizás en el futuro, se reduce el coste de los calculadores de impuestos, y de esta manera, por razones de tolerancia a fallos o de rendimiento, la arquitectura cambia para utilizar varios calculadores de impuestos.

32.8. Lecturas adicionales

Existe un cuerpo creciente de patrones relacionados con la arquitectura, y consejos generales sobre la arquitectura del software. Sugerencias:

- *Pattern-Oriented Software Architecture*, ambos volúmenes.
- *Software Architecture in Practice* [BCK98].
- *Pattern Languages of Program Design*, todos los volúmenes. Cada volumen contiene una sección de patrones relacionados con la arquitectura.
- Artículos disponibles a través de la web sobre patrones de arquitectura (como las arquitecturas J2EE), en Sun, IBM, y otros sitios web.
- Artículos disponibles a través del web sobre arquitectura en el Instituto de Ingeniería del Software (SEI, *Software Engineering Institute*) de la Universidad Carnegie Mellon, que desde hace mucho tiempo es un centro de investigación sobre arquitectura (www.sei.cmu.edu).

DISEÑO DE MÁS REALIZACIONES DE CASOS DE USO CON OBJETOS Y PATRONES

*En dos ocasiones me han preguntado (miembros del Parlamento),
“Por favor, Mr. Babbage, si introduce en la máquina cifras incorrectas,
¿obtendrá la respuesta correcta?” No soy capaz de comprender exactamente el
tipo de confusión de ideas que podría provocar tal pregunta.*

Charles Babbage

Objetivos

- Aplicar los patrones GRASP y GoF en el diseño.
-

Introducción

Este capítulo estudia algunos diseños parciales para la iteración actual, manejando requisitos tales como el mantenimiento de los servicios ante fallos mediante servicios locales, la gestión de dispositivos de PDV y la autorización de pagos.

33.1. Mantenimiento de los servicios ante los fallos mediante servicios locales; rendimiento con el almacenamiento local

Uno de los requisitos de NuevaEra es un cierto grado de recuperación ante el fallo (*failover*) del servicio remoto, como una base de datos de productos no disponible (temporalmente).

El acceso a la información de los productos es el primer caso que se utiliza para estudiar la estrategia de recuperación y mantenimiento del servicio ante un fallo. Poste-

riormente, se explora el acceso al servicio de contabilidad, que tiene una solución ligeramente distinta.

Revisemos parte del memorándum técnico:

Memorándum Técnico

Asunto: Fiabilidad—Recuperación de fallos en los servicios remotos

Resumen de la solución: Ubicación transparente utilizando un servicio de búsqueda, mantenimiento del servicio ante los fallos pasando de remoto a local, y replicación parcial en el servicio local.

Factores

- Recuperación robusta de los fallos en los servicios remotos (ej. calculador de impuestos, inventario).
- Recuperación robusta de los fallos en la base de datos de productos (ej. descripciones y precios) remota.

Solución

Conseguir variaciones protegidas con respecto a la ubicación de los servicios utilizando un Adaptador creado en una *FactoriaDeServicios*. Donde sea posible, ofrecer implementaciones locales de servicios remotos, normalmente con comportamiento simplificado o restringido. Por ejemplo, el calculador de impuestos local utilizará porcentajes de impuestos constantes. La base de datos local con información de los productos será una pequeña caché de los productos más comunes. Se almacenarán las actualizaciones del inventario y se remitirán cuando se restablezca la conexión.

Véase también el memorándum técnico de *Adaptabilidad—Servicios de terceras partes* para conocer los aspectos de adaptabilidad de esta solución, porque las implementaciones de los servicios remotos variarán en cada instalación.

Para satisfacer los escenarios de calidad de reconexión con los servicios remotos, utilizar para los servicios objetos Proxy inteligentes, que en cada llamada a un servicio comprueban si se puede reactivar el servicio remoto y lo redireccionan a éste cuando es posible.

Motivación

¡Las tiendas realmente no quieren dejar de vender! Por tanto, si el PDV NuevaEra ofrece este nivel de fiabilidad y recuperación, será un producto muy atractivo, ya que ninguno de nuestros competidores proporciona esta capacidad.

Antes de solucionar los aspectos de recuperación y de mantenimiento del servicio ante los fallos, observe que tanto por razones de rendimiento como para mejorar la recuperación cuando falla la base de datos remota, el arquitecto (yo) ha recomendado una caché local (que persiste de manera fiable en el disco duro local en un simple fichero) de objetos *EspecificacionDelProducto*. Por tanto, siempre se debería buscar en la caché local por si hubiera un “acierto de caché” antes de intentar un acceso remoto.

Esto se puede conseguir de manera elegante con nuestro diseño existente del adaptador y la factoría:

1. La *FactoriaDeServicios* siempre devolverá un adaptador a un servicio de información de producto local.
2. El “adaptador” de los productos locales realmente no es un adaptador a otro componente. Implementará él mismo las responsabilidades del servicio local.

3. El valor inicial del servicio local es una referencia a un segundo adaptador del verdadero servicio de productos remoto.
4. Si el servicio local encuentra el dato en su caché, lo devuelve; en otro caso, remite la petición al adaptador del servicio externo.

Obsérvese que existen dos niveles de caché del lado del cliente:

1. El objeto *CatalogoDeProductos* en memoria mantendrá una colección en memoria (como un *HashMap* de Java) de algunos (por ejemplo, 1.000) objetos *EspecificacionDelProducto* que se han obtenido a través del servicio de información de productos. Se puede ajustar el tamaño de esta colección en función de la disponibilidad de memoria local.
2. El servicio de productos local mantendrá una caché persistente más grande (en el disco duro) que mantiene cierta cantidad de información de productos (como 1 o 100 MB de espacio en ficheros). De nuevo, se puede ajustar dependiendo de la configuración local. Esta caché persistente es importante para la tolerancia a fallos, de manera que incluso si la aplicación del PDV cae y se pierde la caché de memoria del objeto *CatalogoDeProductos*, permanece la caché persistente.

Este diseño no afecta al código existente —el nuevo objeto de servicio local se inserta sin afectar al diseño del objeto *CatalogoDeProductos* (que colabora con el servicio de productos)—.

Hasta el momento, no se han introducido nuevos patrones; se han utilizado un Adaptador y una Factoría.

La Figura 33.1 ilustra los tipos del diseño y la Figura 33.2 ilustra la inicialización.

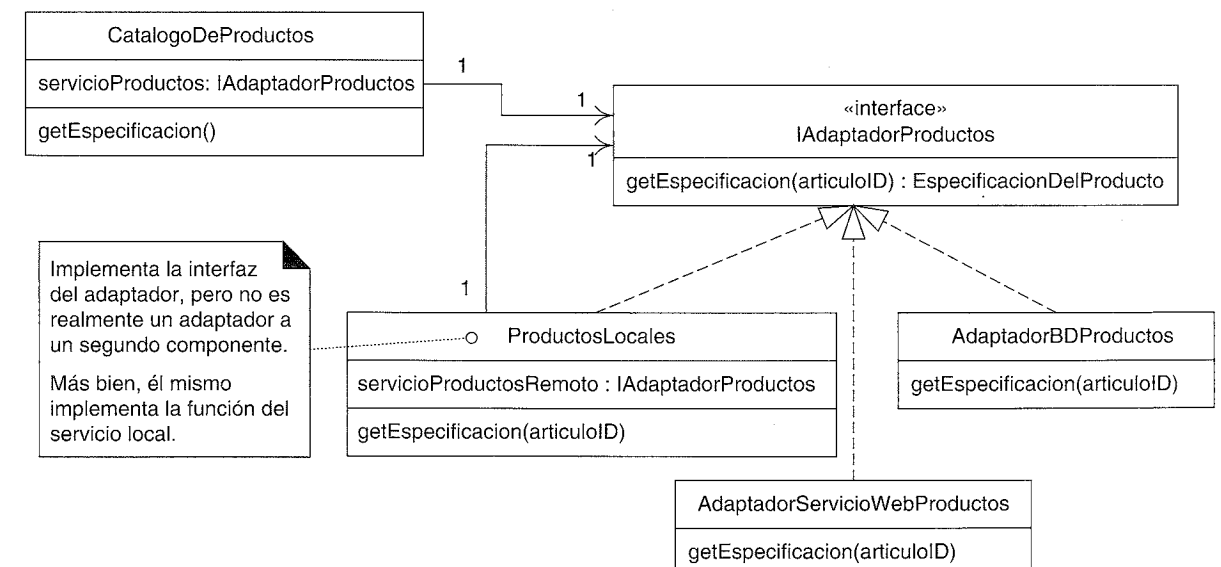


Figura 33.1. Adaptadores para la información de productos.

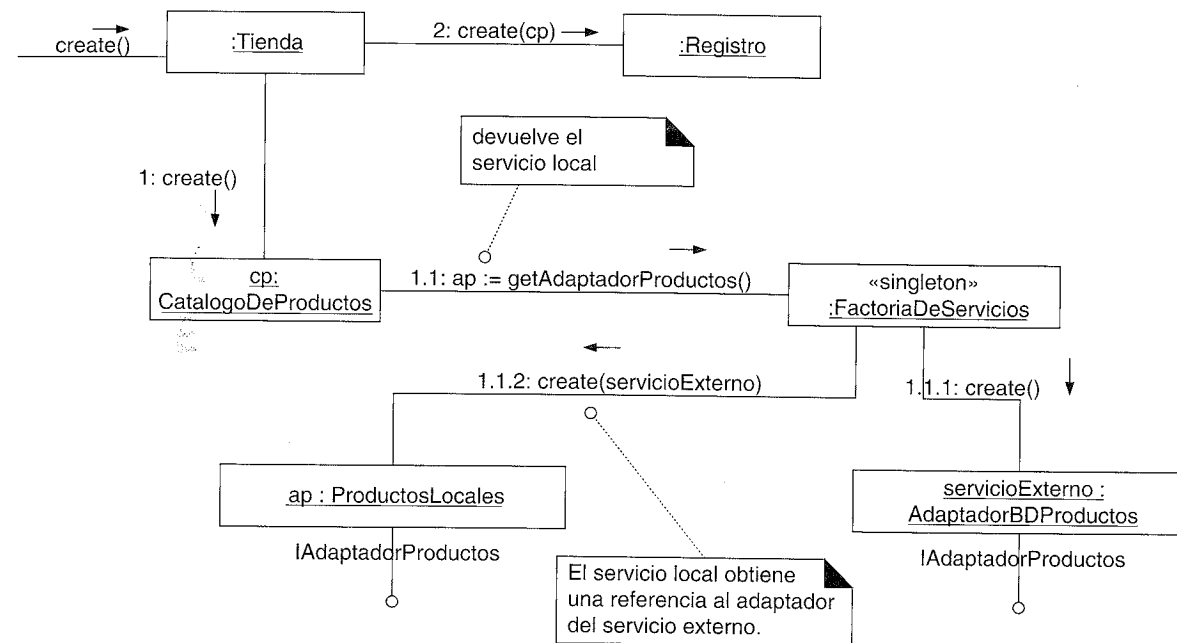


Figura 33.2. Inicialización del servicio de información de productos.

La Figura 33.3 muestra la colaboración inicial desde el catálogo al servicio de productos.

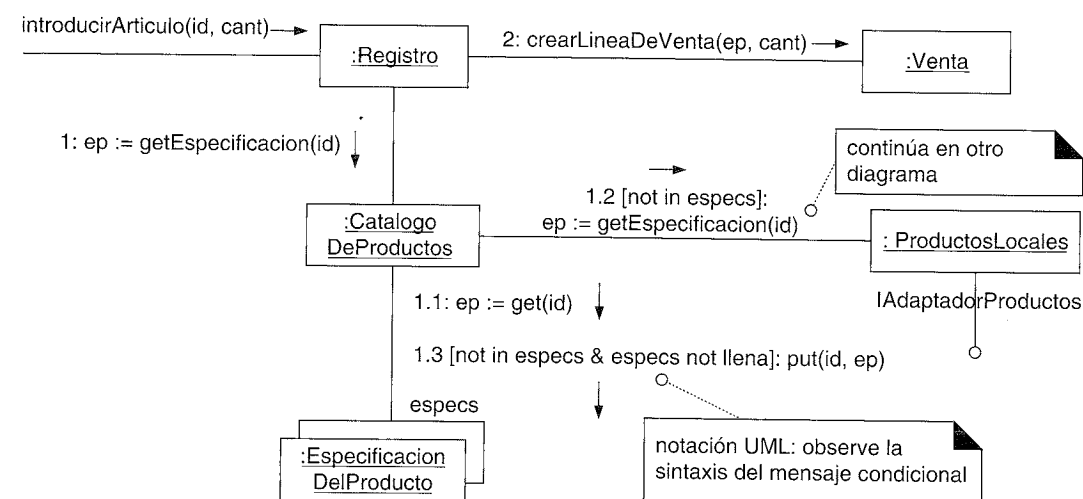


Figura 33.3. Comienzo de la colaboración con el servicio de productos.

Si el servicio de productos local no tiene el producto en su caché, colabora con el adaptador del servicio externo, como se muestra en la Figura 33.4. Nótese que el servicio de productos local almacena los objetos *EspecificacionDelProducto* como verdaderos objetos serializados.

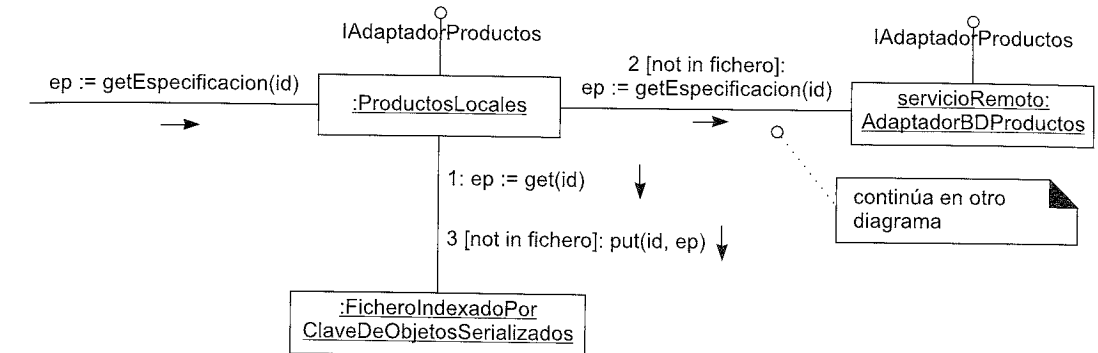


Figura 33.4. Continuación de la colaboración para obtener la información del producto.

Si se cambió el servicio externo de una base de datos a un nuevo servicio Web, sólo es necesario cambiar la configuración de la factoría del servicio remoto. Véase la Figura 33.5.

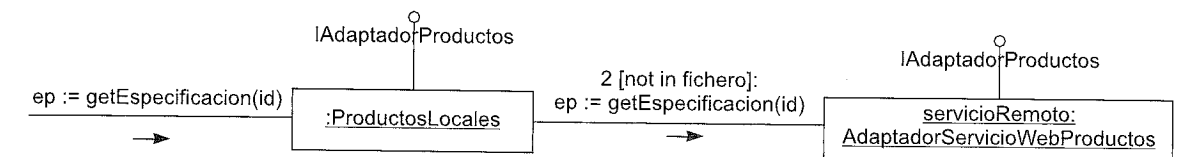


Figura 33.5. Los nuevos servicios externos no afectan al diseño.

Continuando con el caso de la colaboración con el *AdaptadorBDProductos*, interactuará con un subsistema de persistencia que tiene que establecer la correspondencia objeto-relacional (*mapping O-R*) (ver Figura 33.6).

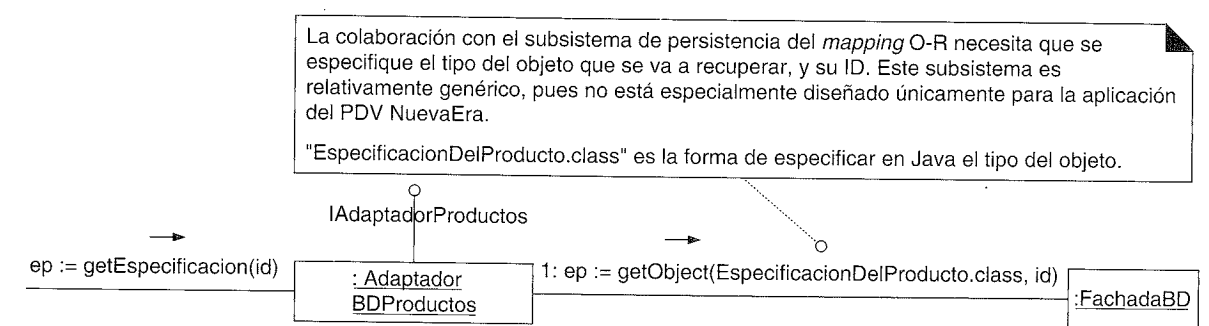


Figura 33.6. Colaboración con el subsistema de persistencia.

Estrategias de almacenamiento en caché

Considere las alternativas para cargar en memoria la caché del *CatalogoDeProductos* y la caché basada en ficheros de *ProductosLocales*: un enfoque es la inicialización perezosa, en el que las cachés se cargan lentamente cuando se recupera la información del producto externo; otro enfoque es la inicialización impaciente, en el que las cachés se cargan du-

rante el caso de uso *PonerEnMarcha*. Si el diseñador no está seguro del enfoque a utilizar y quiere experimentar con las alternativas, una familia de diferentes objetos *Estrategia-Cache* basada en el patrón Estrategia puede solucionar el problema de manera elegante.

Caché antigua

Puesto que los precios de los productos cambian rápidamente, y quizás al antojo del encargado de la tienda, almacenar en la caché el precio de los productos crea un problema —la caché contiene datos antiguos—; esto es siempre una preocupación cuando se replican los datos. Una solución es añadir una operación al servicio remoto que responda con los cambios actuales a lo largo de un día; el objeto *ProductosLocales* la consulta cada n minutos y actualiza su caché.

Hilos en UML

Si el objeto *ProductosLocales* va a solucionar el problema de la cache con datos antiguos cada n minutos, un enfoque de diseño es hacerlo un **objeto activo** que posea un hilo de control. El hilo dormirá durante n minutos, se despertará, el objeto obtendrá los datos, y

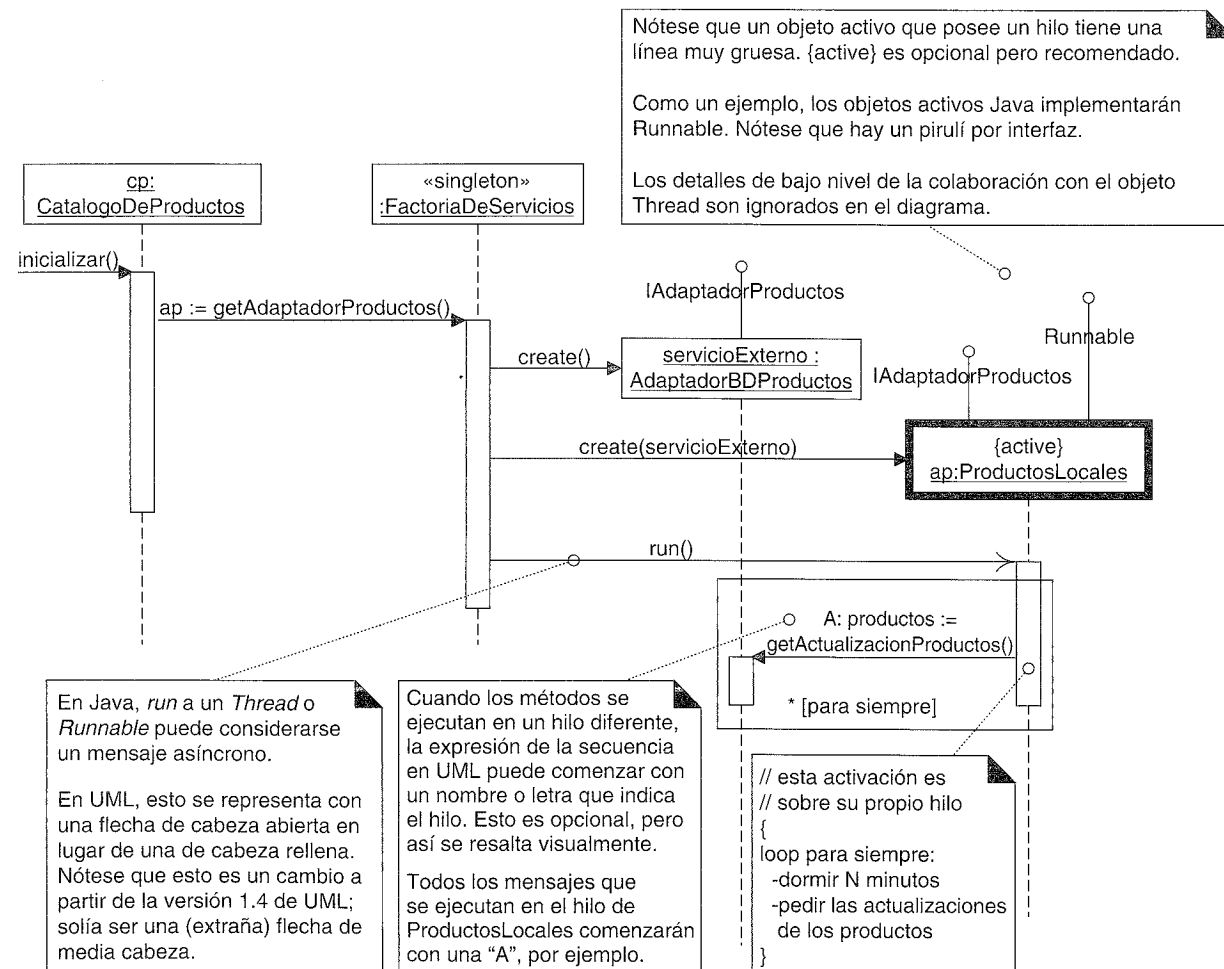


Figura 33.7. Hilos y mensajes asíncronos en UML.

el hilo volverá a dormirse. UML proporciona la notación para representar hilos y las llamadas asíncronas, como se muestra en la Figura 33.7.

En un diagrama de interacción, una instancia de un objeto activo podría etiquetarse con la propiedad {active}. En un diagrama de clases, una clase de objetos activos (una **clase activa**) que posee su propio hilo se puede estereotipar con «thread». Véase la Figura 33.8.

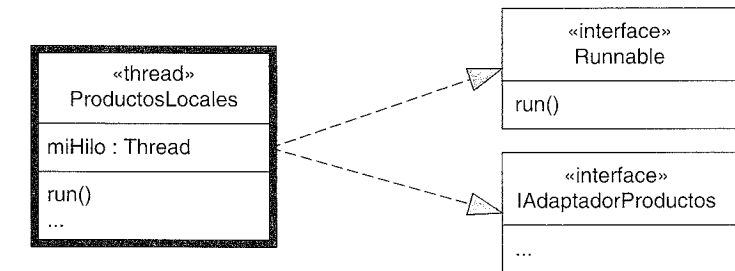


Figura 33.8. Notación para las clases activas.

33.2. Manejo de fallos

El diseño anterior proporciona una solución para el almacenamiento en la caché del lado del cliente de los objetos *EspecificacionDelProducto* en un fichero persistente, para mejorar el rendimiento, y también para proporcionar al menos una solución parcial a la que recurrir si no se puede acceder al servicio de productos externo. Quizás se almacenen en la caché 10.000 productos en el fichero local, que podría satisfacer la mayoría de las peticiones de información de los productos incluso cuando falla el servicio externo.

¿Qué hacemos en el caso de que no se encuentre en la caché y falla el acceso al servicio de productos externo? Suponga que las personas involucradas nos piden que creamos una solución que indique al cajero que introduzca manualmente el precio y la descripción, o cancele la entrada de la línea de venta.

Éste es un ejemplo de una condición de error o fallo, y se utilizará como contexto para describir algunos patrones generales que se ocupan del manejo de los fallos y excepciones. El manejo de excepciones y errores es un tema amplio, y esta introducción únicamente se centrará en algunos patrones específicos para el contexto del caso de estudio. En primer lugar, algo de terminología:

- **Defecto:** el origen último o causa del mal comportamiento.
 - El programador escribe mal el nombre de la base de datos.
- **Error:** una manifestación del defecto en el sistema en ejecución. Los errores se detectan (o no).
 - Cuando estamos invocando al servicio de nombres para obtener una referencia a la base de datos (con el nombre mal escrito), se señala un error.
- **Fallo:** la denegación de un servicio causada por un error.
 - El subsistema de Productos (y el PDV NuevaEra) falla al proporcionar un servicio de información de productos.

Lanzamiento de excepciones

Un enfoque directo para señalar el fallo que se está produciendo es lanzar una excepción.

Las excepciones son apropiadas especialmente cuando estamos tratando con fallos de los recursos (disco, memoria, acceso a la red o a una base de datos y otros servicios externos).

Se lanzará una excepción desde el interior del subsistema de persistencia (realmente, es muy probable que a partir de algo como una implementación JDBC de Java), donde se detecte en primer lugar un fallo al utilizar la base de datos de productos externa. La excepción desenrollará la pila de llamadas hacia atrás hasta un punto apropiado donde se pueda manejar¹.

Suponga que la excepción original (utilizando Java como ejemplo) es *java.sql.SQLException*. ¿Debería propagarse una *SQLException* todo el camino hacia arriba hasta llegar a la capa de presentación? No. No está en el nivel correcto de abstracción. Esto nos lleva a un patrón común para el manejo de excepciones:

Patrón: Convertir Excepciones [Brown01]

En un subsistema, evite la emisión de excepciones de bajo nivel que proceden de los subsistemas inferiores o servicios. Más bien, convierta una excepción de bajo nivel en una que sea significativa para el nivel del subsistema. La excepción de más alto nivel normalmente encapsula la excepción de un nivel inferior, y añade información, para hacer que la excepción sea más significativa de acuerdo con el contexto de los niveles más altos.

Esto es una guía, no una regla absoluta.

Aquí "Excepción" se utiliza en el sentido vernáculo de algo que se puede lanzar; en Java, el equivalente es un *Throwable*.

También conocido como Abstracción de Excepción [Renzel97].

Por ejemplo, el subsistema de persistencia captura una *SQLException* concreta, y (asumiendo que no puede manejarla²) lanza una nueva *BDNoDisponibleException*, que contiene la *SQLException*. Nótese que el *AdaptadorBDProductos* es como una fachada sobre un subsistema lógico para la información de los productos. Por tanto, el *AdaptadorBDProductos* de más alto nivel (como representante de un subsistema lógico) captura la *BDNoDisponibleException* de un nivel inferior y (asumiendo que no puede manejarla) lanza una nueva *InfoProductoNoDisponibleException*, que encapsula la *BDNoDisponibleException*.

¹ No se trata el manejo de excepciones comprobadas frente a las no comprobadas, ya que no es soportado en todos los lenguajes OO conocidos—C++, C# y Smalltalk, por ejemplo.

² Resolver una excepción cerca del nivel donde se produce es un objetivo loable pero difícil, puesto que los requisitos acerca de la manera de manejar un error con frecuencia son específicos de la aplicación.

Considere los nombres de estas excepciones: ¿por qué decir *BDNoDisponibleException* en lugar de *SubsistemaPersistenciaException*? Existe un patrón para esto:

Patrón: Nombre El Problema No El Lanzador [Grosso00]

¿Cómo llamar a una excepción? Asigne un nombre que describa por qué se va a lanzar la excepción, no quién la lanza. La ventaja es que facilita al programador la comprensión del problema, y resalta la similitud fundamental de muchas clases de excepciones (de una manera que no ocurre nombrando al que la lanza).

Excepciones en UML

Éste es un momento adecuado para introducir la notación de UML para el lanzamiento³ y captura de excepciones.

Notación UML: UML tiene una sintaxis "por defecto" para las operaciones. Pero no incluye una solución oficial para mostrar las excepciones que lanza una operación. Existen al menos tres soluciones:

1. UML permite que se utilice para las operaciones la sintaxis de cualquier otro lenguaje, como Java. Además, algunas herramientas CASE UML permiten mostrar por pantalla las operaciones explícitamente en la sintaxis de Java. De este modo,

Object get(Clave, Class) throws BDNoDisponibleException, FatalException

2. La sintaxis por defecto permite que el último elemento sea un "string de propiedades" (*property string*). Esto es una lista arbitraria de pares propiedad+valor, como {autor=Craig, niños=(Hannah, Haley),...}. Así,

put(Object, id){ throws=(BDNoDisponibleException, FatalException)}

3. Algunas herramientas CASE UML le permiten a uno especificar (en un cuadro de diálogo especial) las excepciones que lanza una operación.

Las excepciones que se capturan se modelan como un tipo de operación que maneja una señal.

Las excepciones que se lanzan se pueden listar en otro compartimento etiquetado "excepciones"

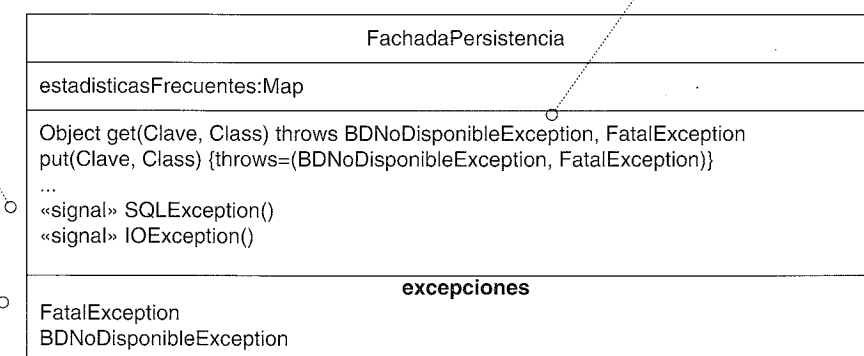


Figura 33.9. Excepciones capturadas y lanzadas por una clase.

³ Oficialmente en UML, uno *envía* una excepción, pero *lanza* es un término adecuado y más familiar.

Dos preguntas típicas sobre la notación UML son:

1. En un diagrama de clases, ¿cómo representamos las excepciones que captura y lanza una clase?
2. En un diagrama de interacción, ¿cómo representamos el lanzamiento de una excepción?

La Figura 33.9 presenta la notación para un diagrama de clases.

En UML, una *Exception* es una especialización de una *Signal*, que es la especificación de una comunicación asíncrona entre objetos. Esto significa que en un diagrama de interacción, las excepciones se representan como **mensajes asíncronos**⁴.

La Figura 33.10 muestra la notación, utilizando como ejemplo la descripción anterior de *SQLException* que se transforma en una *BDNoDisponibleException*.

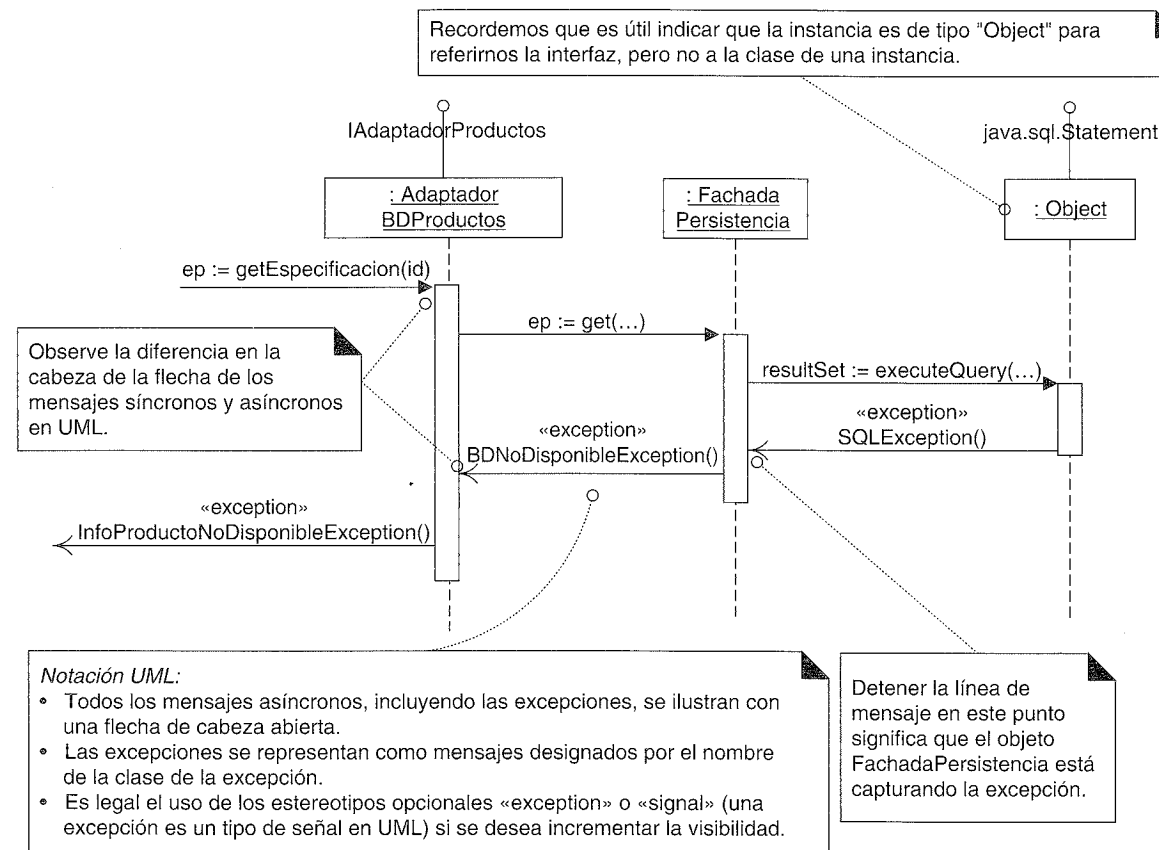


Figura 33.10. Excepciones en un diagrama de interacción.

En resumen, existe notación UML para representar las excepciones. Sin embargo, pocas veces se utiliza.

⁴ Nótese que a partir de la versión 1.4 de UML, cambió la notación para los mensajes asíncronos de una flecha de media cabeza a una de cabeza abierta.

No es que recomendemos evitar las consideraciones iniciales sobre el manejo de excepciones. Más bien lo contrario: al nivel de arquitectura es necesario establecer pronto los patrones básicos, políticas y colaboraciones para el manejo de las excepciones, puesto que es difícil insertar el manejo de las excepciones como una ocurrencia tardía. Sin embargo, muchos desarrolladores consideran que el diseño a bajo nivel del manejo de excepciones concretas se decide de manera más apropiada durante la programación o mediante descripciones de diseño menos detalladas, en lugar de mediante diagramas UML detallados.

Manejo de errores

Se ha considerado una parte del diseño: el lanzamiento de excepciones, en cuanto a la conversión, asignación de nombres y la manera de representarlas. La otra parte es el manejo de una excepción.

Dos patrones que se pueden aplicar en éste y en la mayoría de los casos son:

Patrón: Registro Centralizado de los Errores [Renzel97]

Utilice un objeto central de registro de errores con acceso mediante el patrón Singleton y notifíquelo todas las excepciones. Si se trata de un sistema distribuido, cada singleton local colaborará con un registro de error central. Beneficios:

- Informes consistentes.
- Definición flexible de la información de salida y el formato.

También conocido como Registrador de Diagnóstico (*Diagnostic Logger*) [Harrison98].

Es un patrón sencillo. El segundo es:

Patrón: Diálogo de Error [Renzel97]

Para notificar los errores a los usuarios utilice un objeto que no pertenezca a la interfaz de usuario, independiente de la aplicación y al que se accede siguiendo el patrón Singleton. Actúa como envoltorio (*wrapper*) de uno o más objetos de "diálogo" de la UI (tales como diálogo modal de la UI, consola de texto, emitir un sonido, o generador de voz) y delega la notificación del error a los objetos del UI. De esta manera, la salida podría ir tanto a un diálogo de la GUI como a un generador de voz. También informará de la excepción al registro centralizado de errores. Una Factoría que lee los parámetros del sistema creará los objetos apropiados de la UI. Beneficios:

- Variaciones Protegidas con respecto a los cambios en el mecanismo de salida.
- Estilo consistente de los informes de error; por ejemplo, todas las ventanas de la GUI pueden invocar a este singleton para mostrar el diálogo de error.
- Control centralizado de la estrategia común para la notificación de los errores.
- Ganancia de rendimiento pequeña; si se utiliza un recurso "caro" como un diálogo de la GUI, es fácil ocultarlo y almacenarlo en caché para volver a utilizarlo después, en lugar de volver a crear un diálogo por cada error.

¿Debería un objeto de la UI (por ejemplo, *FrameProcesarVenta*) manejar un error capturando la excepción e informando al usuario? Para aplicaciones con pocas ventanas,

y caminos de navegación entre ventanas simples y estables, este diseño directo es bueno. Esto se cumple actualmente para la aplicación NuevaEra.

Tenga presente sin embargo, que esto sitúa algo de la “lógica de la aplicación” relacionada con el manejo de los errores en la capa de presentación (GUI). El manejo de errores está relacionado con la notificación a los usuarios, luego es lógica, pero es una tendencia que hay que tener en cuenta. No es inherentemente un problema de UI simples con pocas oportunidades de reemplazar la UI, sino que es un punto de fragilidad. Por ejemplo, suponga que un equipo quiere sustituir la UI Swing de Java por el framework de GUI Java MicroView de IBM para ordenadores portátiles. Ahora existe algo de la lógica de la aplicación en la versión Swing que se tiene que identificar y replicar en la versión MicroView. Hasta cierto punto, esto es inevitable en las sustituciones de UI; pero se agravará cuanto más lógica de la aplicación se migre hacia arriba. En general, cuantas más responsabilidades de la lógica de la aplicación que no tiene que ver con el UI se migren a la capa de presentación, mayor es la probabilidad de que se produzcan quebraderos de cabeza relacionados con el diseño o el mantenimiento.

Para sistemas con muchas ventanas y caminos de navegación complejos (quizás incluso cambiantes), existen otras soluciones. Por ejemplo, se puede insertar una capa de aplicación de uno o más controladores entre las capas de presentación y del dominio.

Además, se puede insertar un objeto “mediador en el manejo de la vista” [GHJV95, BMRSS96] que es responsable de mantener una referencia a todas las ventanas abiertas, y conocer las transiciones entre ventanas, dado algún evento E1 (como un error).

Este mediador es de manera abstracta una máquina de estados que encapsula los estados (la ventana que se muestra) y las transiciones entre los estados, en base a los eventos. Podría leer el modelo de transición de estados (ventanas) de un fichero externo, de manera que los caminos de navegación pueden ser dirigidos por los datos (no es necesario ningún cambio en el código fuente). También puede cerrar todas las ventanas de la aplicación, o disponerlas de algún modo o minimizarlas, puesto que tiene una referencia a todas las ventanas.

En este diseño, podría diseñarse un controlador de la capa de aplicación con una referencia a este mediador para el manejo de la vista (por ello, el controlador de la aplicación se acopla “hacia arriba” con la capa de presentación). El controlador de la aplicación podría capturar la excepción y colaborar con el mediador para el manejo de la vista para producir la notificación (basado en el patrón Diálogo de Error). De esta manera, el controlador de la aplicación está involucrado con flujos de trabajo de la aplicación, y se deja fuera de las ventanas algo del manejo lógico de errores.

El diseño detallado del control de la UI y de la navegación quedan fuera del alcance de esta introducción, y el simple diseño de la ventana que captura la excepción será suficiente. Un diseño que utiliza un Diálogo de Error se muestra en la Figura 33.11.

33.3. Mantenimiento de los servicios ante los fallos mediante un Proxy (GoF)

El mantenimiento de los servicios ante los fallos (*failover*) mediante un servicio local para la información de los productos se consiguió insertando el servicio local delante del servicio externo; el servicio local siempre se prueba primero. Sin embargo, este diseño

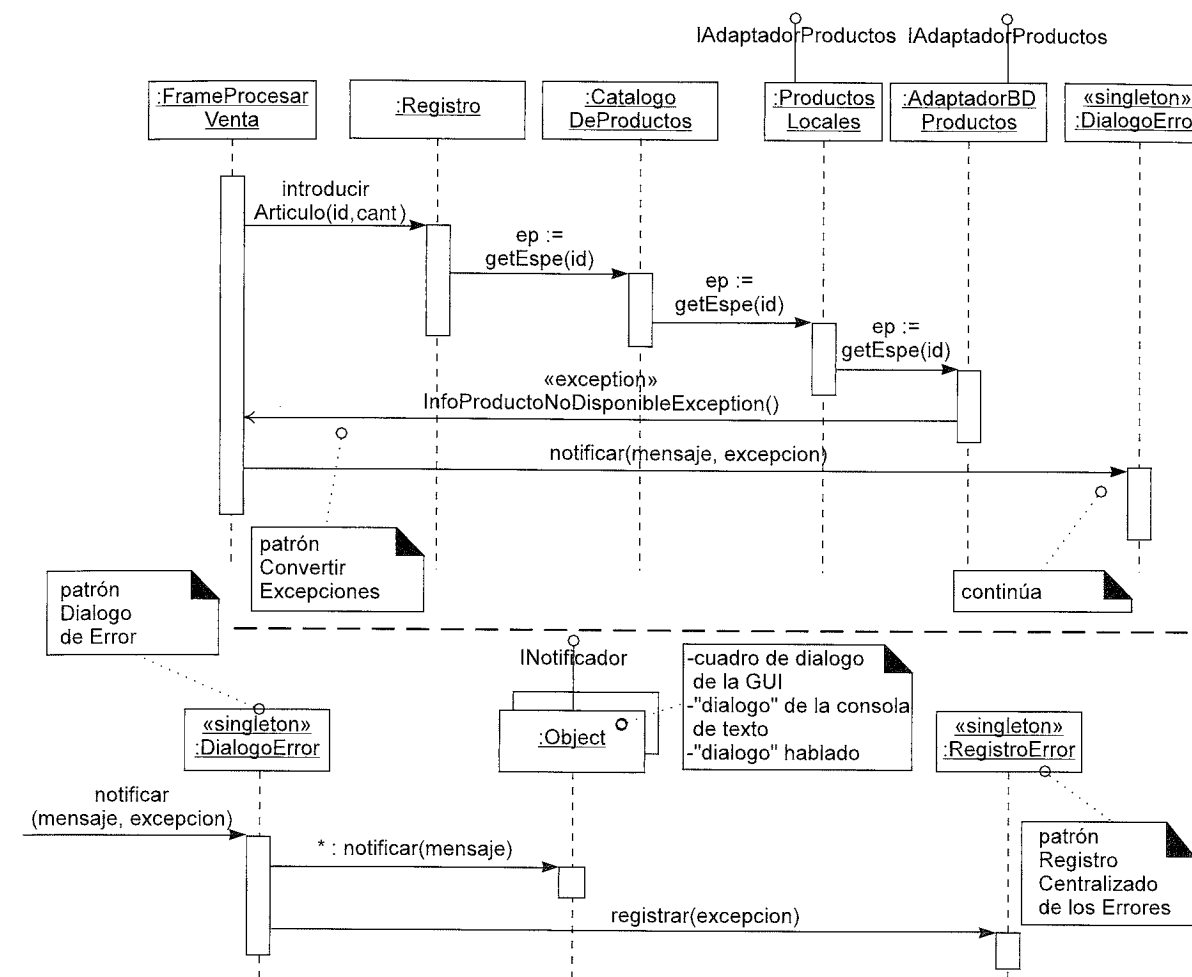


Figura 33.11. Manejo de la excepción.

no es apropiado para todos los servicios; algunas veces se debería probar primero el servicio externo, y la versión local en segundo lugar. Por ejemplo, considere el registro de las ventas en el servicio de contabilidad. El negocio quiere que se registren lo antes posible, para controlar en tiempo real la actividad de la tienda y el registro.

En este caso, otro patrón GoF puede solucionar el problema: Proxy⁵. Proxy es un patrón sencillo, y se utiliza ampliamente su variante **Proxy Remoto**. Por ejemplo, en RMI de Java y en CORBA, un objeto local del lado del cliente (denominado “*stub*”) se invoca para que acceda a los servicios de un objeto remoto. El stub del lado del cliente es un proxy local, o un representante o sustituto del objeto remoto.

Este ejemplo de uso del Proxy en NuevaEra no es la variante de Proxy Remoto, sino la variante **Proxy de Redirección** (*Redirection Proxy*) (también conocido como **Proxy de Mantenimiento de Servicio**, *Failover Proxy*).

⁵ N. del T. No se ha traducido por el mismo motivo que en el caso del patrón Singleton.

Independientemente de la variante, la estructura de un Proxy es siempre la misma; las variantes tienen que ver con lo que hace el proxy una vez que se le ha llamado.

Un proxy es simplemente un objeto que implementa la misma interfaz que el objeto al que se quiere acceder en realidad, manteniendo una referencia al sujeto real, y suele controlar el acceso a él. La Figura 33.12 muestra la estructura general.

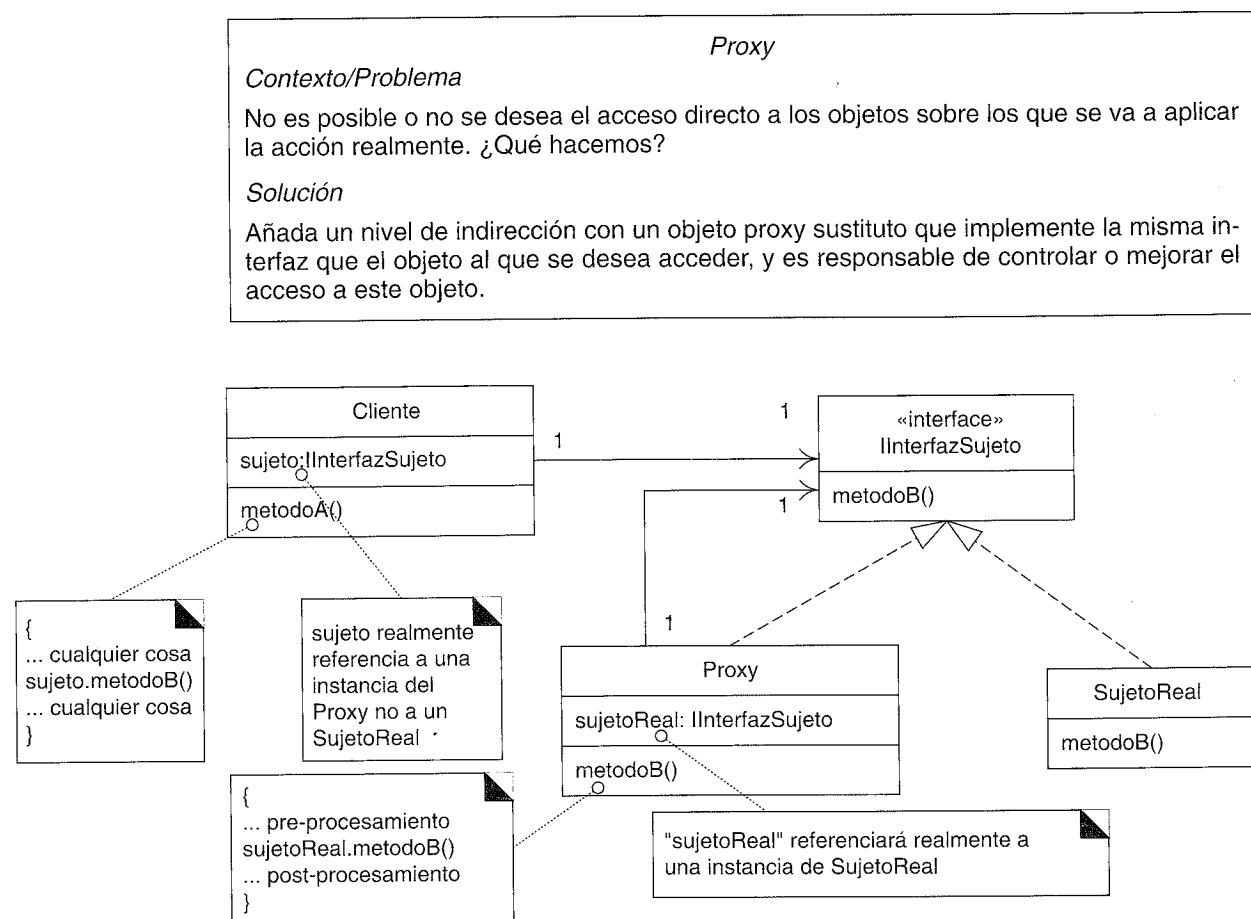


Figura 33.12. Estructura general del patrón Proxy.

Aplicado al caso de estudio de NuevaEra para el acceso al servicio de contabilidad externo, se utiliza un proxy de redirección como sigue:

1. Enviar el mensaje *anotarVenta* al proxy de redirección, tratándolo como si se pensase que es el servicio de contabilidad externo real.
2. Si falla el proxy de redirección al establecer el contacto con el servicio externo (mediante su adaptador), entonces redireccionar el mensaje *anotarVenta* a un servicio local, que almacena localmente las ventas para remitirlas al servicio de contabilidad, cuando esté disponible.

La Figura 33.13 ilustra un diagrama de clases de los elementos interesantes.

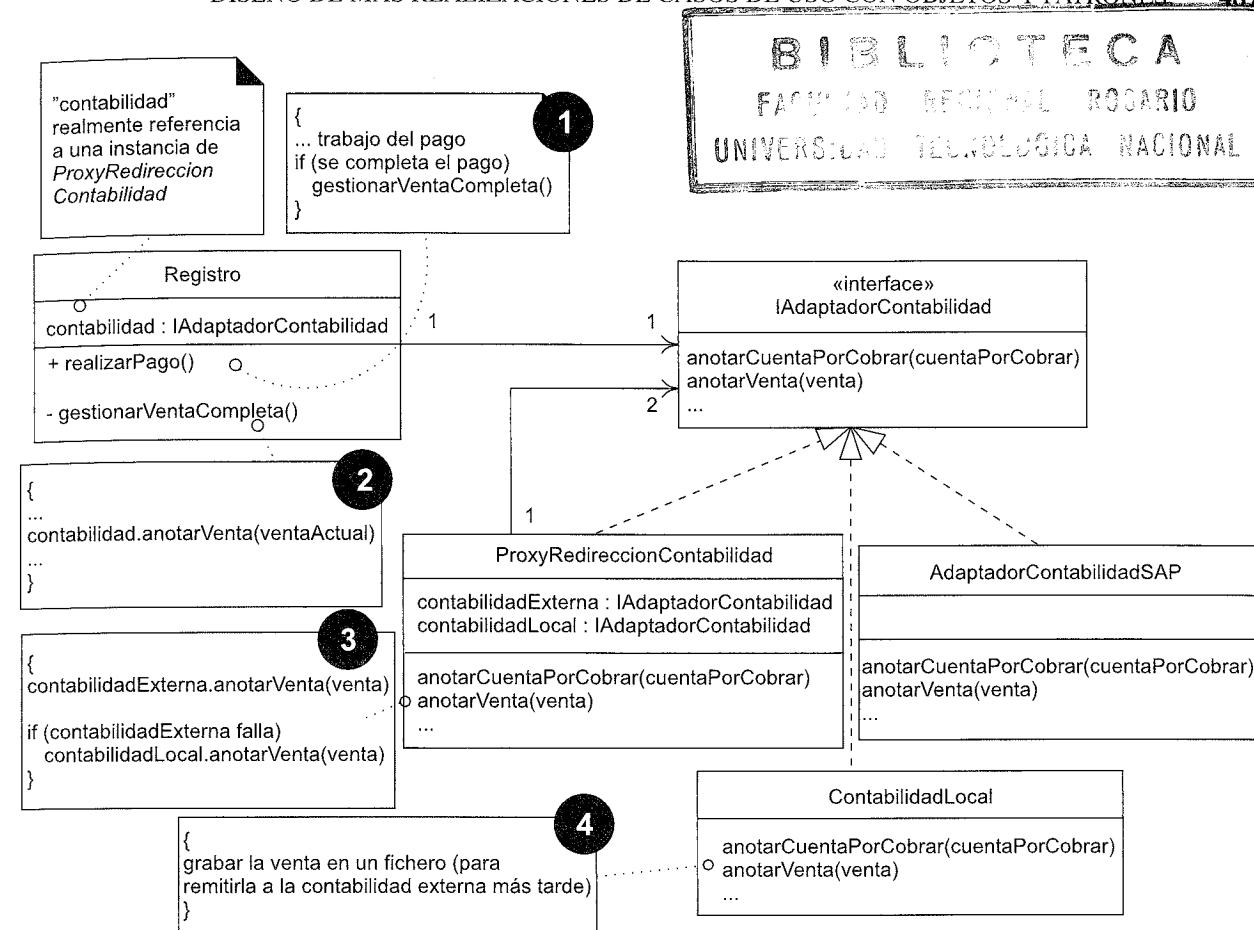


Figura 33.13. Uso en NuevaEra de un proxy de redirección.

Notación UML:

- Para evitar crear un diagrama de interacción que muestre el comportamiento dinámico, obsérvese cómo este diagrama estático utiliza la numeración para mostrar la secuencia de interacción. Normalmente es preferible un diagrama de interacción, pero se presenta este estilo para ilustrar un estilo alternativo.
- Obsérvese los marcadores de visibilidad pública y privada (+, -) junto a los métodos del *Registro*. Si no se indican, significa que no se especifica en lugar de asumir un valor por defecto público o privado. Sin embargo, de acuerdo con la práctica habitual, la mayoría de las personas que leen diagramas UML (y las herramientas CASE que generan código) interpretan la visibilidad no especificada como privada para los atributos y pública para los métodos. Sin embargo, en este diagrama, en particular quiero transmitir el hecho de que *realizarPago* es público, y en cambio, *gestionarVentaCompleta* es privado. El ruido visual y la sobrecarga de información son siempre una preocupación en la comunicación, luego, es conveniente hacer uso de las interpretaciones convencionales para mantener los diagramas simples.

Resumiendo, un proxy es un objeto externo que envuelve un objeto interno, y ambos implementan la misma interfaz. Un objeto cliente (como un *Registro*) no sabe que refe-

rencia a un proxy —se diseña de manera que piense que está colaborando con el sujeto real (por ejemplo, el *AdaptadorContabilidadSAP*)—. El Proxy intercepta las llamadas para mejorar el acceso al sujeto real, en este caso redireccionando la operación a un servicio local (*ContabilidadLocal*) si no está accesible el servicio externo.

33.4. Diseño para los requisitos no funcionales o de calidad

Antes de pasar a la siguiente sección, observe que el trabajo de diseño realizado hasta este punto del capítulo no está relacionado con la lógica del negocio, sino con los requisitos no funcionales o de calidad relacionados con la fiabilidad y recuperación.

Curiosamente —y es un punto clave en la arquitectura del software— es habitual que los diseños den forma antes a los temas, patrones y estructuras de la arquitectura del software de gran escala para resolver los requisitos no funcionales o de calidad, en lugar de a la lógica del negocio básica.

33.5. Acceso a los dispositivos físicos externos con adaptadores; comprar vs. construir

Otro requisito de esta iteración es interactuar con dispositivos físicos que componen un terminal de PDV, como abrir el cajón de caja, entregar el cambio del dispensador de monedas y capturar la firma de un dispositivo para las firmas digitales.

El PDV NuevaEra debe trabajar con una variedad de equipamiento de PDV, que incluye el que vende IBM, Epson, NCR, Fujitsu, etcétera.

Afortunadamente, el arquitecto de software ha investigado algo, y ha descubierto que ahora existe un estándar industrial, UnifiedPOS⁶ (www.nrf-arts.org), que define las interfaces orientadas a objetos estándar (en el sentido de UML) para todos los dispositivos comunes del PDV. Además, existe el JavaPOS (www.javapos.com) —la correspondencia en Java del UnifiedPOS—.

Por tanto, en el Documento de la Arquitectura del Software, el arquitecto añade un memorándum técnico para comunicar esta alternativa significativa para la arquitectura:

Memorándum Técnico
Asunto: Fiabilidad—Control de los dispositivos hardware del PDV

Resumen de la solución: Utilizar el software de Java de los fabricantes de dispositivos que se ajusta a las interfaces estándares de JavaPOS.

Factores

- Controlar correctamente los dispositivos
- Coste de comprar vs. construir y mantener

⁶ N. del T. POS es acrónimo de *Point-Of-Sale* (Punto-De-Venta), por tanto en esta sección aparece tanto POS como PDV.

Solución

UnifiedPOS (www.nrf-arts.org) define un modelo de interfaces UML que es un estándar industrial para los dispositivos de PDV. JavaPOS (www.javapos.com) es un estándar industrial que establece la correspondencia del UnifiedPOS a Java. Los fabricantes de dispositivos de PDV (ej. IBM, NCR) venden las implementaciones Java de estas interfaces que controlan sus dispositivos.

Comprarlos en lugar de construirlos.

Utilizar una Factoría que lea una propiedad del sistema para cargar un conjunto de clases de IBM o NCR (etc.) y devuelva instancias basadas en sus interfaces.

Motivación

Basado en una encuesta informal, creemos que funcionan bien, y los fabricantes tienen un proceso de actualización regular para mejorarlos. Es difícil conseguir la experiencia y otros recursos para escribirlos nosotros mismos.

Alternativas consideradas

Escribirlos nosotros mismos —difícil y arriesgado—.

La Figura 33.14 muestra algunas de las interfaces, que se han añadido como otro paquete de la capa del dominio en nuestro Modelo de Diseño.

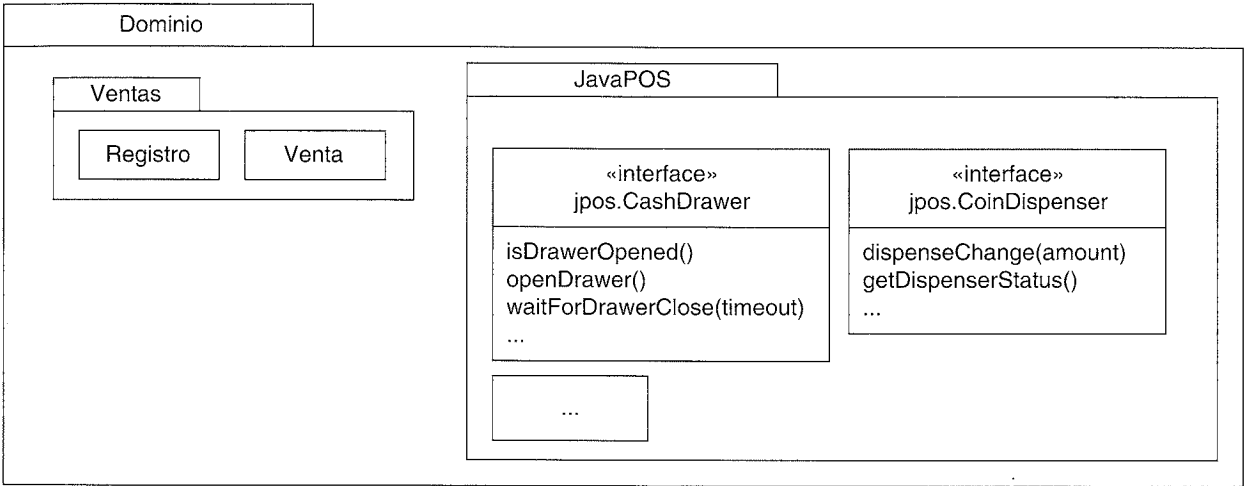


Figura 33.14. Interfaces JavaPOS estándares.

Asuma que los fabricantes importantes de los equipos de PDV ahora suministran las implementaciones JavaPOS. Por ejemplo, si compramos un terminal de PDV IBM con un cajón de caja, dispensador de monedas, etcétera, IBM también nos puede proporcionar las clases Java que implementan las interfaces JavaPOS, y que controlan los dispositivos físicos.

En consecuencia, esta parte de la arquitectura se resuelve comprando componentes software, en lugar de construyéndolos. Fomentar el uso de componentes existentes es una de las buenas prácticas del UP.

¿Cómo funcionan? A bajo nivel, un dispositivo físico tiene un controlador de dispositivo para las operaciones del sistema operativo subyacente. Una clase Java (por ejemplo, una que implemente *jpos.CashDrawer*) utiliza JNI (*Java Native Interface*) para hacer las llamadas a estos controladores de dispositivos.

Esas clases de Java adaptan los controladores de dispositivos de bajo nivel a las interfaces de JavaPOS, y así se pueden caracterizar como objetos Adaptador en el sentido del patrón GoF. También se les puede llamar objetos Proxy —objetos proxy locales que controlan o mejoran el acceso a los dispositivos físicos—.

No es raro que seamos capaces de clasificar un diseño en función de múltiples patrones.

33.6. Factoría Abstracta (GoF) para familias de objetos relacionados

Se comprarán a los fabricantes las implementaciones JavaPOS. Por ejemplo⁷:

```
//controladores IBM
com.ibm.pos.jpos.CashDrawer (implements jpos.CashDrawer)
com.ibm.pos.jpos.CoinDispenser (implements jpos.CoinDispenser)
...
//Controladores NCR
com.ncr.posdrivers.CashDrawer (implements jpos.CashDrawer)
com.ncr.posdrivers.CoinDispenser (implements jpos.CoinDispenser)
...
```

Ahora, ¿cómo diseñamos la aplicación de PDV NuevaEra para utilizar los controladores Java de IBM, si es que se utiliza el hardware de IBM, los controladores NCR si fueran los apropiados, etcétera?

Nótese que hay familias de clases (*CashDrawer*+*CoinDispenser*+...) que es necesario que se creen, y cada familia implementa las mismas interfaces.

Para esta situación, existe un patrón GoF que se utiliza comúnmente: Factoría Abstracta (*Abstract Factory*).

Factoría Abstracta (Abstract Factory)

Contexto/Problema

¿Cómo crear familias de clases relacionadas que implementen una interfaz común?

Solución

Defina una interfaz para la factoría (la factoría abstracta). Defina una clase factoría concreta para cada familia de cosas que hay que crear. Opcionalmente, defina una clase abstracta que implemente la interfaz factoría y proporcione servicios comunes a las factorías concretas que la extienden.

⁷ Éstos son nombres de paquetes ficticios.

La Figura 33.15 ilustra la idea básica, que se mejora en la siguiente sección.

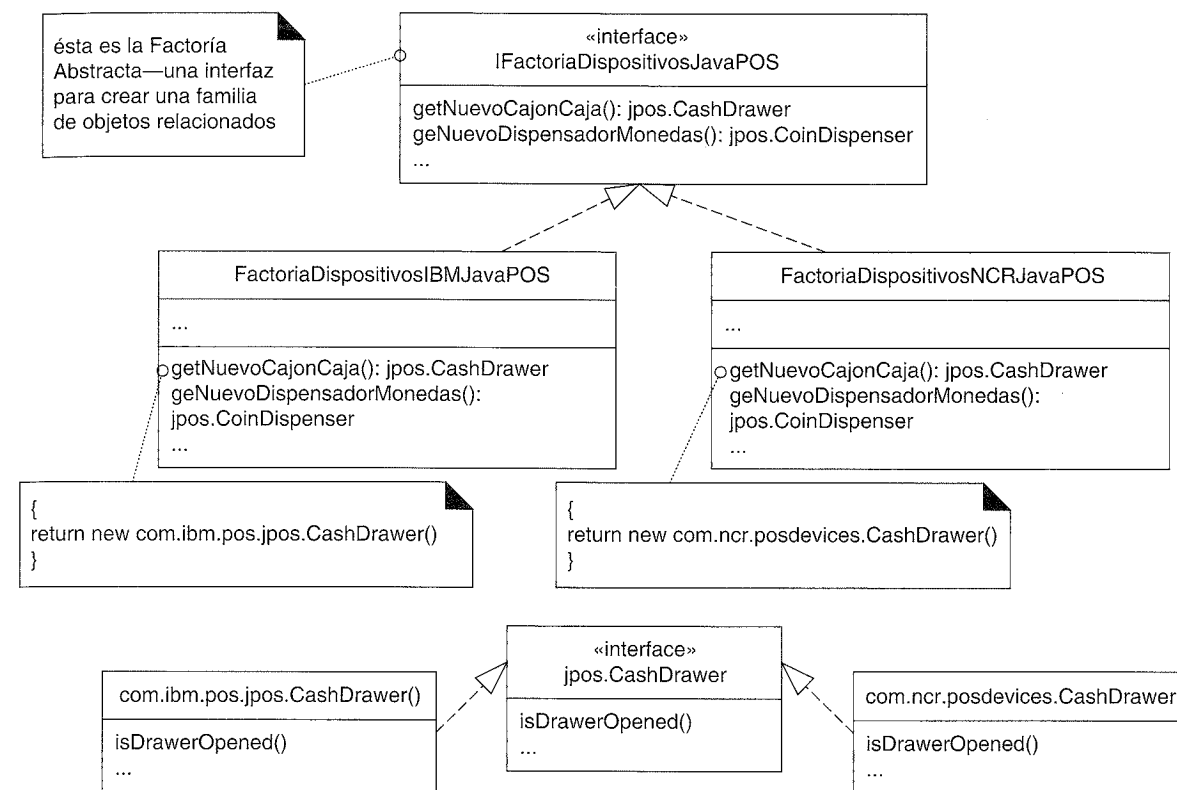


Figura 33.15. Una factoría abstracta básica.

Factoría Abstracta mediante una clase abstracta

Una variación típica de la Factoría Abstracta es crear una clase abstracta factoría a la que se accede utilizando el patrón Singleton; lee una propiedad del sistema para decidir cuál de sus subclases factoría crear, y entonces devuelve la instancia de la subclase apropiada. Esto se utiliza, por ejemplo, en las librerías de Java con la clase *java.awt.Toolkit*, que es una clase abstracta que representa una factoría abstracta para la creación de familias de elementos gráficos del GUI para diferentes sistemas operativos y subsistemas de GUI.

La ventaja de este enfoque es que soluciona este problema: ¿cómo sabe la aplicación qué factoría abstracta utilizar? ¿*FactoriaDispositivosIBMJavaPOS*? ¿*FactoriaDispositivosNCRJavaPOS*?

El siguiente refinamiento soluciona este problema. La Figura 33.16 ilustra la solución.

Con esta clase abstracta como factoría y el método *getInstancia* del patrón Singleton, los objetos pueden colaborar con la superclase abstracta, y obtener una referencia a una de las instancias de sus subclases. Por ejemplo, considere la declaración:

```
cajonCaja=FactoriaDispositivosJavaPOS.getInstancia().getNuevoCajonCaja();
```

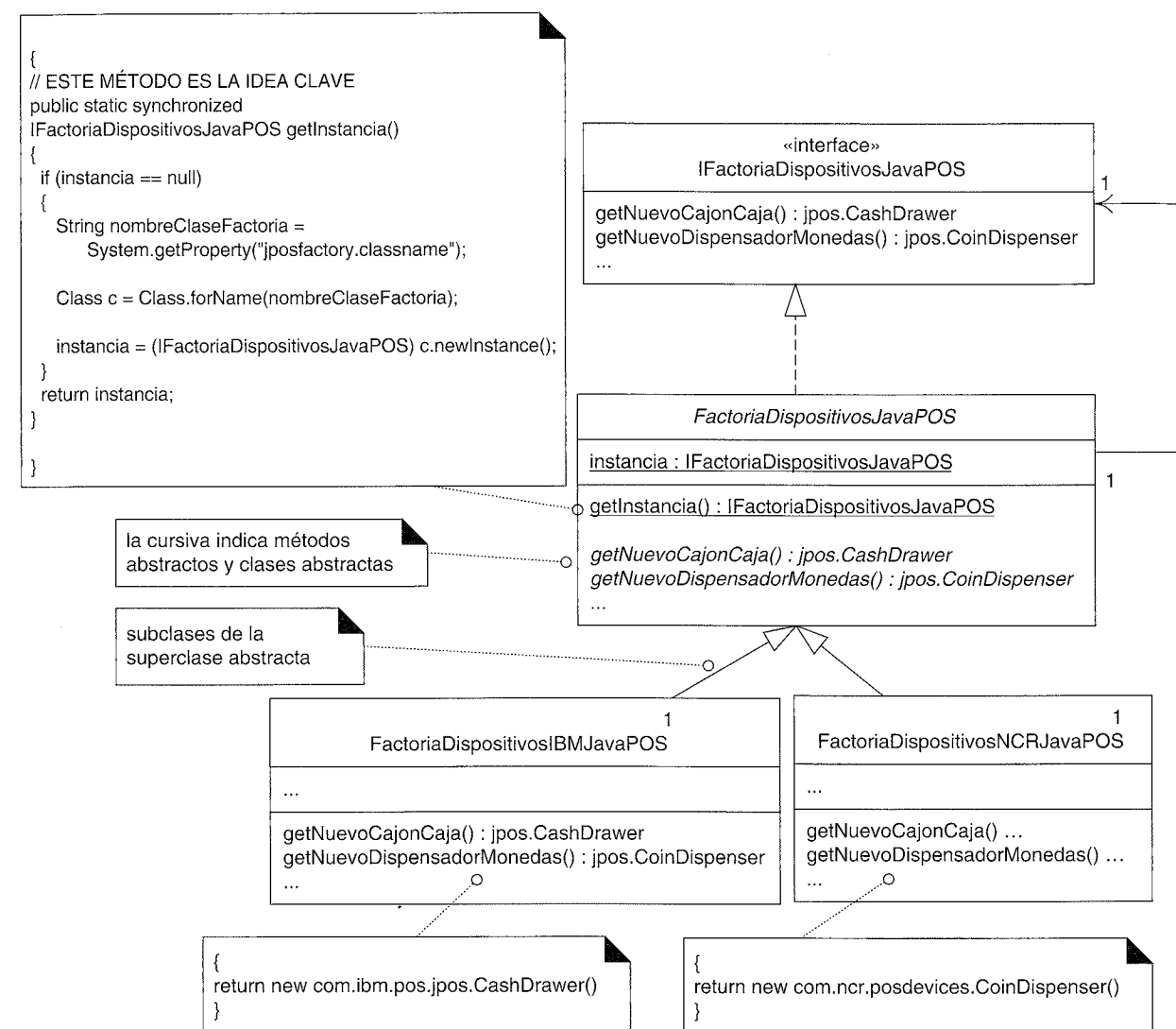


Figura 33.16. Factoría Abstracta mediante una clase abstracta.

La expresión *FactoriaDispositivosJavaPOS.getInstancia()* devolverá una instancia de *FactoriaDispositivosIBMJavaPOS* o *FactoriaDispositivosNCRJavaPOS*, dependiendo de la propiedad del sistema que se lea. Obsérvese que cambiando la propiedad del sistema externa "jposfactory.classname" (que es el nombre de la clase como un String) en un fichero de propiedades, el sistema NuevaEra utilizará una familia diferente de controladores JavaPOS. Se consiguen Variaciones Protegidas con respecto a los cambios de factoría con un diseño de programación dirigido por los datos (leyendo un fichero de propiedades) y reflexivo, utilizando la expresión *c.newInstance()*.

La interacción con la factoría tendrá lugar en un *Registro*. De acuerdo con el objetivo de salto en la representación bajo, es razonable que el *Registro* software (cuyo nom-

bre sugiere el terminal de PDV global) mantenga una referencia a los dispositivos como el *CajonCaja*. Por ejemplo:

```

class Registro
{
    private jpos.CashDrawer cajonCaja;
    private jpos.CoinDispenser dispensadorMonedas;

    public Registro()
    {
        cajonCaja =
            FactoriaDispositivosJavaPos.getInstancia().getNuevoCajonCaja();
        //...
    }
    //...
}

```

33.7. Gestión de pagos con Polimorfismo y Hacerlo Yo Mismo

Una de las formas habituales de aplicación del polimorfismo (y el Experto en Información) es en el contexto de lo que Peter Coad denomina la estrategia o patrón "Hacerlo Yo Mismo" [Coad95]. Es decir:

Hacerlo Yo Mismo

"Yo (un objeto software) hago las cosas que hace normalmente el objeto actual del que yo soy una abstracción." [Coad95]

Éste es el estilo de diseño orientado a objetos clásico: los objetos *Circulo* se dibujan ellos mismos, los objetos *Cuadrado* se dibujan ellos mismos, los objetos *Texto* hacen ellos mismos la comprobación ortográfica, y así sucesivamente.

Nótese que el hecho de que un objeto *Texto* compruebe él mismo la ortografía es un ejemplo del Experto en Información: *El objeto que tiene la información relacionada con el trabajo lo hace* (un *Diccionario* también es un candidato, según el Experto).

Hacerlo Yo Mismo y el Experto en Información normalmente nos llevan a la misma elección.

Análogamente, observe que el hecho de que el *Circulo* y el *Cuadrado* se dibujen ellos mismos son ejemplos de Polimorfismo: *Cuando alternativas relacionadas varían según el tipo, asigne la responsabilidad utilizando operaciones polimórficas a los tipos para los que varía el comportamiento*.

Hacerlo Yo Mismo y el Polimorfismo normalmente nos llevan a la misma elección.

No obstante, como se estudió en la discusión acerca de la Fabricación Pura, con frecuencia está contraindicado debido a problemas con el acoplamiento y la cohesión, y en

lugar de eso, un diseñador utiliza fabricaciones puras como estrategias, factorías y otras por el estilo.

A pesar de todo, cuando es apropiado, Hacerlo Yo Mismo es atractivo en parte porque favorece un salto en la representación bajo. El diseño para la gestión de pagos se llevará a cabo con los patrones Hacerlo Yo Mismo y Polimorfismo.

Uno de los requisitos de esta iteración es gestionar múltiples tipos de pagos, que esencialmente significa gestionar los pasos de autorización y contabilidad. Diferentes tipos de pagos se autorizan de distintas formas:

- Los pagos a crédito y débito se autorizan con un servicio de autorización externo. Ambos requieren que se registre una entrada en las cuentas por cobrar —dinero que debe la institución financiera que hace la autorización—.
- En algunas tiendas (es una tendencia en algunos países) se autorizan los pagos en efectivo utilizando un analizador especial de billetes unido al terminal de PDV que comprueba si el dinero es falso. Otras tiendas no lo hacen.
- En algunas tiendas se autorizan los pagos con cheque utilizando un servicio de autorización informatizado. Otras tiendas no autorizan los cheques.

Los *PagosACredito* se autorizan de una forma; los *PagosConCheque* se autorizan de otra. Éste es un caso clásico de Polimorfismo.

De esta manera, como se muestra en la Figura 33.17, cada subclase de *Pago* tiene su propio método *autorizar*.

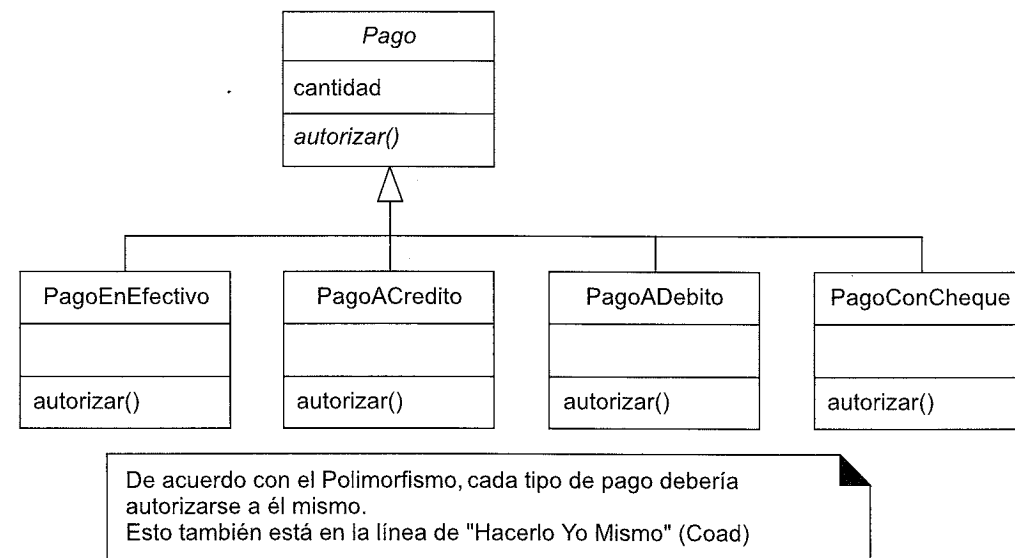


Figura 33.17. Polimorfismo clásico con múltiples métodos *autorizar*.

Por ejemplo, como se ilustra en las Figuras 33.18 y 33.19, una *Venta* instancia un *PagoACredito* o un *PagoConCheque* y le pide que se autorice él mismo.

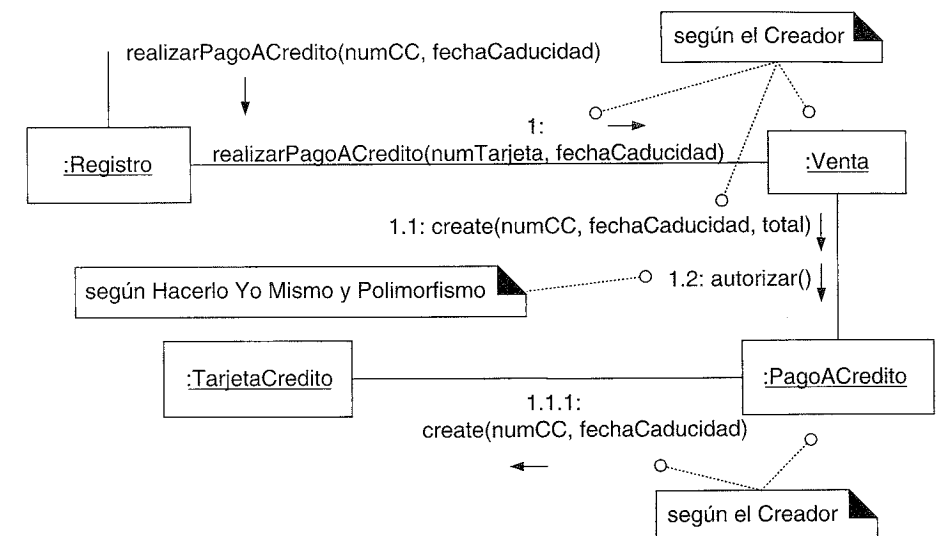


Figura 33.18. Creación de un *PagoACredito*.

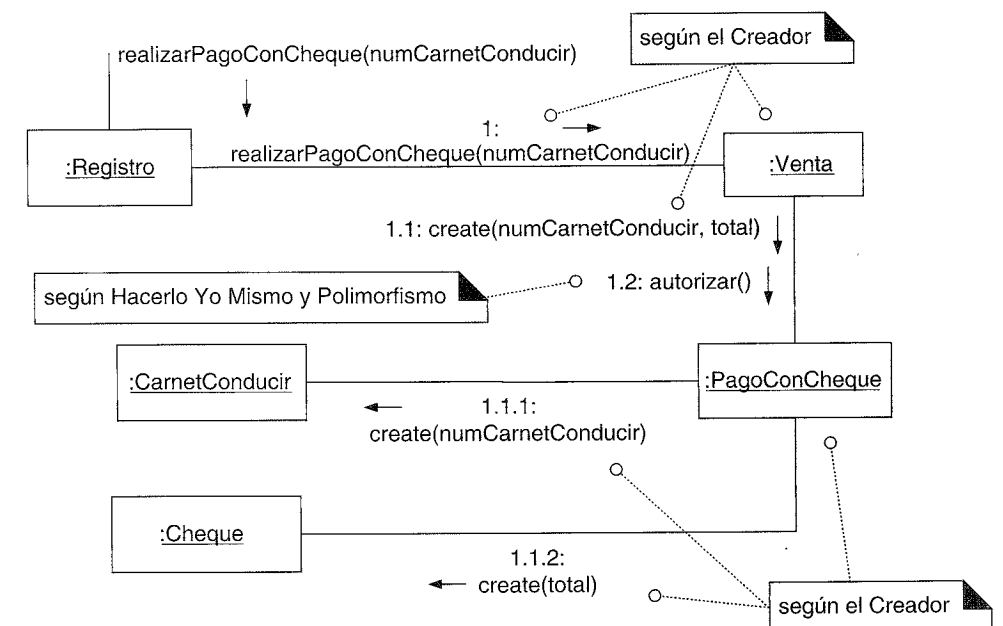


Figura 33.19. Creación de un *PagoConCheque*.

¿Clases pequeñas?

Considere la creación de los objetos software *TarjetaCredito*, *CarnetConducir* y *Cheque*. Nuestro primer impulso podría ser simplemente registrar los datos que contienen en sus clases de pago relacionadas, y eliminar tales clases pequeñas. Sin embargo, normalmente utilizarlas es una estrategia más beneficiosa; a menudo terminan proporcionando com-

portamiento útil y acaban siendo reutilizables. Por ejemplo, la *TajetaCredito* es un Experto natural en decirnos el tipo de compañía de crédito (Visa, MasterCard, etcétera). Este comportamiento resultará necesario para nuestra aplicación.

Autorización de pago a crédito

El sistema se debe comunicar con un servicio externo de autorización de crédito, y ya hemos creado la base del diseño para dar soporte a esto basada en adaptadores.

Información del dominio sobre el pago a crédito relevante

Establezcamos el contexto para el diseño que vamos a realizar a continuación:

- Los sistemas PDV se conectan físicamente con los servicios externos de autorización de varias formas, entre las que se encuentran líneas telefónicas (que se deben marcar) y conexiones permanentes de Internet de banda ancha.
- Se utilizan diferentes protocolos del nivel de la aplicación y formatos de datos asociados, como Transacciones Electrónicas Seguras (SET, *Secure Electronic Transaction*). Podrían hacerse populares otros nuevos, como XMLPay.
- La autorización de pagos se puede ver como una operación síncrona ordinaria: un hilo de ejecución del PDV se bloquea, esperando una respuesta del servicio remoto (dentro de los límites de un periodo de tiempo de espera).
- Todos los protocolos de autorización de pago conllevan el envío de identificadores que identifican de manera única a la tienda (con un "ID de comerciante"), y al terminal del PDV (con un "ID del terminal"). Una respuesta incluye un código de aprobación o denegación, y un ID de transacción único.
- Una tienda podría utilizar diferentes servicios de autorización externos para distintos tipos de tarjetas de crédito (uno para la VISA y otro para la MasterCard). Para cada servicio, la tienda tiene un ID de comerciante distinto.
- El tipo de la compañía de crédito se puede deducir a partir del número de la tarjeta. Por ejemplo, los números que comienzan por 5 son MasterCard; los números que empiezan por 4 son Visa.
- Las implementaciones de los adaptadores protegerán a las capas superiores del sistema contra todas estas variaciones en la autorización de los pagos. Cada adaptador es responsable de asegurar que la transacción de solicitud de la autorización tiene el formato adecuado, y de la colaboración con el servicio externo. Como se discutió en la iteración anterior, la *FactoriaDeServicios* es responsable de enviar la implementación de *IAdaptadorServicioAutorizacionCredito* adecuada.

Un escenario de diseño

La Figura 33.20 comienza la presentación de un diseño con anotaciones que satisfacen estos detalles y requisitos. Los mensajes tienen notas aclaratorias para mostrar el razonamiento.

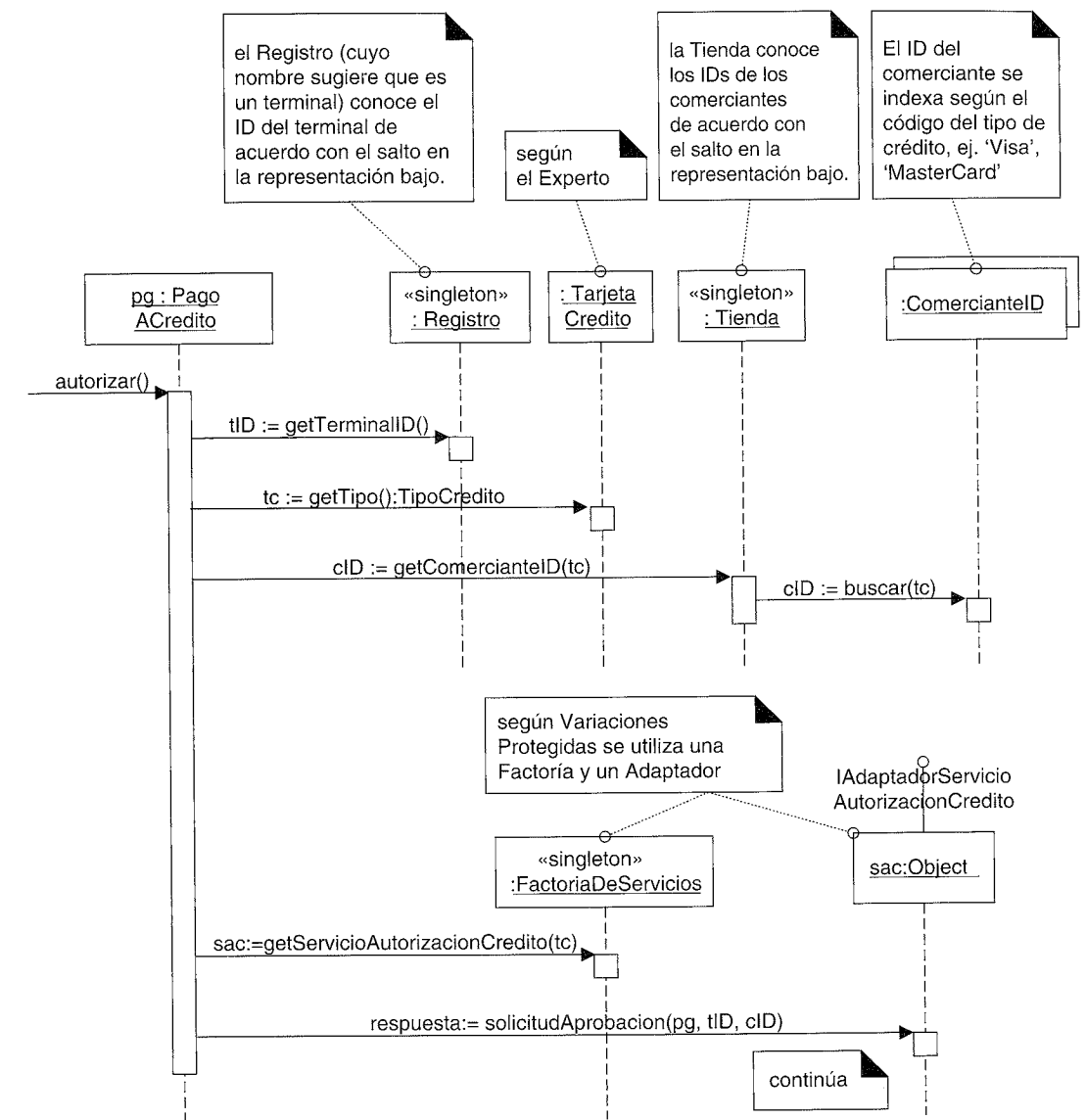


Figura 33.20. Gestión de un pago a crédito.

Una vez que se encuentra el *IAdaptadorServicioAutorizacionCredito* correcto, se le otorga la responsabilidad de completar la autorización, como se muestra en la Figura 33.21.

En el momento que se obtiene la respuesta del *PagoACredito* (al que se le ha dado la responsabilidad de gestionar su finalización de acuerdo con el Polimorfismo y Hacerlo Yo Mismo), asumiendo que se aprueba, completa sus tareas, como se muestra en la Figura 33.22.

Notación UML: Obsérvese en este diagrama de secuencia que se apilaron algunos objetos. Esto es legal, aunque pocas herramientas CASE lo soportan. Es de utilidad para incluirlo en un libro o documento, donde el ancho de página está limitado.

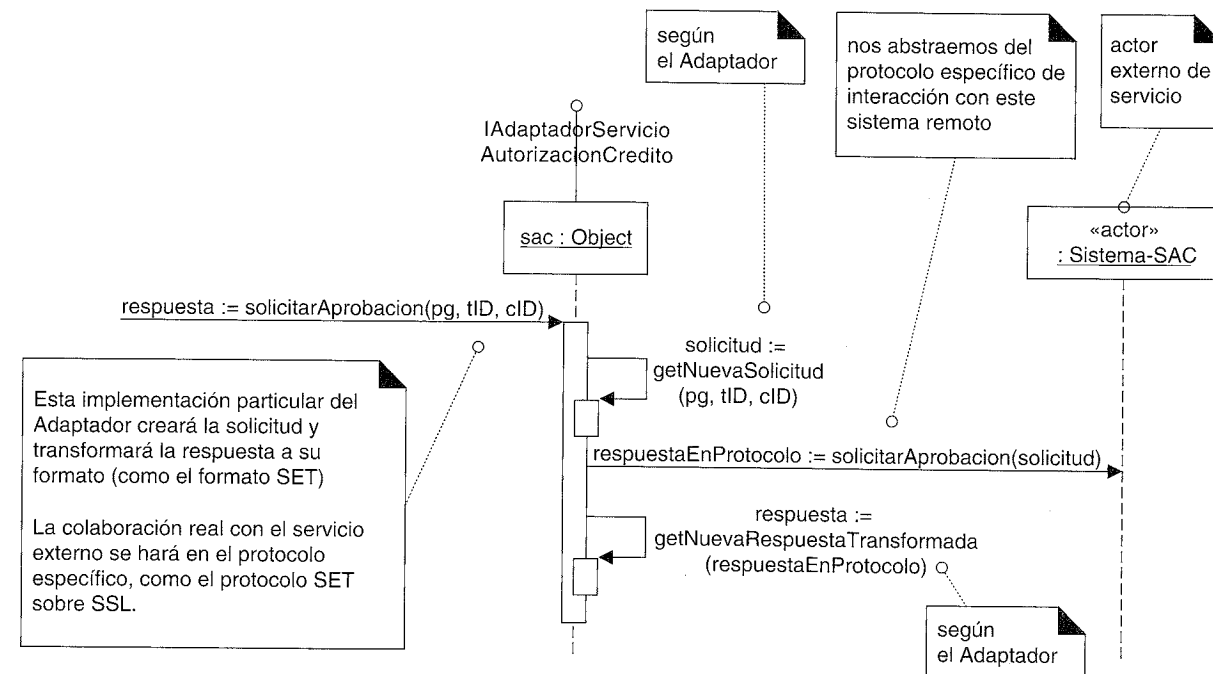


Figura 33.21. Finalización de la autorización.

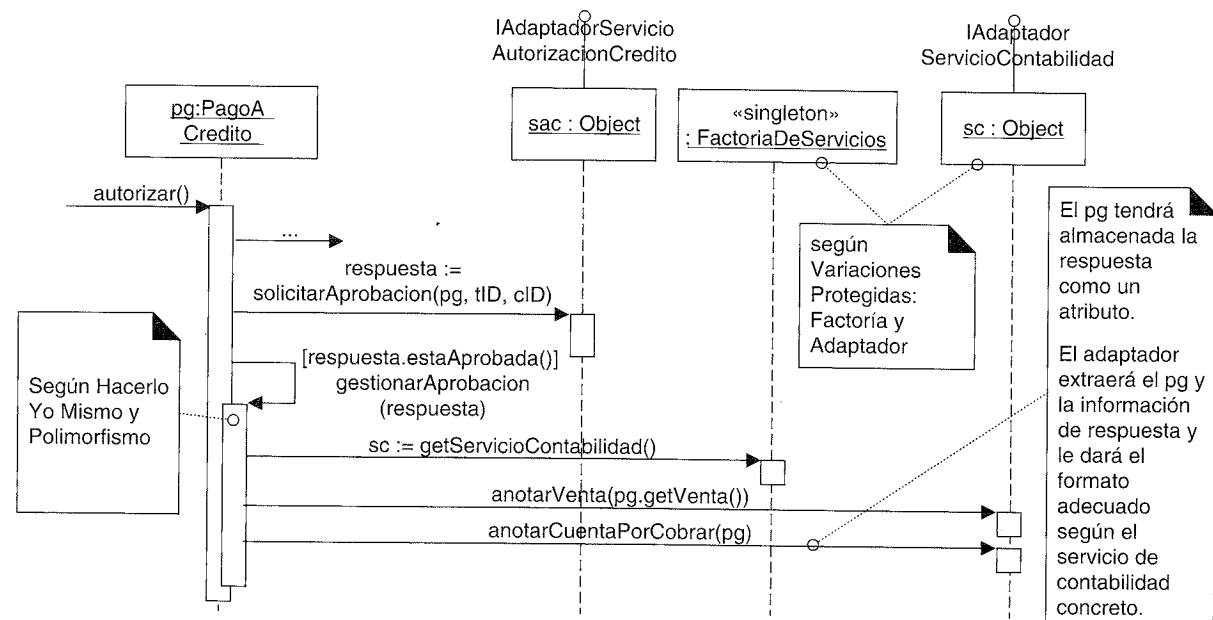


Figura 33.22. Finalización de un pago a crédito aprobado.

33.8. Conclusión

Lo importante de este caso de estudio no era mostrar la solución correcta —no existe una única solución que sea la mejor—, y estoy seguro de que los lectores pueden me-

jorar lo que he propuesto. Espero sinceramente haber demostrado que se puede llevar a cabo el diseño de objetos basado en un razonamiento que siga principios básicos como bajo acoplamiento y la aplicación de patrones, en lugar de ser un proceso misterioso.

Advertencia: Patron-itis

Esta presentación ha utilizado los patrones de diseño GoF en muchos puntos, lo cual es una de las cosas importantes del caso de estudio como ayuda al aprendizaje. Pero, ha habido informes de diseñadores preocupados excesivamente por introducir los patrones de una manera forzada en un frenesí creativo de patron-itis. La conclusión que podemos extraer de esto es que es necesario estudiar los patrones en múltiples ejemplos para digerirlos bien. Un método de aprendizaje extendido es formar un grupo de estudio a la hora de la comida o después del trabajo en el que los participantes comparten las formas que han visto o podrían ver de la aplicación de los patrones, y discutir una sección de un libro sobre patrones.

DISEÑO DE UN FRAMEWORK DE PERSISTENCIA CON PATRONES

*Le temps est un grand professeur, mais
malheureusement il tue tous ses élèves*
(El tiempo es un gran profesor, pero desgraciadamente mata a todos sus alumnos).

Hector Berlioz

Objetivos

- Diseñar parte de un framework¹ con los patrones Método Plantilla, Estado y Command.
 - Introducir las cuestiones de la correspondencia (*mapping*) objeto-relacional (O-R).
 - Implementar la materialización perezosa con Proxies Virtuales.
-

Introducción

La aplicación NuevaEra —como la mayoría— requiere que se almacene y recupere la información en mecanismos de almacenamiento persistente, como una base de datos relacional (BDR). Este capítulo presenta el diseño de un framework para el almacenamiento de objetos persistentes.

Normalmente es mejor conseguir o comprar uno de éstos, ya sea un producto independiente o parte de un contenedor que maneja la persistencia para beans entidad (*entity beans*) si se utiliza EJBs y otras tecnologías de Java. Construir un servicio de persistencia O-R de calidad industrial puede llevar mucho esfuerzo persona-año, y existen cuestiones sutiles que requieren una experiencia especializada. Además, las tecnologías como las que se basan en *Java Data Objects* (JDO) ofrecen soluciones parciales.

Por tanto, la intención no es mostrar un framework industrial o sugerir que se ignoren las tecnologías como JDO, sino más bien utilizar un framework de persistencia como

¹ *N. del T.* Los miembros de la comunidad software de habla hispana utilizan el término original en inglés, aunque a veces se traduce framework por marco, elección que no nos parece muy acertada.

medio para explicar el diseño general de frameworks con patrones, puesto que constituye un caso de estudio especialmente bueno. Es también otro ejemplo de utilización de UML para comunicar un diseño software.

Este framework se presenta para introducir el diseño de los frameworks, no como un enfoque recomendado para el diseño de un servicio de persistencia industrial.

34.1. El problema: objetos persistentes

Asuma que en la aplicación NuevaEra, los datos de la *EspecificacionDelProducto* residen en una base de datos relacional. Estos datos deben traerse a la memoria local durante el uso de la aplicación. Los **objetos persistentes** son aquellos que requieren almacenamiento persistente, como las instancias de *EspecificacionDelProducto*.

Mecanismos de almacenamiento y objetos persistentes

Bases de datos de objetos: Si se utiliza una base de datos de objetos para almacenar y recuperar los objetos, no se necesita ningún servicio de persistencia a medida o de terceras partes adicional. Éste es uno de los diversos atractivos de su uso.

Bases de datos relacionales: Debido al predominio de las BDR, a menudo es necesario que se utilicen, en lugar de las bases de datos de objetos que son más convenientes. Si es éste el caso, surgen algunos problemas debido a la incompatibilidad entre la representación de los datos orientada a registros y la orientada a objetos; estos problemas se estudiarán más adelante. Se requiere un servicio especial para establecer la correspondencia O-R (*mapping O-R*).

Otros: Además de las BDR, a veces se desea almacenar los objetos en otros mecanismos de almacenamiento o formatos, como simples ficheros, estructuras XML, ficheros Palm OS PDB, bases de datos jerárquicas, etcétera. Como con las bases de datos relacionales, existen incompatibilidades entre las representaciones de los objetos y estos formatos que no son orientados a objetos. Y como con las BDR, se requieren servicios especiales que hagan que funcionen con objetos.

34.2. La solución: un servicio de persistencia a partir de un framework de persistencia

Un **framework de persistencia** es un conjunto de tipos de propósito general, reutilizable y extensible, que proporciona funcionalidad para dar soporte a los objetos persistentes. Un **servicio de persistencia** (o subsistema) realmente proporciona el servicio, y se creará con un framework de persistencia. Un servicio de persistencia se escribe normalmente para que trabaje con BDR, en cuyo caso también se conoce como **servicio de correspondencia O-R**. Generalmente, un servicio de persistencia tiene que traducir los objetos a registros (o a alguna otra forma de datos estructurada como XML) y guardarlos en una base de datos, y traducir los registros a objetos cuando los recuperamos de la base de datos.

En cuanto a la arquitectura en capas de la aplicación NuevaEra, un servicio de persistencia es un subsistema dentro de la capa de servicios técnicos.

34.3. Frameworks

Aun a riesgo de simplificar en exceso, un framework es un conjunto *extensible* de objetos para funciones relacionadas. El ejemplo prototípico es un framework de GUI, como las AWT o Swing de Java.

La señal de calidad de un framework es que proporciona una implementación para las funciones básicas e invariables, e incluye un mecanismo que permite al desarrollador conectar las funciones que varían, o extender las funciones.

Por ejemplo, el framework Swing de GUI de Java proporciona muchas clases e interfaces para las funciones principales de la GUI. Los desarrolladores pueden incluir elementos gráficos especializados creando subclases de las clases Swing y redefiniendo ciertos métodos. Los desarrolladores también pueden conectar diversos comportamientos de respuesta a los eventos en las clases de los elementos gráficos predefinidas (como *JButton*) registrando oyentes o suscriptores basados en el patrón Observador. Eso es un framework.

En general, un **framework**:

- Es un conjunto cohesivo de interfaces y clases que colaboran para proporcionar los servicios de la parte central e invariable de un subsistema lógico.
- Contiene clases concretas (y especialmente) abstractas que definen las interfaces a las que ajustarse, interacciones de objetos en las que participar, y otras invariantes.
- Normalmente (aunque no necesariamente) requiere que el usuario del framework defina subclases de las clases del framework existentes para utilizar, adaptar y extender los servicios del framework.
- Tiene clases abstractas que podrían contener tanto métodos abstractos como concretos.
- Confía en el **Principio Hollywood** —“*No nos llame, nosotros le llamaremos*”. Esto significa que las clases definidas por el usuario (por ejemplo, nuevas clases) recibirán mensajes desde las clases predefinidas del framework. Estos mensajes normalmente se manejan implementando métodos abstractos en las superclases.

El siguiente ejemplo del framework de persistencia describirá y explicará estos principios.

Los frameworks son reutilizables

Los frameworks ofrecen un alto grado de reutilización —mucho más que con clases individuales—. En consecuencia, si una organización está interesada (¿y quién no lo está?) en incrementar su grado de reutilización del software, entonces debería enfatizar la creación de frameworks.

34.4. Requisitos para el servicio y framework de persistencia

Para la aplicación de PDV NuevaEra, necesitamos un servicio de persistencia que se construya con un framework de persistencia (que se podría utilizar también para crear otros servicios de persistencia). Llamemos al framework FWP (Framework de Persis-

tencia). FWP es un framework simplificado —un framework de persistencia desarrollado de calidad industrial queda fuera del alcance de esta introducción—.

El framework debería proporcionar funciones para:

- almacenar y recuperar los objetos en un mecanismo de almacenamiento persistente
- *confirmar y deshacer (commit y rollback)* las transacciones

El diseño debe ser extensible para dar soporte a diferentes mecanismos de almacenamiento, como BDRs, registros en ficheros simples, o XML en ficheros.

34.5. Ideas claves

Las siguientes ideas claves se estudiarán en las secciones que vienen a continuación:

- **Correspondencia:** Se debe establecer alguna correspondencia (*mapping*) entre una clase y su almacenamiento persistente (por ejemplo, una tabla en una base de datos), y entre los atributos de los objetos y los campos (columnas) en un registro. Es decir, debe existir un **correspondencia de esquemas** entre los dos esquemas.
- **Identidad de objeto:** Los registros y los objetos tienen un único identificador de objeto para relacionar fácilmente los registros con los objetos, y asegurar que no hay duplicados inapropiados.
- **Conversor de base de datos:** Una Fabricación Pura conversor (*mapper*) de base de datos es responsable de la materialización y desmaterialización.
- **Materialización y desmaterialización:** La materialización es el acto de transformar una representación de datos no orientada a objetos (por ejemplo, registros) de un almacenamiento persistente en objetos. La desmaterialización es la actividad opuesta (también conocida como *passivation*).
- **Caché:** Los servicios persistentes almacenan en una caché los objetos materializados por razones de rendimiento.
- **Estado de transacción de los objetos:** Es útil conocer el estado de los objetos en función de sus relaciones con la transacción actual. Por ejemplo, es útil conocer qué objetos se han modificado (están *sucios*) de manera que es posible determinar si es necesario que se guarden de nuevo en su almacenamiento persistente.
- **Operaciones de transacción:** Operaciones confirmar y deshacer (*commit y rollback*).
- **Materialización perezosa:** No todos los objetos se materializan de una vez; una instancia particular sólo se materializa bajo demanda, cuando se necesita.
- **Proxies virtuales:** La materialización perezosa se puede implementar utilizando una referencia inteligente que se conoce como proxy virtual.

34.6. Patrón: Representación de Objetos como Tablas

¿Cómo conviertes un objeto en un registro o esquema de base de datos relacional?

El patrón **Representación de Objetos como Tablas** [BW96] propone la definición de una tabla en una BDR por cada clase de objeto persistente. Los atributos de los

objetos que contienen tipos de datos primitivos (número, cadena de texto, booleano, etcétera) se corresponden con las columnas.

Si un objeto sólo tiene atributos de tipos de datos primitivos, la correspondencia es directa. Pero como veremos, las cosas no son tan simples puesto que los objetos podrían tener atributos que hacen referencia a otros objetos complejos, mientras que el modelo relacional requiere que los valores sean atómicos (esto es, la Primera Forma Normal) (ver Figura 34.1).

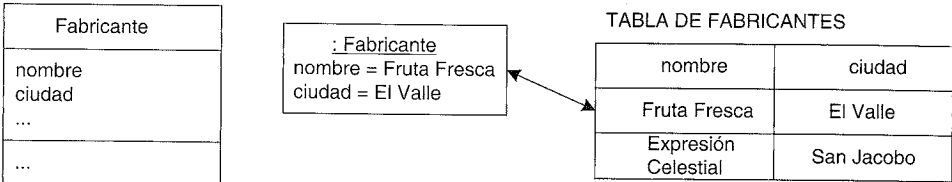


Figura 34.1. Correspondencia entre objetos y tablas.

34.7. Perfil (Profile) de modelado de datos en UML

A propósito de las BDR, no es sorprendente que UML se haya convertido en una notación muy utilizada para los **modelos de datos**. Fíjese que uno de los artefactos oficiales del UP es el Modelo de Datos, que forma parte de la disciplina de Diseño. La Figura 34.2 ilustra alguna notación UML para el modelado de datos.

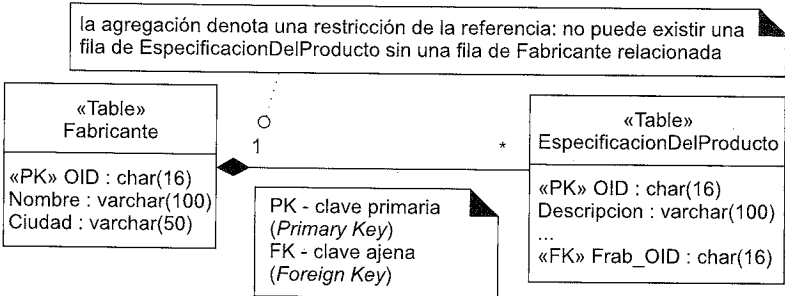


Figura 34.2. Ejemplo del Perfil (Profile) de Modelado de Datos en UML.

Estos estereotipos no forman parte del núcleo de UML —son extensiones—. Generalizando, UML tiene el concepto de **perfil** (perfil de UML): un conjunto coherente de estereotipos de UML, valores etiquetados y restricciones para un propósito específico. La Figura 34.2 ilustra parte del Perfil de Modelado de Datos de UML propuesto (al OMG); en el momento en el que se escribió este libro no se había aprobado. Un perfil no tiene que estar aprobado por el OMG para ser un perfil, aunque se están enviando algunos de uso muy extendido —como el modelado de datos— para que se aprueben.

34.8. Patrón: Identificador de Objeto

Es conveniente contar con una forma consistente de relacionar los objetos con los registros, y ser capaces de asegurar que la materialización repetida de un registro no da como resultado objetos duplicados.

El patrón **Identificador de Objeto** [BW96] propone asignar un **identificador de objeto** (OID) a cada registro y objeto (o proxy de un objeto).

Un OID normalmente es un valor alfanumérico; cada uno es único para un objeto específico. Existen varios enfoques para generar identificadores únicos para los OIDs, variando desde únicos para una base de datos, a únicos globalmente: generadores de secuencia de bases de datos, la estrategia de generación de claves Alto-Bajo [Ambler00], y otros.

En el campo de los objetos, un OID se representa mediante una interfaz o clase OID que encapsula el valor real y su representación. En un BDR, normalmente se almacena como un valor de tipo carácter de longitud fija.

Cada tabla tendrá un OID como clave primaria, y cada objeto también tendrá (directa o indirectamente) un OID. Si se asocia cada objeto con un OID, y cada tabla tiene un OID como clave primaria, cada objeto se corresponde de manera única con una fila de alguna tabla (ver Figura 34.3).

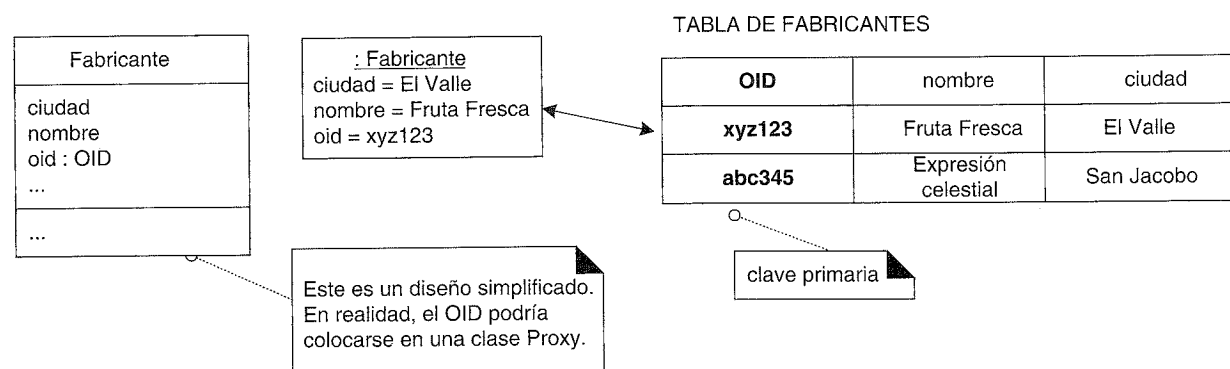


Figura 34.3. Los identificadores de los objetos enlazan objetos y registros.

Ésta es una vista simplificada del diseño. En realidad, el OID podría no colocarse exactamente en el objeto persistente —aunque sea posible—. En lugar de eso, se podría colocar en un objeto Proxy que envuelve al objeto persistente. En el diseño influye la elección del lenguaje.

Un OID también proporciona un tipo de clave consistente para utilizarla en la interfaz con el servicio de persistencia.

34.9. Acceso al servicio de persistencia con una Fachada

El primer paso del diseño de este subsistema es definir una fachada para estos servicios; recordemos que la Fachada es un patrón común para proporcionar una interfaz uniforme a un subsistema. Para empezar, se necesita una operación para recuperar un objeto dado un OID. Pero además del OID, el subsistema necesita conocer el tipo del objeto que se va a materializar; por tanto, también se debe proporcionar el tipo de la clase. La Figura 34.4 ilustra algunas operaciones de la fachada y su uso en colaboración con uno de los adaptadores de servicios de NuevaEra.

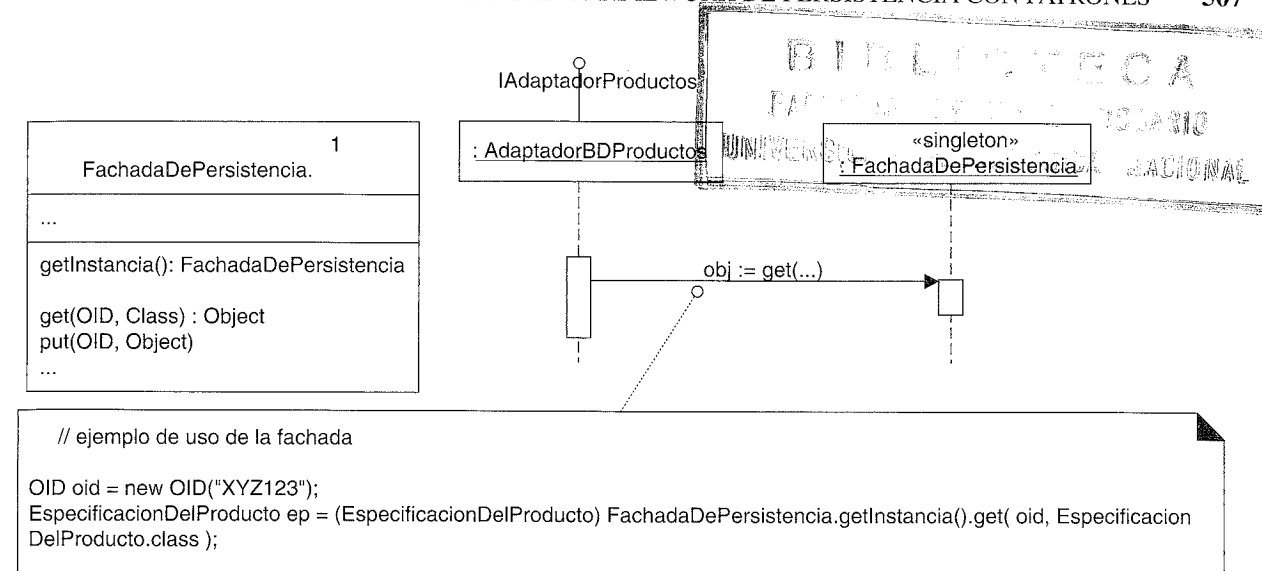


Figura 34.4. La FachadaDePersistencia.

34.10. Correspondencia de los objetos: patrón Conversor (Mapper) de Base de Datos o Intermediario (Broker) de Base de Datos

La *FachadaDePersistencia* —como se cumple en todas las fachadas— no hace ella misma el trabajo, sino que delega las peticiones en objetos del subsistema.

¿Quién debe ser el responsable de la materialización y desmaterialización de los objetos (por ejemplo, una *EspecificacionDelProducto*) procedentes de un almacenamiento persistente?

El patrón Experto en Información sugiere que la propia clase del objeto (*EspecificacionDelProducto*) persistente es candidata, porque tiene algunos de los datos (los datos que se van a almacenar) que necesita la responsabilidad.

Si una clase de objetos persistentes define el código para almacenarse ella misma en una base de datos, se denomina diseño de **correspondencia directa**. Se puede utilizar la correspondencia directa si el código relacionado de la base de datos se genera y se inyecta automáticamente en la clase mediante un compilador de post-procesamiento, y el desarrollador nunca tiene que ver o mantener este código de base de datos complejo que añade confusión a su clase.

Pero si la correspondencia directa se añade y mantiene de manera manual, tiene varios defectos y tiende a no ser escalable en cuanto a la programación y el mantenimiento. Entre los problemas encontramos:

- Fuerte acoplamiento de la clase de objetos persistentes y el conocimiento del almacenamiento persistente —violación de Bajo Acoplamiento—.
- Responsabilidades complejas en un área nueva y no relacionada con las responsabilidades previas del objeto —violación de Alta Cohesión y mantenimiento de la

separación de intereses—. Cuestiones relacionadas con servicios técnicos se mezclan con otras propias de la lógica de la aplicación.

Estudiaremos un enfoque clásico de **correspondencia indirecta**, que utiliza otros objetos para establecer la correspondencia con los objetos persistentes.

Parte de este enfoque es utilizar el patrón **Intermediario (Broker) de Base de Datos** [BW95]. Éste propone crear una clase que sea responsable de materializar y desmaterializar un objeto almacenado. También se le ha llamado patrón **Conversor (Mapper) de Base de Datos** en [Fowler01], que es un nombre más adecuado que Broker de Bases de Datos, puesto que describe su responsabilidad, y el término “broker” en el diseño de los sistemas distribuidos [BMRSS96] tiene un significado distinto, que existe desde hace mucho tiempo².

Se define una clase diferente que establece la correspondencia para cada clase de los objetos persistentes. La Figura 34.5 ilustra que cada objeto persistente podría tener su propia clase que lleve a cabo la correspondencia, y que podrían existir diferentes tipos de conversores para diferentes tipos de mecanismos de almacenamiento. Un extracto del código:

```
class FachadaDePersistencia
{
    //...
    public Object get(OID oid, Class clasePersistente)
    {
        //la clave de IConversor es la Clase del objeto persistente
        IConversor conversor = (IConversor)conversores.get(clasePersistente);

        //delega
        return conversor.get(oid);
    }
    //...
}
```

Aunque este diagrama señala dos conversores para la *EspecificacionDelProducto*, sólo uno de ellos estará activo en un servicio de persistencia en funcionamiento.

Conversores basados en metadatos

Un diseño de conversores más flexible, pero más difícil, se basa en **metadatos** (datos sobre los datos). A diferencia de las clases hechas a mano que establecen la correspondencia de manera individual para diferentes tipos persistentes, los conversores basados en metadatos generan dinámicamente la correspondencia entre un esquema de objetos y otro esquema (como el relacional) en base a la lectura de metadatos que describen la correspondencia, como “La TablaX se corresponde con la Clase Y; la columna Z se corresponde con la propiedad P del objeto” (llega a ser mucho más complejo). Este enfoque es viable en lenguajes con capacidades de programación reflexiva, como Java, C# o Smalltalk, y resulta difícil para aquellos que no las tienen, como C++.

Estableciendo la correspondencia en base a metadatos, podemos cambiar la correspondencia de esquemas en un almacenamiento externo y tendrá efecto en el sistema en marcha, sin que haya que cambiar el código fuente —Variaciones Protegidas con respecto a las variaciones en los esquemas—.

No obstante, una cualidad útil del framework que se ha presentado aquí es que se pueden utilizar los conversores escritos a mano o basados en metadatos, sin afectar a los clientes —encapsulación de la implementación—.

34.11. Diseño del framework con el patrón Método Plantilla

La siguiente sección describe algunas de las características esenciales del diseño de los Conversores de Bases de Datos, que constituyen una parte central del FWP. Estas características de diseño se basan en el patrón de diseño GoF **Método Plantilla (Template Method)** [GHJV95]³. Este patrón es una parte esencial del diseño del framework⁴, y es familiar para la mayoría de los programadores OO por la práctica si no por el nombre.

³ Este patrón no está relacionado con las plantillas (template) de C++. El patrón describe la *plantilla* de un algoritmo.
⁴ De manera más específica, de los **frameworks de caja blanca**. Normalmente éstos son frameworks orientados a la definición de subclases y jerarquías de clases que requieren que los usuarios conozcan algo acerca de su diseño y estructura; de ahí lo de caja blanca.

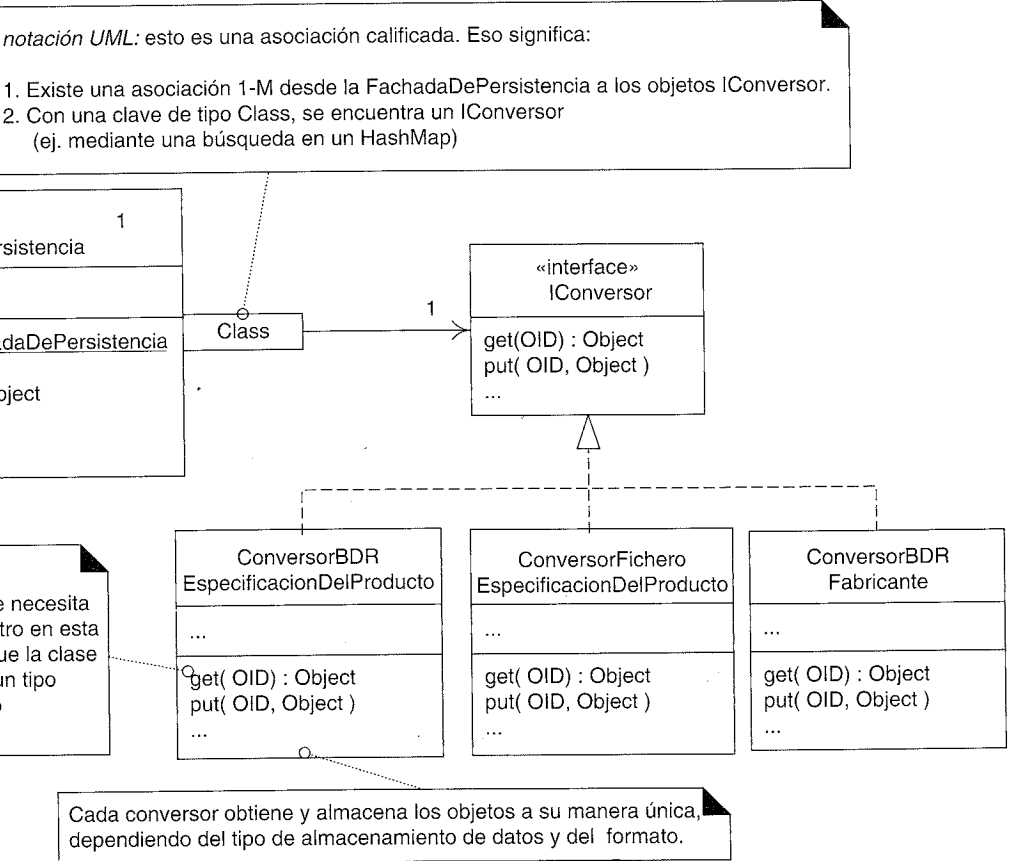


Figura 34.5. Conversores de bases de datos.

² En los sistemas distribuidos, un **broker** es un proceso del servidor front-end que delega las tareas en los procesos del servidor back-end.

La idea es crear un método (el Método Plantilla) en una superclase que define el esqueleto de un algoritmo, con sus partes variables e invariables. El Método Plantilla invoca otros métodos, algunos de los cuales podrían redefinirse en una subclase. De esta manera, las subclases pueden redefinir los métodos que varían para añadir su propio comportamiento único en los puntos de variabilidad (ver Figura 34.6).

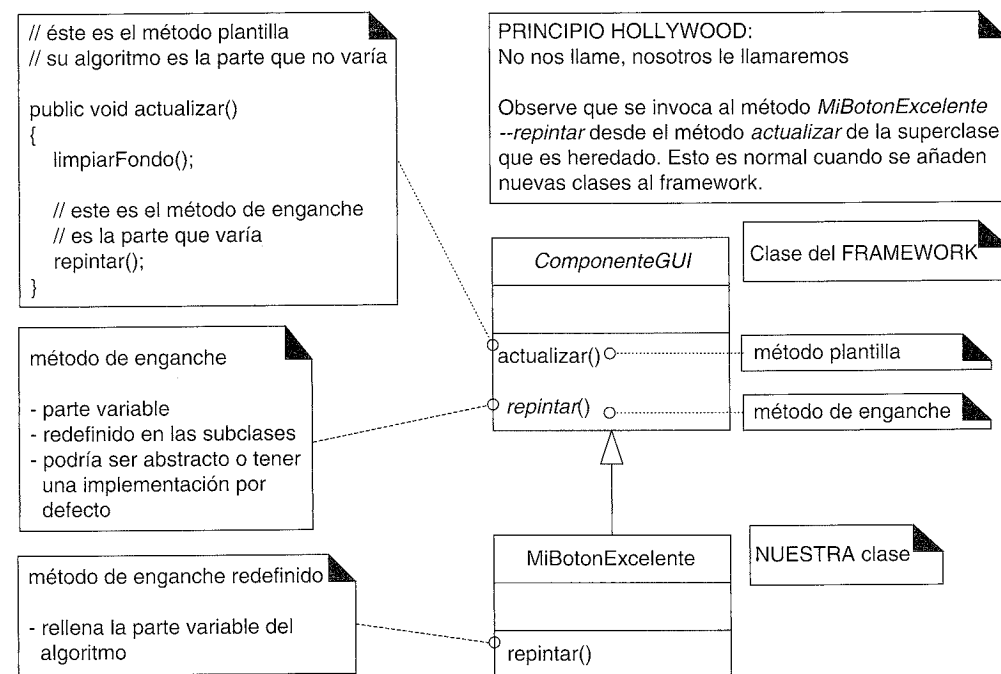


Figura 34.6. Patrón Método Plantilla en un framework de GUI.

34.12. Materialización con el patrón Método Plantilla

Si tuviéramos que programar dos o tres clases para establecer la correspondencia con el mecanismo de almacenamiento permanente, apreciaríamos partes comunes en el código. La estructura básica que se repite del algoritmo para materializar un objeto es:

```
if (objeto está en cache)
    return obj
else
    crear el objeto a partir de su representación en el almacenamiento
    guardar el objeto en la cache
    return obj
```

El punto de variación es la manera de crear el objeto a partir del almacenamiento.

Crearemos el método *get* que será el método plantilla en una superclase abstracta *ConversorPersistenciaAbstracto* que define la plantilla, y utiliza un método “de enganche” (*hook method*) en las subclases para la parte que varía. La Figura 34.7 muestra el diseño esencial.

Como se muestra en este ejemplo, es normal que el método plantilla sea *público*, y que el método de enganche sea *protegido*. El *ConversorPersistenciaAbstracto* e *Iconversor* forman parte del FWP. Ahora, un programador de aplicaciones puede incorporar

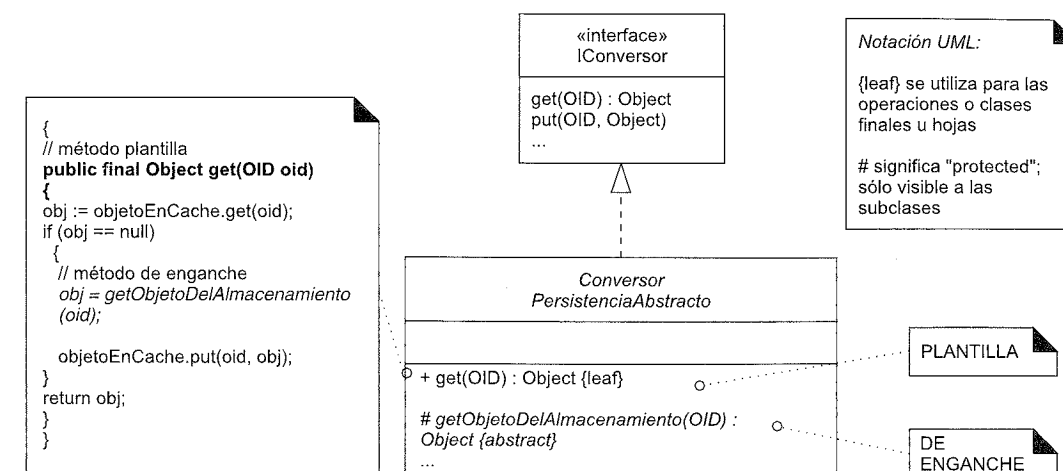


Figura 34.7. Método Plantilla para objetos conversores.

elementos en este framework añadiendo una subclase, y redefiniendo o implementando el método de enganche *getObjetoDelAlmacenamiento*. La Figura 34.8 muestra un ejemplo.

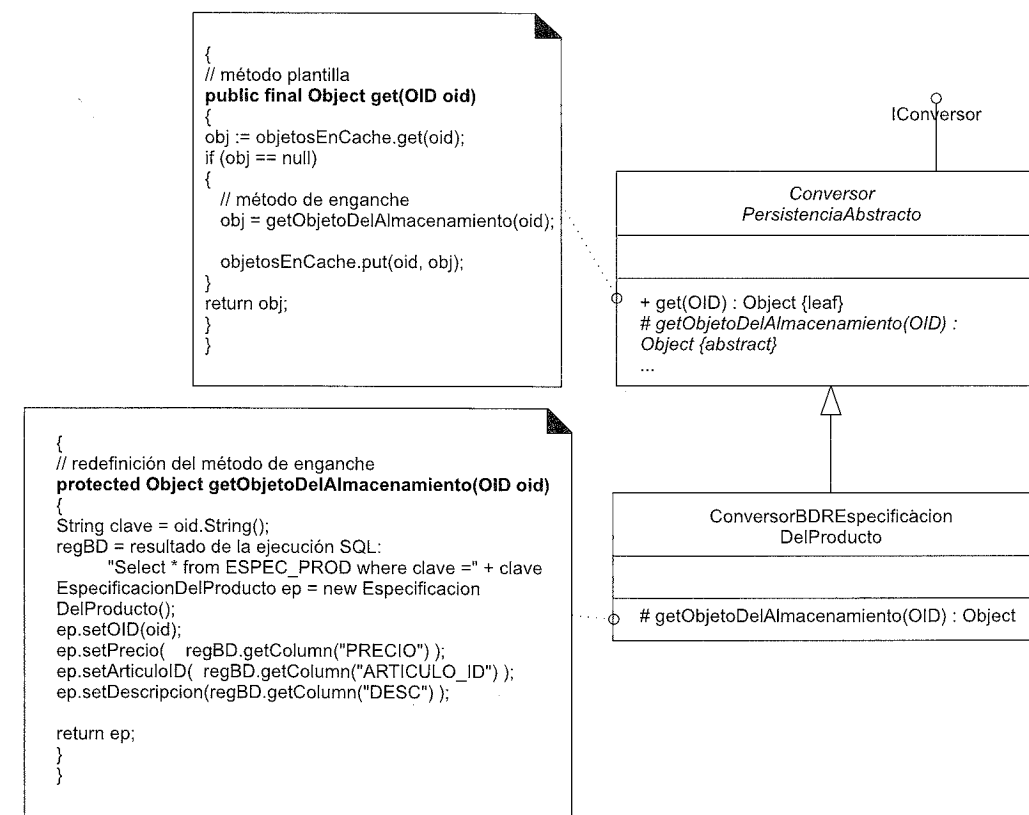


Figura 34.8. Redefinición del método de enganche⁵.

⁵ En Java por ejemplo, el *regBD* que devuelve la ejecución de la consulta SQL sería un *ResultSet* de JDBC.

Asuma que en la implementación del método de enganche de la Figura 34.8, la primera parte del algoritmo —la ejecución del SELECT de SQL— es la misma para todos los objetos, sólo varía el nombre de la tabla de la base de datos⁶. Si se sostiene esta suposición, entonces una vez más, podría aplicarse el patrón Método Plantilla para factorizar por separado las partes que varían y las que no. En la Figura 34.9, la parte artificiosa es que *ConversorAbstractoBDR*--*getObjetoDelAlmacenamiento* es un método de enganche con respecto al método *ConversorPersistenciaAbstracto*--*get*, pero es un método plantilla con respecto al nuevo método de enganche *getObjetoDelRegistro*.

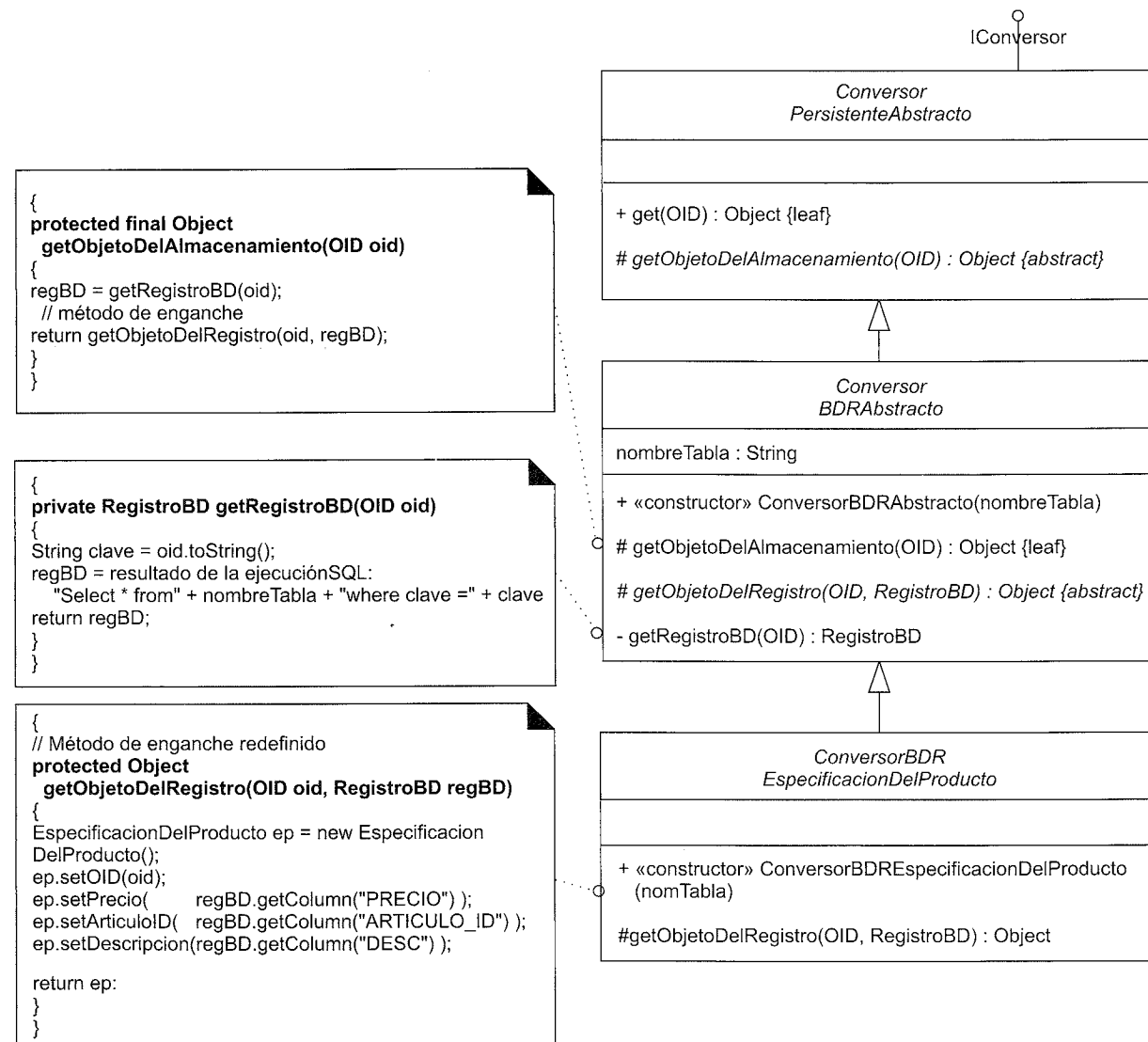


Figura 34.9. Ajustando el código de nuevo con el Método Plantilla.

⁶ En muchos casos, la situación no es tan simple. Un objeto podría derivarse a partir de los datos contenidos en dos o más tablas o a partir de múltiples bases de datos, en cuyo caso, la primera versión del diseño del Método Plantilla es más flexible.

Notación UML: Observe cómo se pueden declarar los constructores en UML. El estereotipo es opcional, y si se utiliza la convención de que el nombre del constructor sea igual al nombre de la clase, probablemente es innecesario.

Ahora *IConversor*, *ConversorPersistenciaAbstracto* y *ConversorBDRAbstracto* forman parte del framework. El programador de la aplicación sólo necesita añadir su subclase, como *ConversorBDREspecificacionDelProducto*, y asegurar que se crea con el nombre de la tabla (se pasa mediante el encadenamiento de constructores hasta el *ConversorBDRAbstracto*).

La jerarquía de clases del Conversor de Base de Datos es una parte esencial del framework; el programador de la aplicación podría añadir nuevas subclases para adaptarlo a nuevos tipos de mecanismos de almacenamiento persistente o nuevas tablas o ficheros específicos en un mecanismo de almacenamiento existente. La Figura 34.10 muestra la estructura de algunos de los paquetes y clases. Nótese que las clases específicas del proyecto NuevaEra no pertenecen al paquete general de servicios técnicos *Persistencia*.

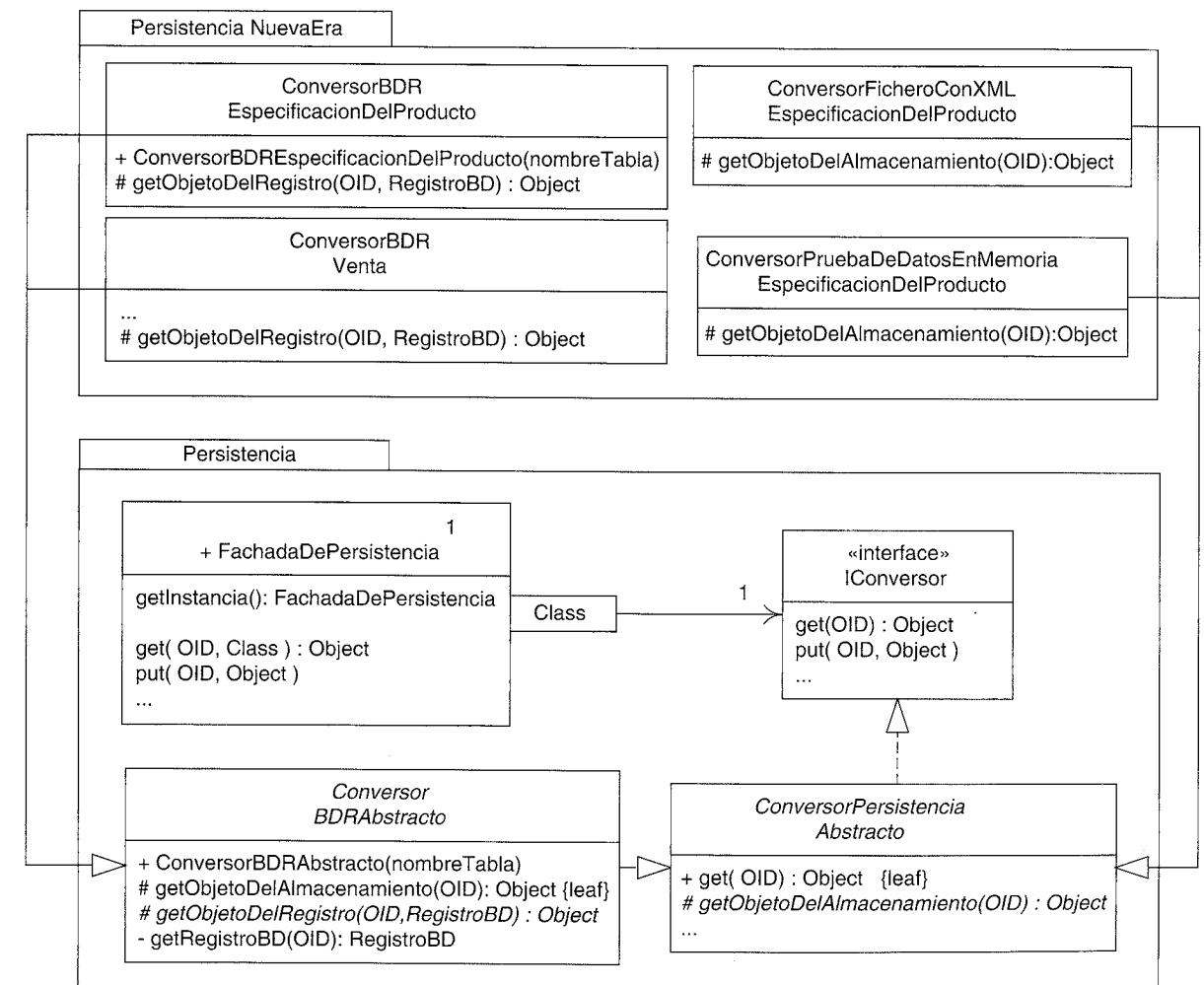


Figura 34.10. El framework de persistencia.

Creo que este diagrama, combinado con la Figura 34.9, ilustra el valor de un lenguaje visual como UML para describir las partes del software; transmite de manera concisa mucha información.

Obsérvese la clase *ConversorPruebaDeDatosEnMemoriaEspecificacionDelProducto*. Tales clases se pueden utilizar para servir objetos con valores definidos directamente en el código para hacer pruebas, sin acceder a ningún almacenamiento persistente externo.

El UP y el Documento de la Arquitectura del Software (SAD)

En cuanto al UP y la documentación, recordemos que el SAD sirve de ayuda para el aprendizaje de futuros desarrolladores, que contiene vistas de la arquitectura con las ideas claves relevantes. La inclusión de diagramas como los de las Figuras 34.9 y 34.10 en el SAD para el proyecto NuevaEra está muy en la línea del tipo de información que un SAD debería contener.

Métodos sincronizados o de guarda en UML

El método *ConversorPersistenciaAbstracto--get* contiene código con secciones críticas que no es seguro en el hilo de ejecución —se podría materializar el mismo objeto de manera concurrente en hilos diferentes—. Como subsistema de servicios técnicos, el servicio de persistencia necesita que se diseñe teniendo presente la seguridad de los hilos. De hecho, el subsistema completo podría estar distribuido en un proceso separado en otro ordenador, transformando la *FachadaDePersistencia* en un objeto servidor remoto, y con muchos hilos ejecutándose simultáneamente en el subsistema, sirviendo a múltiples clientes.

Por tanto, el método debería tener control de la concurrencia de los hilos —si se utiliza Java, se añadirá la palabra clave *synchronized*—. La Figura 34.11 ilustra un método sincronizado en un diagrama de clases.

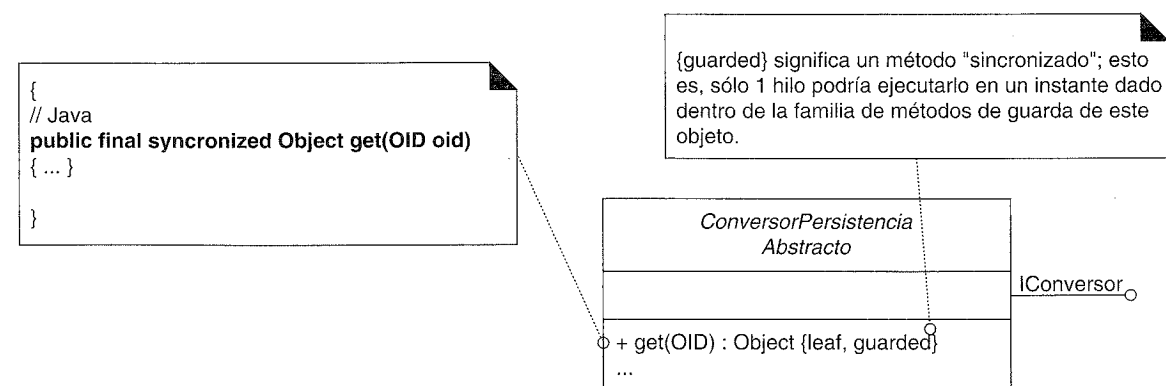


Figura 34.11. Métodos de guarda en UML.

34.13. Configuración de conversores con una FactoriaDeConversores

Análogamente a los ejemplos anteriores de factorías en el caso de estudio, la configuración de la *FachadaDePersistencia* con un conjunto de objetos *IConversor* se puede conseguir con un objeto factoría, *FactoriaDeConversores*. Sin embargo, un pequeño giro, no es conveniente nombrar cada conversor con una operación diferente. Por ejemplo, no es deseable:

```

class FactoriaDeConversores
{
public IConversor getConversorEspecificacionDelProducto() {...}
public IConversor getConversorVenta() {...}
...
}
  
```

Esto no soporta Variaciones Protegidas con respecto a la lista creciente de conversores —y crecerá—. En consecuencia, es preferible lo siguiente:

```

class FactoriaDeConversores
{
public Map getTodosLosConversores() {...}
...
}
  
```

donde las claves del *java.util.Map* (probablemente implementado con un *HashMap*) son los objetos *Class* (los tipos persistentes), y los objetos *IConversor* son los valores.

Entonces, la fachada puede inicializar su colección de objetos *IConversor* como sigue:

```

class FachadaDePersistencia
{
private java.util.Map conversores =
    FactoriaDeConversores.getInstancia().getTodosLosConversores();
...
}
  
```

La factoría puede asignar un conjunto de objetos *IConversor* utilizando un diseño dirigido por los datos. Es decir, la factoría puede leer las propiedades del sistema para descubrir qué clases *IConversor* debe instanciar. Si se utiliza un lenguaje con capacidades de programación reflexiva, como Java, entonces la instanciación se puede basar en la lectura de los nombres de las clases como cadenas de texto, y la utilización de algo como la operación *Class.newInstance* para crear las instancias. De esta manera, se puede reconfigurar el conversor sin cambiar el código fuente.

34.14. Patrón: Gestión de Caché

Es conveniente mantener los objetos materializados en una caché local para mejorar el rendimiento (la materialización es relativamente lenta) y dar soporte a las operaciones de gestión de las transacciones como commit.

El patrón **Gestión de Caché** (*Cache Management*) [BW96] propone que el *Conversor de Base de Datos* sea el responsable de mantener esta caché. Si se utiliza un conversor diferente para cada clase de objetos persistentes, cada conversor puede mantener su propia caché.

Cuando se materializan los objetos, se colocan en la caché, con su OID como clave. Las siguientes peticiones al conversor para obtener un objeto provocará que el conversor busque primero en la caché, evitando de esta manera materializaciones innecesarias.

34.15. Reunir y ocultar sentencias SQL en una clase

Sentencias SQL embebidas en diferentes clases conversores no es un pecado terrible, pero se puede mejorar. Suponga que en lugar de eso:

- Existe una única clase Fabricación Pura (y es un singleton) *OperacionesBDR* donde se reúnen todas las operaciones SQL (SELECT, INSERT,...).
- Las clases conversores de BDR colaboran con ella para obtener un registro de BD o conjunto de registros (por ejemplo, *ResultSet*).
- Su interfaz es algo parecido a lo siguiente:

```
class OperacionesBDR
{
public ResultSet getDatosEspecificacionDelProducto(OID oid){...}
public ResultSet getDatosVenta(OID oid) {...}
...
}
```

De manera que, por ejemplo, un conversor tiene código como éste:

```
class ConversorBDREspecificacionDelProducto extends ConversorPersistenciaAbstracto
{
protected Object getObjetoDelAlmacenamiento(OID oid)
{
ResultSet rs = OperacionesBDR.getInstancia().getDatosEspecificacionDelProducto(oid);

EspecificacionDelProducto ep = new EspecificacionDelProducto();
ep.setPrecio(rs.getDouble("PRECIO"));
ep.setOID(oid);
return ep;
}
```

A partir de esta Fabricación Pura se obtienen los siguientes beneficios:

- Se facilita el mantenimiento y rendimiento que es ajustado por un experto. La optimización del SQL requiere un programador SQL, en lugar de un programador de objetos. Con todo el SQL embebido en esta única clase, facilita al programador en SQL encontrarlo y trabajar con él.
- Encapsulación de los métodos y detalles de acceso. Por ejemplo, SQL construido mediante código podría sustituirse por una llamada a un procedimiento almacenado en la BDR para obtener los datos. O se podría insertar un enfoque más sofisticado basado en los **metadatos** para generar el SQL, en el que se genera el SQL de manera dinámica a partir de la descripción del esquema de los metadatos que se lee de una fuente externa.

Como arquitecto, el aspecto interesante de esta decisión de diseño es que en ella influyen las habilidades del desarrollador. Se llegó a un compromiso entre alta cohesión y la comodidad de un especialista. No todas las decisiones de diseño están motivadas por cuestiones de ingeniería del software “puras” como el acoplamiento y la cohesión.

34.16. Estados transaccionales y el patrón Estado

Las cuestiones relacionadas con el soporte de las transacciones pueden complicarse, pero para mantener las cosas simples de momento —para centrarnos en el patrón GoF Estado— asuma lo siguiente:

- Los objetos persistentes pueden insertarse, eliminarse o modificarse.
- Operar sobre un objeto persistente (por ejemplo, modificarlo) no provoca una actualización inmediata de la base de datos; más bien se debe ejecutar una operación *commit* explícita.

Además, la respuesta a una operación depende del estado de la transacción del objeto. Como ejemplo, las respuestas se podrían mostrar en la máquina de estados de la Figura 34.12.

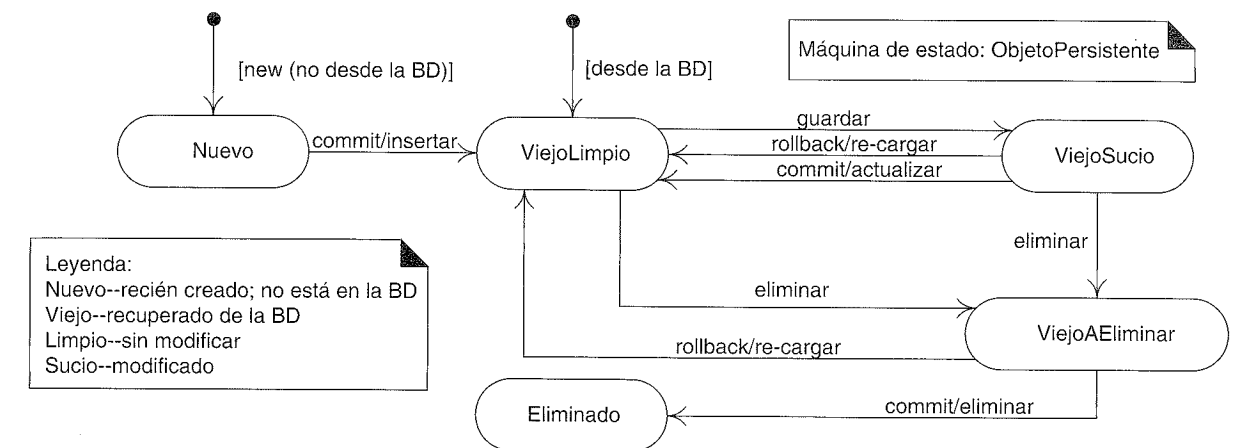


Figura 34.12. Máquina de estados para ObjetoPersistente.

Por ejemplo, un objeto “viejo sucio” (*old dirty*) es aquél recuperado de la base de datos y modificado después. En una operación commit, debería actualizarse en la base de datos —a diferencia del que se encuentra en el estado “viejo limpio”, con el que no debería hacer nada (porque no ha cambiado)—. En el FWP orientado a objetos, cuando se ejecuta una operación eliminar o guardar, no origina que se elimine o se guarde inmediatamente en la base de datos; sino, las transiciones de los objetos persistentes al estado apropiado, esperando una operación commit o rollback para realmente hacer algo.

Como comentario UML, esto es un buen ejemplo de donde es útil una máquina de estados para transmitir información concisa que de otra manera sería difícil de expresar.

En este diseño, asuma que haremos que todas las clases de objetos persistentes extenderán la clase *ObjetoPersistente*⁷, que proporciona los servicios técnicos comunes para la persistencia⁸. Por ejemplo, véase la Figura 34.13.

⁷ [Ambler00b] es una buena referencia acerca de la clase *ObjetoPersistente* y las capas de persistencia, aunque la idea es más antigua.

⁸ Algunas cuestiones relacionadas con la extensión de la clase *ObjetoPersistente* se discuten más tarde. Siempre que una clase de objetos del dominio extienda una clase de los servicios técnicos, se debería hacer una pausa para reflexionar, ya que se está mezclando intereses de la arquitectura (persistencia y lógica de la aplicación).

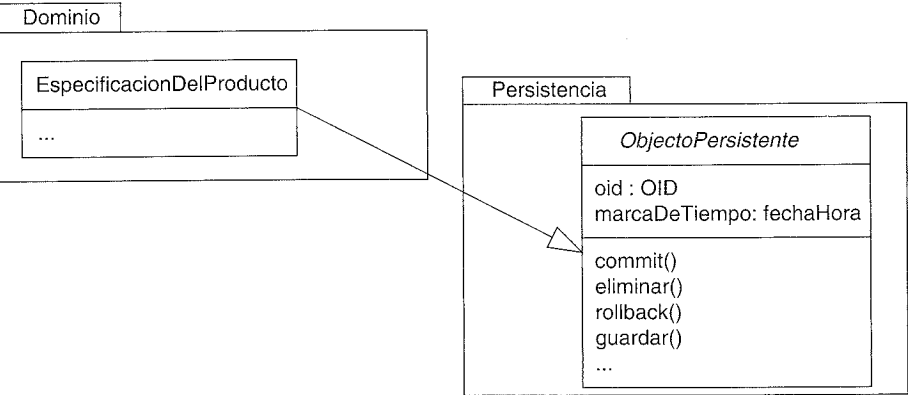


Figura 34.13. Objetos persistentes.

Ahora —y ésta es la cuestión que se resolverá con el patrón Estado— observe que los métodos *commit* y *rollback* requieren una estructura similar de lógica de casos, basada en el código del estado de la transacción. *commit* y *rollback* ejecutan diferentes acciones en cada caso, pero tienen estructuras lógicas similares.

<pre>public void commit() { switch(estado) { case VIEJO_SUCIO: //... break; case VIEJO_LIMPIO: //... break; ... } }</pre>	<pre>public void rollback() { switch(estado) { case VIEJO_SUCIO: //... break; case VIEJO_LIMPIO: //... break; ... } }</pre>
---	---

Una alternativa a esta estructura lógica de casos que se repite es el patrón GoF Estado (*State*).

Estado (*State*)

Contexto/Problema

El comportamiento de un objeto depende de su estado, y sus métodos contienen la lógica de casos que reflejan las acciones condicionales dependientes del estado. ¿Existe una alternativa a la lógica condicional?

Solución

Cree clases estado para cada estado, que implementan una interfaz común. En lugar de definir en el objeto de contexto las operaciones que dependen del estado, deléguelas en su objeto del estado actual. Asegure que el objeto de contexto siempre referencie al objeto estado que refleja su estado actual.

La Figura 34.14 ilustra su aplicación en el subsistema de persistencia.

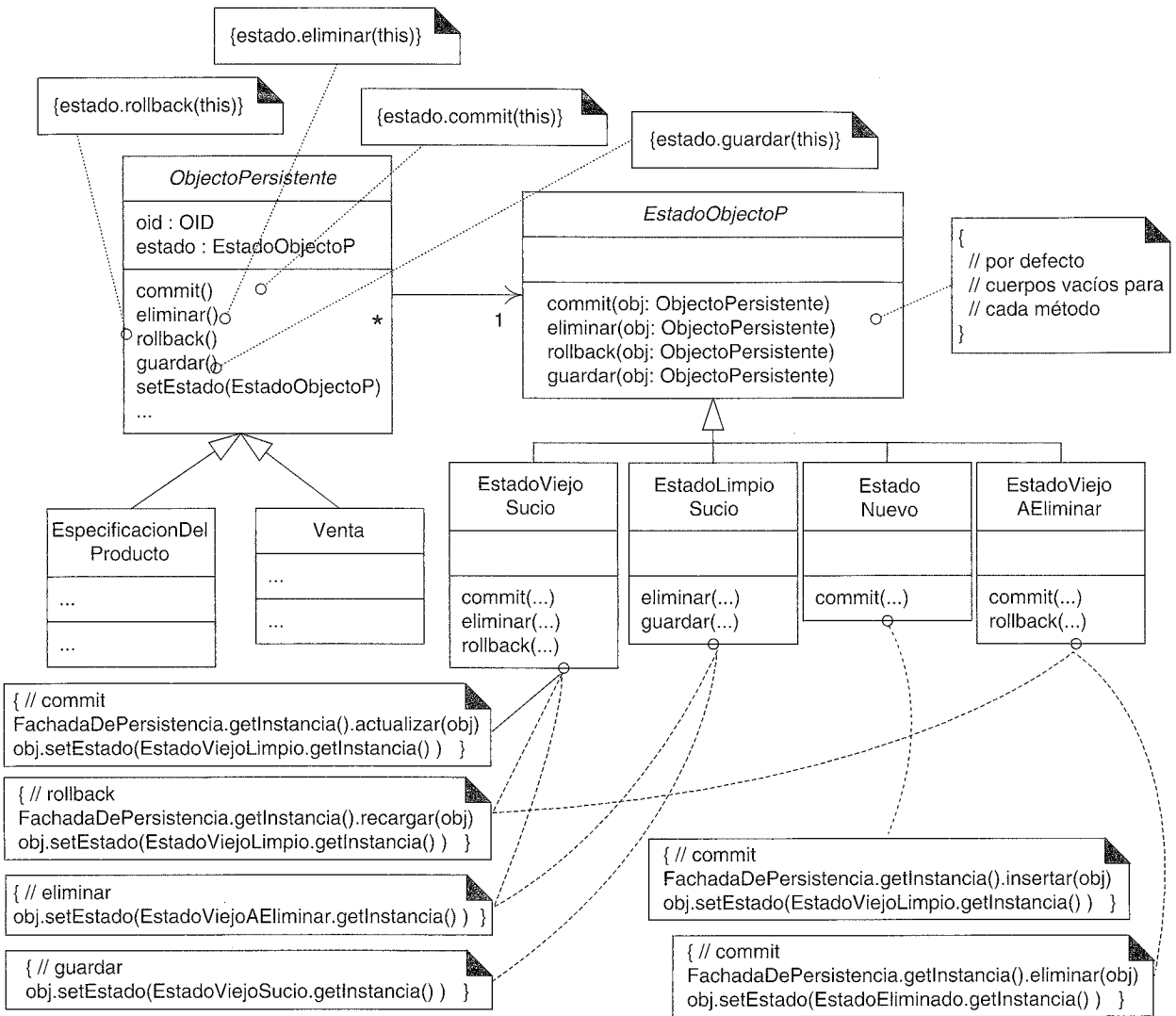


Figura 34.14. Aplicación del patrón Estado⁹.

Los métodos dependientes del estado del *ObjetoPersistente* delegan su ejecución a un objeto estado asociado. Si el objeto de contexto referencia a *EstadoViejoSucio* entonces 1) el método *commit* provocará una actualización de la base de datos, y 2) el objeto de contexto pasará a referenciar a *EstadoViejoLimpio*. Por otro lado, si el objeto de contexto está referenciando a *EstadoViejoLimpio*, se ejecuta el método *commit* heredado como que no hace nada y no hace nada (como se esperaba, puesto que el objeto está limpio).

Observe en la Figura 34.14 que las clases estado y su comportamiento se corresponden con la máquina de estados de la Figura 34.12. El patrón Estado es un mecanismo

⁹ La clase *Borrado* se ha omitido por restricciones de espacio en el diagrama.

para implementar un modelo de transición de estados en software¹⁰. Esto da lugar a que se produzca una transición de un objeto a estados diferentes en respuesta a los eventos.

Un comentario acerca del rendimiento, estos objetos estado no tienen —irónicamente— estado (sin atributos). Por tanto, no es necesario que haya múltiples instancias de una clase —cada una es un singleton—. Miles de objetos persistentes pueden referenciar a la misma instancia *EstadoViejoSucio*, por ejemplo.

34.17. Diseño de una transacción con el Patrón Command

La última sección consideró una vista simplificada de las transacciones. Esta sección amplía la discusión, pero no cubre todas las cuestiones relacionadas con el diseño de las transacciones. Informalmente, una transacción es una unidad de trabajo —un conjunto de tareas— cuyas tareas deben completarse todas con éxito, o no se debe completar ninguna. Es decir, la terminación es atómica.

En cuanto a los servicios de persistencia, las tareas de una transacción incluyen la inserción, actualización y eliminación de los objetos. Una transacción podría contener dos inserciones, una actualización y tres eliminaciones, por ejemplo. Para representar esto, se añade una clase *Transaccion* [Ambler00b]¹¹. Como se señala en [Fowler01], el orden de las tareas de la base de datos dentro de una transacción puede influir en su éxito (y rendimiento).

Por ejemplo:

1. Suponga que la base de datos tiene una restricción de integridad referencial de manera que cuando se actualiza un registro en la TablaA que contiene una clave ajena a un registro de la TablaB, la base de datos requiere que el registro de la TablaB ya exista.
2. Suponga que una transacción contiene una tarea INSERT para añadir el registro de la TablaB, y una tarea UPDATE para actualizar el registro de la TablaA. Si se ejecuta UPDATE antes de INSERT, surge un error de integridad referencial.

La ordenación de las tareas de la base de datos puede ayudar. Algunas cuestiones de la ordenación son específicas del esquema, pero una estrategia general es primero realizar las inserciones, luego las actualizaciones y entonces las eliminaciones.

Tenga en cuenta que el orden en el que una aplicación añade las tareas a una transacción podría no reflejar el mejor orden de ejecución. Las tareas necesitan que se ordenen justo antes de su ejecución.

Esto nos lleva a otro patrón GoF: Command¹².

¹⁰ Existen otros, entre los que se encuentran lógica condicional construida mediante código, intérpretes de máquinas de estados, y generadores de código dirigidos por tablas de estados.

¹¹ Se denomina UnidadDeTrabajo en [Fowler01].

¹² N. del T. No se ha traducido por el mismo motivo que en el caso del patrón Singleton.

Contexto/Problema

¿Cómo gestionar las solicitudes o tareas que necesitan funciones como ordenar (estableciendo prioridades), poner en cola, retrasar, anotar en registro o deshacer?

Solución

Defina una clase por cada tarea que implemente una interfaz común.

Éste es un patrón sencillo con muchas aplicaciones útiles; las acciones se convierten en objetos, y de esta manera se pueden ordenar, anotar en un registro, poner en cola, etcétera. Por ejemplo, en el FWP, la Figura 34.15 muestra las clases Command (o tareas) para las operaciones de la base de datos.

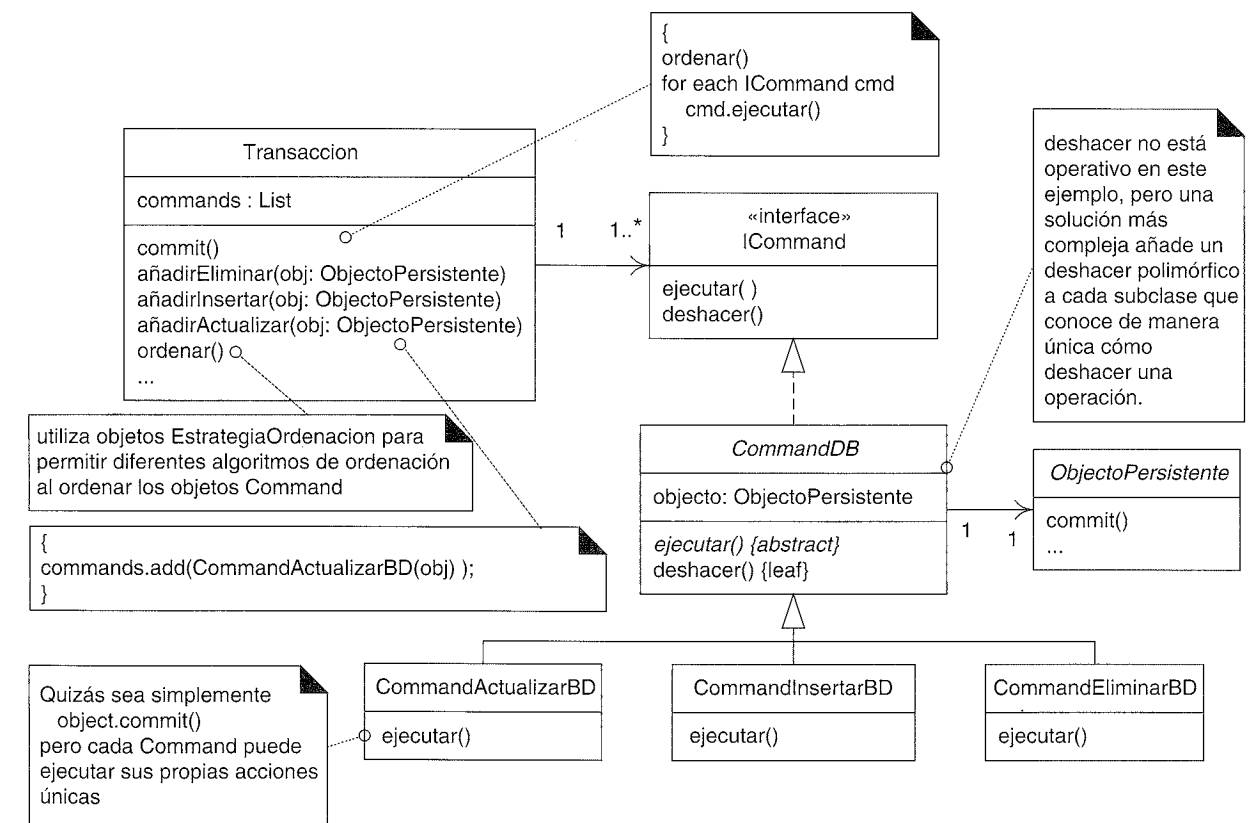


Figura 34.15. Clases Command para las operaciones de la base de datos.

Hay mucho más para completar una solución de transacción, pero la idea clave de esta sección es representar cada tarea o acción de la transacción como un objeto con un método *ejecutar* polimórfico; esto hace accesible un mundo de flexibilidad tratando la respuesta como un objeto en sí.

El ejemplo prototípico de Command es el de las acciones de una GUI, como cortar y pegar. Por ejemplo, el método *ejecutar* de *CommandCortar* realiza la acción cortar, y su

método *deshacer* deshace el corte. *CommandCortar* también retendrá los datos necesarios para llevar a cabo la acción de deshacer. Todos los objetos *Command* de la GUI se pueden mantener en una pila que recoge la historia de las ejecuciones, de manera que se puedan desapilar por turnos, y deshacerse cada una.

Otro uso típico del patrón *Command* es para la gestión de las peticiones del lado del servidor. Cuando un objeto servidor recibe un mensaje (remoto), crea un objeto *Command* para esta petición, y lo entrega al *CommandProcesador* [BMRSS96], que puede poner en cola, anotar en un registro, priorizar y ejecutar los objetos *Command*.

34.18. Materialización perezosa con un Proxy Virtual

Algunas veces es conveniente diferir la materialización de los objetos hasta que sea absolutamente necesario, normalmente por razones de rendimiento. Por ejemplo, suponga que los objetos *EspecificacionDelProducto* referencian a un objeto *Fabricante*, pero rara vez es necesario que se materialice desde la base de datos. Sólo provocan una petición de la información del fabricante escenarios poco frecuentes, como los escenarios relacionados con las rebajas de los fabricantes en el que se necesita el nombre y la dirección de la compañía.

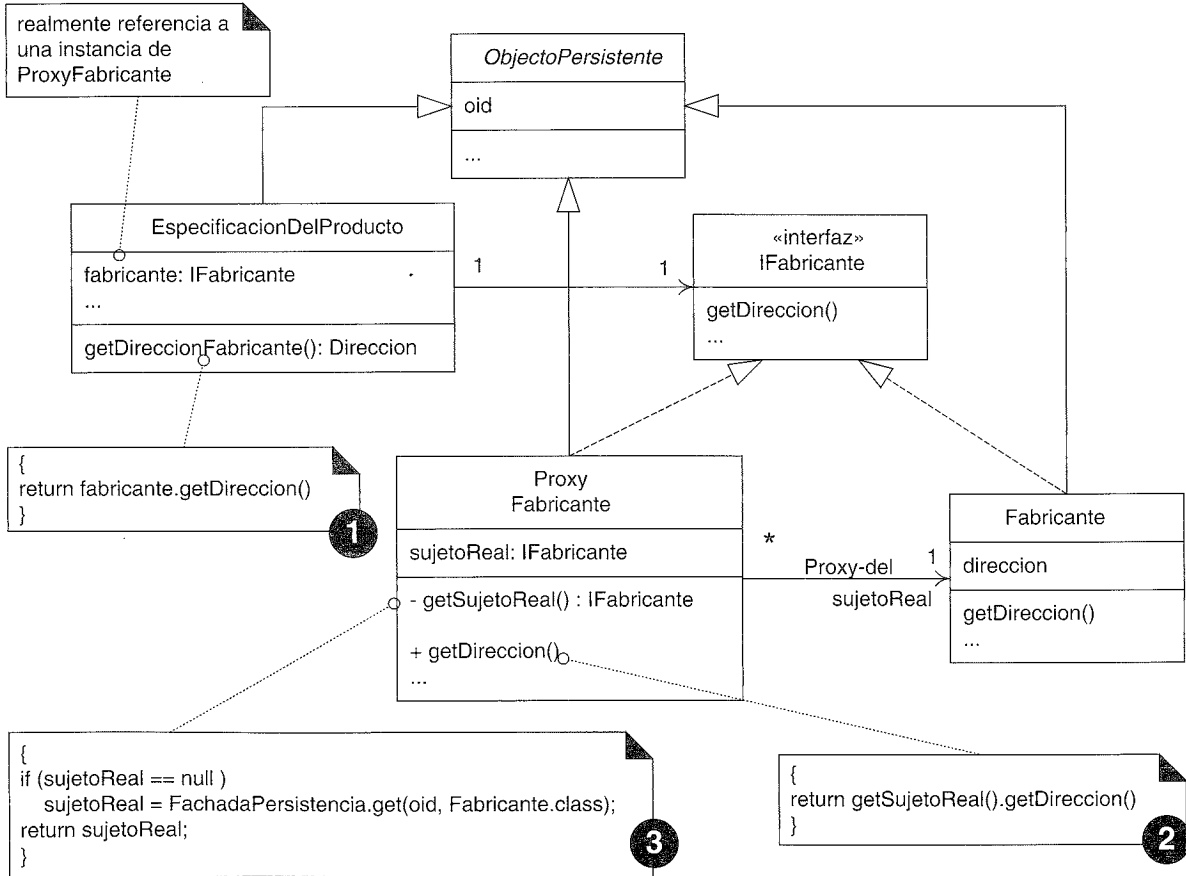


Figura 34.16. Proxy Virtual del Fabricante.

La materialización diferida de los objetos “hijos” se conoce como **materialización perezosa**. La materialización perezosa se puede implementar utilizando el patrón *GoF Proxy Virtual* —una de las muchas variaciones del *Proxy*—.

Un **Proxy Virtual** es un proxy para otro objeto (el *sujeto al que se quiere acceder realmente*) que materializa el sujeto real cuando se referencia por primera vez; por tanto, implementa la materialización perezosa. Es un objeto ligero que simboliza el objeto “real” que puede o no ser materializado.

Un ejemplo concreto del patrón *Proxy Virtual* con la *EspecificacionDelProducto* y el *Fabricante* se muestra en la Figura 34.16. Este diseño se basa en la suposición de que los proxies conocen el *OID* de sus sujetos reales, y cuando se requiere la materialización, se utiliza el *OID* para ayudar a identificar y recuperar el sujeto real.

Nótese que la *EspecificacionDelProducto* tiene visibilidad de atributo de la instancia *IFabricante*. El *Fabricante* para esta *EspecificacionDelProducto* podría no haberse materializado todavía en memoria. Cuando la *EspecificacionDelProducto* envía un mensaje *getDireccion* al *ProxyFabricante* (creyendo que es el objeto fabricante materializado), el proxy materializa el *Fabricante* real, utilizando el *OID* del *Fabricante* para recuperarlo y materializarlo.

Implementación de un Proxy Virtual

La implementación de un *Proxy Virtual* varía según el lenguaje. Los detalles quedan fuera del alcance de este capítulo, pero a continuación presentamos un resumen:

Lenguaje	Implementación del Proxy Virtual
C++	Define una clase plantilla de puntero inteligente (<i>smart pointer</i>). Realmente no se necesita la definición de la interfaz <i>IFabricante</i> .
Java	Se implementa la clase <i>ProxyFabricante</i> . Se define la interfaz <i>IFabricante</i> . Sin embargo, normalmente no se codifican manualmente. Más bien, uno crea un generador de código que analiza las clases a las que se quiere acceder realmente (ej. <i>Fabricante</i>) y genera <i>IFabricante</i> y <i>ProxyFabricante</i> . Otra alternativa de Java es utilizar el API <i>Proxy Dinámico</i> .
Smalltalk	Definir un <i>Proxy Morphing Virtual</i> (o <i>Proxy Fantasma</i>), que utiliza <i>#doesNotUnderstand</i> : y <i>#become</i> : para transformar en el sujeto real. No se necesita la definición de <i>IFabricante</i> .

¿Quién crea el Proxy Virtual?

Observe en la Figura 34.16 que el *ProxyFabricante* colabora con la *FachadaDePersistencia* para materializar su sujeto real. ¿Pero quién crea el *ProxyFabricante*? Respuesta: la clase que establece la correspondencia entre la *EspecificacionDelProducto* y la base de datos. Esta clase es la responsable de decidir, cuándo se materializa un objeto, cuál de los objetos “hijos” debería también materializarse de manera impaciente, y cuáles deberían materializarse de manera perezosa con un proxy.

Considere estas soluciones alternativas: una utiliza materialización impaciente, y la otra materialización perezosa.

//MATERIALIZACIÓN IMPACIENTE DEL FABRICANTE

```
class ConversorBDRespecificacionDelProducto extends ConversorPersistenciaAbstracto
{
protected Object getObjetoDelAlmacenamiento(OID oid)
{
ResultSet rs =
OperacionesBDR.getInstancia().getDatosEspecificacionDelProducto(oid);

EspecificacionDelProducto ep = new EspecificacionDelProducto();
ep.setPrecio(rs.getDouble("PRECIO"));

//aquí está la esencia

String claveAjenaFabricante = rs.getString("FAB_OID");
OID oidFab = new OID(claveAjenaFabricante);
ep.setFabricante((IFabricante)
FachadaDePersistencia.getInstancia().get(oidFab, Fabricante.class);
...
}
```

A continuación presentamos la solución de materialización perezosa:

//MATERIALIZACIÓN PEREZOSA DEL FABRICANTE

```
class ConversorBDRespecificacionDelProducto extends ConversorPersistenciaAbstracto
{
protected Object getObjetoDelAlmacenamiento(OID oid)
{
ResultSet rs =
OperacionesBDR.getInstancia().getDatosEspecificacionDelProducto(oid);

EspecificacionDelProducto ep = new EspecificacionDelProducto();
ep.setPrecio(rs.getDouble("PRECIO"));

//aquí está la esencia

String claveAjenaFabricante = rs.getString("FAB_OID");
OID oidFab = new OID(claveAjenaFabricante);
ep.setFabricante(new ProxyFabricante(oidFab));
...
}
```

34.19. Cómo representar las relaciones en tablas

El código de la sección anterior confía en la clave ajena FAB_OID en la tabla ESPEC_PRODUCTO para conectar con un registro de la tabla FABRICANTE. Esto pone de relieve la pregunta: ¿cómo se representan las relaciones de los objetos en el modelo relacional?

La respuesta se da en el patrón **Representación de las Relaciones de los Objetos en Tablas** [BW96], que propone lo siguiente:

- Asociaciones **uno-a-uno**
 - Colocar una clave ajena OID en una o ambas tablas que representan los objetos de la relación.
 - O, crear una tabla asociación que recoja los OIDs de cada uno de los objetos de la relación.
- Asociaciones **uno-a-muchos**, como una colección
 - Crear una tabla asociativa que registre los OIDs de cada uno de los objetos de la asociación.
- Asociaciones **muchos-a-muchos**
 - Crear una tabla asociativa que registre los OIDs de cada uno de los objetos de la asociación.

34.20. Superclase ObjetoPersistente y separación de intereses

Una solución de diseño parcial típica para proporcionar persistencia a los objetos es crear una clase abstracta de servicios técnicos *ObjetoPersistente* de la que heredan todos los objetos persistentes (ver Figura 34.17). Tal clase, normalmente define atributos para la persistencia, como un OID único, y métodos para almacenarlos en la base de datos.

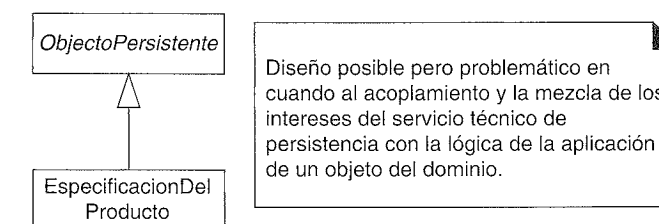


Figura 34.17. Problemas con la superclase ObjetoPersistente.

Esto no es incorrecto, pero adolece de la debilidad del acoplamiento entre la clase y la clase *ObjetoPersistente* —las clases del dominio terminan extendiendo una clase de los servicios técnicos—.

Este diseño no ilustra una clara separación de intereses. Más bien, se mezclan los intereses de los servicios técnicos con los intereses de la lógica del negocio de la capa del dominio en virtud de su extensión.

Por otro lado, la “separación de intereses” no es una virtud absoluta que se debe seguir a cualquier precio. Como se discutió cuando se presentaron las Variaciones Protegidas, los diseñadores necesitan escoger sus batallas en los puntos en los que realmente sea probable una inestabilidad costosa. Si en una aplicación particular, haciendo que la clase sea una subclase de *ObjetoPersistente* nos lleva a una solución ordenada y fácil y no da lugar a problemas de diseño o mantenimiento a largo plazo, ¿por qué no? La respuesta está en entender la evolución de los requisitos y el diseño de la aplicación. También influye el lenguaje: aquéllos con herencia simple (como Java) *consumen* su única preciada superclase.

34.21. Cuestiones sin resolver

Esto ha sido una breve introducción a los problemas y soluciones de diseño de un framework y servicio de persistencia. Se han encubierto muchas cuestiones importantes, entre las que se encuentran:

- Desmaterialización de los objetos.
 - Brevemente, los conversores deben definir los métodos *putObjetoEnAlmacenamiento*. La desmaterialización de jerarquías de composición requiere la colaboración entre múltiples conversores y el mantenimiento de tablas asociativas (si se utiliza un BDR).
- Materialización y desmaterialización de las colecciones.
- Consultas a grupos de objetos.
- Gestión completa de transacciones.
- Gestión de los errores cuando falla una operación de la base de datos.
- Acceso multiusuario y estrategias de bloqueo.
- Seguridad—control del acceso a la base de datos.

SOBRE EL DIBUJO DE DIAGRAMAS Y LAS HERRAMIENTAS

Las burbujas no explotan.

Bertrand Meyer

Objetivos

- Aprender consejos para dibujar los diagramas UML en un proyecto.
 - Ilustrar algunas funciones comunes de las herramientas CASE para UML.
-

Introducción

En un proyecto real, la realización del análisis y diseño mientras se dibujan los diagramas UML no ocurre ordenadamente como en las páginas de un libro. Tiene lugar en el contexto de un equipo de desarrollo de software ocupado, que trabaja en oficinas o habitaciones, hacen garabatos en pizarras y quizás utilizando una herramienta, y a menudo con tendencia a querer comenzar a programar en lugar de trabajar con detalle algunos detalles mediante la realización de diagramas. Si la herramienta para UML, o el proceso para dibujar los diagramas, resultan molestos y engorrosos, o parece que tiene menos valor que la programación, se evitarán.

Este capítulo propone mantener un equilibrio entre la programación y el dibujo de diagramas, y fomentar un entorno de soporte para hacer que la tarea de dibujar los diagramas sea cómoda y útil en lugar de difícil.

35.1. Diseño especulativo y razonamiento visual

Los diseños que se ilustran mediante diagramas UML serán incompletos, y sólo servirán como “trampolín” para la programación. Demasiados diagramas antes de programar nos lleva a una pérdida de tiempo en direcciones de diseño especulativas, o una pérdida de

tiempo obsesionados con herramientas para UML. No hay nada como el código real para decirte qué funciona. Bertrand Meyer lo dice mejor: “Las burbujas no explotan”.

No obstante, promuevo enérgicamente que se anticipe *alguna* idea mediante la realización de diagramas antes de programar, y se que puede ser útil, especialmente para explorar las estrategias de diseño más importantes. La pregunta interesante es “¿cuánto tiempo se tiene que dedicar a dibujar los diagramas antes de programar?” En parte la respuesta está en función de la experiencia y estilo cognitivo de los diseñadores.

Algunas personas son buenas para el razonamiento espacial/visual y expresar sus pensamientos acerca del diseño del software en un lenguaje visual complementa su naturaleza; otros no. Un gran porcentaje del cerebro está dedicado al razonamiento y procesamiento visual o simbólico, en lugar de al procesamiento textual (código). Los lenguajes visuales como UML juegan con una capacidad natural de la mente en la mayoría de la gente. A aquellos a los que se les ha enseñado UML obviamente les resultará más sencillo que a los que no. Y, en general, los diseñadores de objetos con más experiencia pueden diseñar de manera efectiva dibujando sin perderse en especulaciones no realistas, debido a su experiencia y juicio. Aplicados por expertos, los diagramas pueden ayudar a un grupo a moverse más rápidamente hacia un diseño apropiado, debido a la capacidad de ignorar los detalles y centrarse en los verdaderos problemas.

Una excepción a esta sugerencia de dibujar los diagramas “ligeros” son los sistemas que se modelan de manera natural como máquinas de estados. Hay algunas herramientas CASE que pueden hacer un trabajo impresionante generando código completo basado en máquinas de estados UML detalladas para todas las clases. Pero no todos los dominios encajan de manera natural en un enfoque fuertemente centrado en las máquinas de estados; por ejemplo, las máquinas de control y telecomunicaciones a menudo se ajustan bien, los sistemas de información de gestión normalmente no.

35.2. Sugerencias para dibujar los diagramas de UML en el proceso de desarrollo

Nivel de esfuerzo

Como guía, considere realizar los diagramas en parejas durante los siguientes periodos, antes de programar de manera seria en la iteración.

Iteración de 2-semanas	De medio día a un día casi al comienzo de la iteración (ej. lunes o martes)
Iteración de 4-semanas	Uno o dos días cerca del comienzo

En ambos casos, la realización de los diagramas no tiene que parar después de este esfuerzo inicial enfocado a este fin, los desarrolladores podrían dirigirse —idealmente en parejas— “a la pizarra” durante sesiones cortas para esbozar ideas antes de programar más. Y podrían realizar otra sesión más larga de medio día a mitad de la iteración, cuando tropiecen con un problema complejo en el ámbito de su tarea inicial, o terminen su primera tarea y pasen a la segunda.

Otras sugerencias

- Dibuje en parejas, no a solas. Lo más importante, la sinergia nos conduce a diseños mejores. En segundo lugar, la pareja aprende rápidamente técnicas de diseño el uno del otro y, por tanto, ambos llegan a ser mejores diseñadores. Es difícil mejorar como diseñador de software cuando uno diseña individualmente. Cambie de manera regular el compañero de dibujo/diseño para estar periodos de tiempo amplios expuestos al conocimiento de los demás.
- Para aclarar un punto al que se ha hecho alusión varias veces, en los procesos iterativos (como el UP), los programadores son también los diseñadores; no hay un equipo separado que dibuja los diseños y se los entrega a los programadores. Los desarrolladores se ponen su sombrero de UML, y dibujan un poco. Entonces se ponen su sombrero de programador e implementan, y continúan diseñando mientras programan.
- Si hay diez desarrolladores, suponga que hay cinco equipos de dibujo trabajando durante un día en pizarras diferentes. Si el arquitecto dedica tiempo a rotar por los cinco equipos, llegará a ver puntos de dependencia, conflictos e ideas de un equipo que sirven para otro. El arquitecto entonces puede actuar de enlace para llegar a una armonía entre los diseños y clarificar las dependencias.
- Contrate un escritor técnico para el proyecto y enséñele algo de la notación UML y los conceptos básicos del A/DOO (de manera que pueda entender el contexto). Disponga de la ayuda del escritor en la realización del “trabajo engorroso” con las herramientas CASE para UML, en los procesos de ingeniería inversa generando los diagramas a partir del código, en la impresión y presentación de grandes diagramas impresos con plotter, etcétera. Los desarrolladores dedicarán su tiempo (más caro) a hacer lo que hacen mejor: idear diseños y programar. Un escritor técnico les ayuda realizando la gestión de los diagramas, además de las verdaderas responsabilidades de la escritura técnica tales como el trabajo en el documento para el usuario final. Esto se conoce como el patrón Analista Mercenario [Coplien95a].
- Organice el área de desarrollo con muchas pizarras amplias distribuidas muy cerca unas de otras.
- Generalizando, maximice el entorno de trabajo para dibujar cómodamente en las paredes. Cree un entorno “amigable” para dibujar y colgar los diagramas. No puede esperar que se consiga una cultura de modelado visual con éxito en un entorno donde los desarrolladores se están peleando por dibujar en pizarras pequeñas de 60x90 cm, monitores de ordenador de tamaño corriente o trozos de papel. Para dibujar de manera cómoda hacen falta espacios muy amplios y abiertos —físicos o virtuales—.
- Como accesorio de las pizarras, utilice finas hojas blancas de plástico con “adherencia estática” (vienen en paquetes de 20 o más) que se pueden colocar en las paredes; éstas se encuentran disponibles en muchas papelerías. Las hojas permanecen unidas a la pared mediante adherencia estática, y se puede utilizar como una pizarra con un rotulador que se pueda borrar. Se puede empapelar una pared con estas hojas para crear “pizarras” temporales, masivas. He dirigido grupos donde

hemos empapelado todas las paredes —de arriba abajo— de la sala del proyecto con estas hojas, y lo encontramos de gran ayuda para la comunicación.

- Si utiliza una pizarra para los dibujos de UML, utilice un dispositivo (existe al menos uno en el mercado) que capture los dibujos hechos a mano y los transmita a un ordenador como un fichero gráfico. Un diseño involucra una parte receptora en la esquina de la pizarra que captura la imagen para enviarla al ordenador y unas fundas especiales transmisoras en las que se insertan los rotuladores.
- De manera alternativa, si utiliza una pizarra para los dibujos de UML, utilice una cámara digital para capturar las imágenes, normalmente en dos o tres secciones. Ésta es una práctica para dibujar los diagramas, bastante común y efectiva.
- Otra tecnología de pizarra es una pizarra “que imprime”, que normalmente es una pizarra de dos lados con un escáner y una impresora conectada. También son útiles.
- Imprima las imágenes UML dibujadas a mano (capturadas mediante una cámara o un dispositivo de pizarra) y cuélguelas de manera visible *muy* cerca de las estaciones de trabajo para la programación. Lo importante de los diagramas es inspirar la dirección de la programación, de manera que los programadores puedan echarles un vistazo mientras están programando. Si se dibujan pero “se entierran”, no tenía mucho sentido dibujarlos.
- Si dibuja los diagramas UML a mano, utilice una notación simple elegida para acelerar y facilitar la realización de los diagramas.
- Incluso si lleva a cabo el diseño creativo en una pizarra, utilice una herramienta CASE para UML para generar los diagramas de paquetes y de clases mediante un proceso de ingeniería inversa a partir del código fuente (de la última iteración) por lo menos al comienzo de cada iteración posterior. Entonces, utilice estos diagramas generados mediante un proceso de ingeniería inversa como punto de partida para el siguiente diseño creativo.
- Imprima periódicamente los diagramas de paquetes y de clases interesantes/ines- tables/difíciles, generados recientemente mediante un proceso de ingeniería in- versa, en un tamaño ampliado (para facilitar que se vea) con un plotter que pueda imprimir en papel continuo de de 90 ó 120 cm de ancho. Cuélguelos en las paredes muy cerca de los desarrolladores como ayuda visual. El escritor técnico, si está presente, puede hacer este trabajo. Anime a los desarrolladores a que dibujen y ha- gan garabatos sobre los diagramas durante el trabajo de diseño creativo.
- Con respecto a la ingeniería inversa, pocas herramientas para UML soportan el proceso de ingeniería inversa para generar diagramas de secuencia —no única- mente diagramas de clases— a partir del código fuente. Si está disponible, utilice una para generar los diagramas de secuencia para los escenarios significativos para la arquitectura, imprímalos a tamaño grande con el plotter, y cuélguelos para fa- cilitar que se vean.
- Si utiliza una herramienta CASE para UML (de hecho, hágalo para todo el trabajo de programación), utilice una estación de trabajo con un monitor dual (dos moni- tores de pantalla plana de tamaño ordinario son más baratas que un único monitor

de pantalla plana de tamaño grande). Los sistemas operativos modernos soportan (al menos) tarjetas de vídeo duales y de esta manera dos monitores. Organice sus ventanas dentro de la herramienta para UML entre los dos monitores. ¿Por qué? Un monitor pequeño reprime psicológicamente y creativamente en lo que se refiere a los dibujos y lenguajes visuales porque el espacio del área visual es demasiado pe- queño y estrecho. Un desarrollador puede caer en la aptitud desmotivada de “el di- seño ha terminado porque la ventana está llena, y parece demasiado desordenado”.

- Cuando utilice una herramienta CASE para UML y realice el diseño creativo por parejas o pequeños grupos, conecte dos proyectores de ordenador a las dos tarjetas de vídeo del ordenador y alinee las proyecciones en la pared de manera que el equipo pueda ver y trabajar con un espacio amplio para el área visual. Un área pe- queña y unos diagramas difíciles de ver son impedimentos psicológicos y sociales para el diseño visual en colaboración de los pequeños grupos.

35.3. Herramientas y características de ejemplo

Este libro es imparcial respecto a la herramienta

Sería algo extraño no mencionar ninguna herramienta CASE (*Computer Aid Software Engineering*, ingeniería del software asistida por ordenador) para UML, porque este li- bro trata en parte de la realización de dibujos en UML, que tiene lugar con una herra- mienta CASE, o en una pizarra. Al mismo tiempo, no se pueden cubrir por igual todas las herramientas, y las evaluaciones apropiadas quedan fuera del alcance de este libro. Para ser imparcial:

Este libro no avala ninguna herramienta CASE para UML. Los siguientes ejemplos sólo sir- ven para ilustrar algunas de las características típicas y claves que podemos encontrar en las herramientas CASE para UML.

Las herramientas se ajustan de manera inconsistente a UML

Pocas herramientas dibujan toda la notación de UML correctamente, y conforme con la versión actual de la especificación de UML —realmente con cualquier versión—. Aun- que esto estaría bien, no debería ser un factor en la elección de una herramienta, puesto que es mucho más importante su funcionalidad y facilidad de uso.

Ejemplo uno

En las Figuras 35.1 y 35.2 se utiliza Together de TogetherSoft para ilustrar y definir dos funciones claves de una herramienta CASE para UML: **ingeniería directa e inversa**. Es- tas funciones son lo que distinguen esencialmente una herramienta CASE para UML de una herramienta para dibujar.

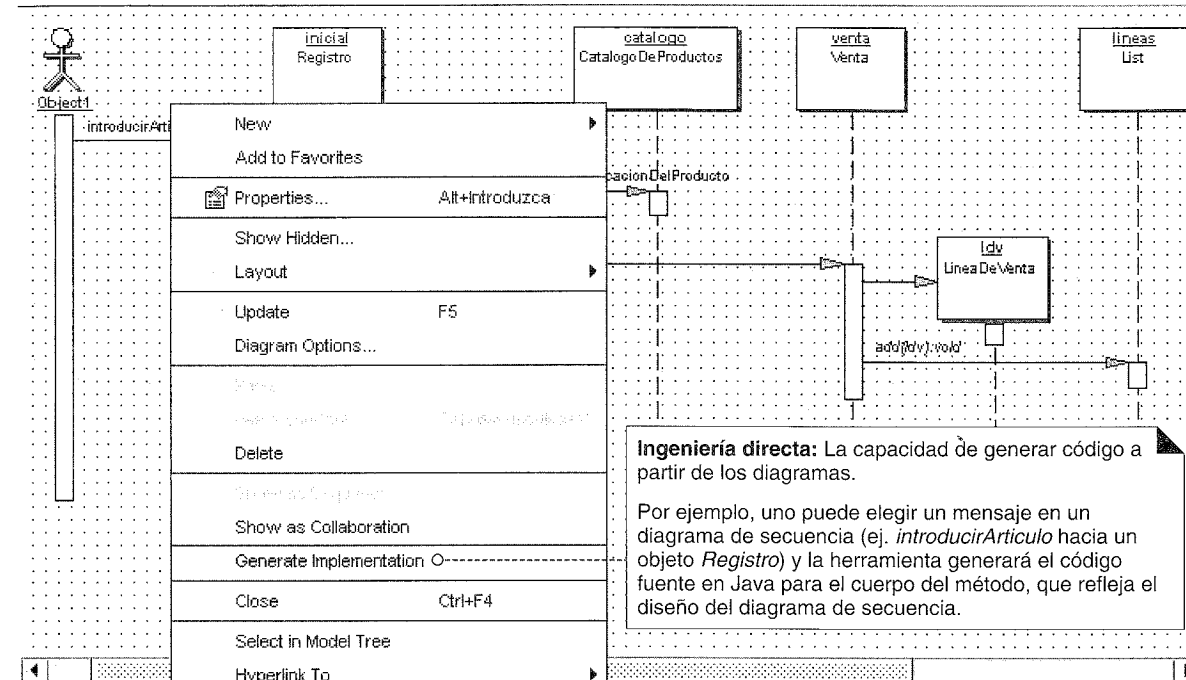


Figura 35.1. Ingeniería directa.

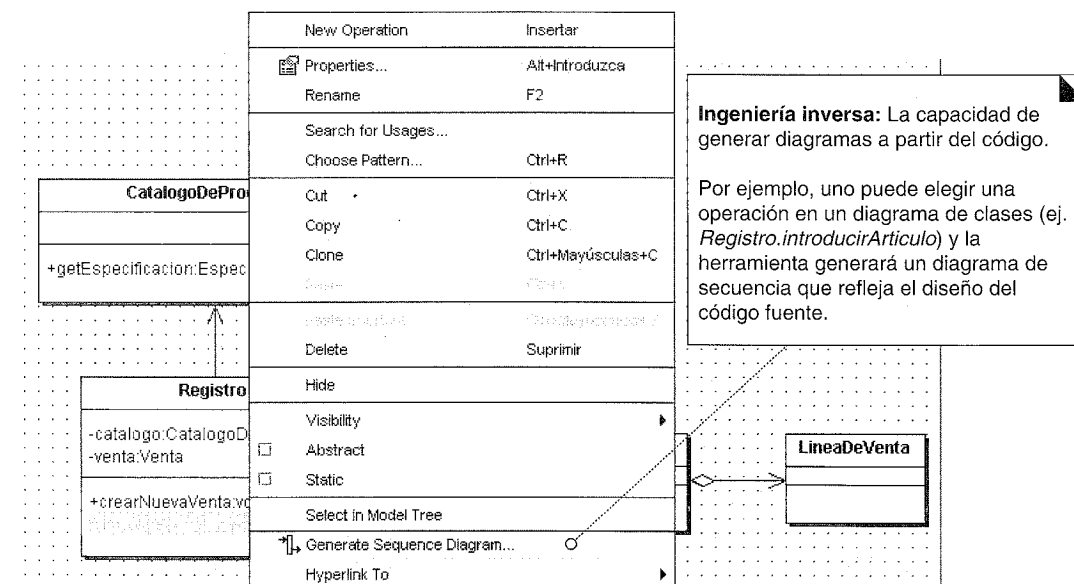


Figura 35.2. Ingeniería inversa.

35.4. Ejemplo dos

En las Figuras 35.3 y 35.4 se utiliza Rational Rose para ilustrar algunas otras funciones básicas de una herramienta CASE para UML.

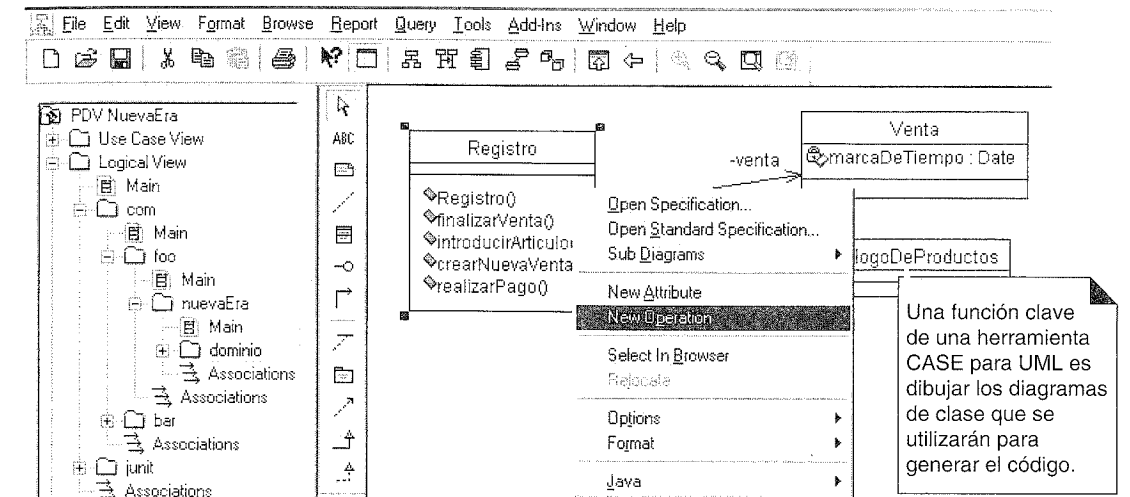


Figura 35.3. Creación de diagramas de clase.

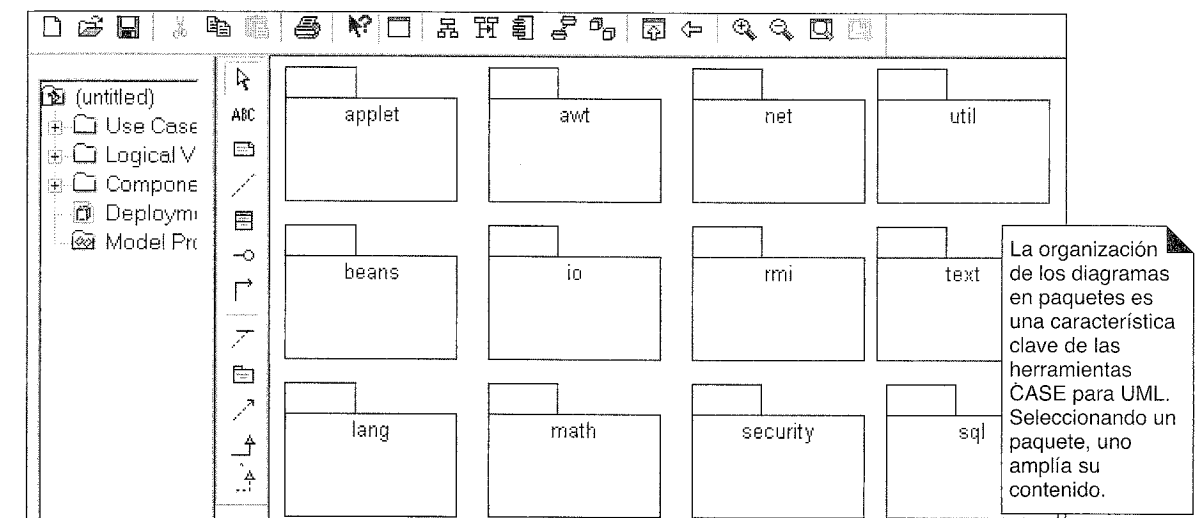


Figura 35.4. Gestión de paquetes.

Peticiones a los vendedores de herramientas CASE para UML

Sugiero que los consumidores hagan cuatro peticiones a los vendedores de herramientas CASE para UML:

1. Implementación correcta de la notación actual de UML en la herramienta.
2. Que el propio equipo de desarrollo de la herramienta CASE haya dibujado, leído y revisado seriamente diagramas UML (incluyendo el proceso de ingeniería inversa de los diagramas) en el proceso de construcción de la propia herramienta para UML.

- 3. Utilizar la versión N de la herramienta para UML para crear la versión N+1.
- 4. Proporcionar el soporte para los procesos de ingeniería directa e inversa de los diagramas de secuencia; la mayoría de las herramientas sólo lo soportan para el diagrama de clases.

Microsoft aboga por que los creadores de las herramientas “coman su propia comida para perros”. Buen consejo.

INTRODUCCIÓN A CUESTIONES RELACIONADAS CON LA PLANIFICACIÓN ITERATIVA Y EL PROYECTO

La predicción es muy difícil, especialmente si es acerca del futuro.
anónimo

Objetivos

- Priorizar los requisitos y los riesgos.
 - Comparar y contrastar la planificación adaptable y predictiva.
 - Definir el Plan de Fase y Plan de Iteración del UP.
 - Introducir las herramientas para mantener la traza de los requisitos para el desarrollo iterativo.
 - Sugerir el modo de organizar los artefactos del proyecto.
-

Introducción

Las cuestiones relacionadas con la gestión y planificación del proyecto son temas amplios, pero sirve de ayuda abordar un breve estudio sobre algunas preguntas claves relacionadas con el desarrollo iterativo y el UP, como:

- ¿Qué hacemos en la siguiente iteración?
- ¿Cómo mantener la traza de los requisitos en el desarrollo iterativo?
- ¿Cómo organizamos los artefactos del proyecto?

36.1. Priorización de los requisitos

Criterios que dirigen las primeras iteraciones: riesgo, cobertura, naturaleza crítica, desarrollo de habilidades

¿Qué hacemos en las primeras iteraciones? Organice los requisitos y las iteraciones de acuerdo con el riesgo, cobertura y naturaleza crítica [Kruchten00]. El riesgo del requisito comprende tanto la complejidad técnica como otros factores, como incertidumbre del esfuerzo, especificación pobre, problemas políticos, o facilidad de uso. Debemos diferenciar la priorización de los riesgos de los requisitos de la priorización de los riesgos del proyecto, que se estudiará en una sección posterior.

La cobertura implica que todas las partes importantes del sistema por lo menos se han tratado brevemente en las primeras iteraciones —quizás una implementación “en anchura y superficial” a través de muchos componentes—. La naturaleza crítica se refiere a las funciones de alto valor para el negocio; es decir, las funciones principales deberían tener al menos implementaciones parciales para los escenarios principales de éxito en las primeras iteraciones, incluso si no son técnicamente arriesgadas.

En algunos proyectos, otro criterio es el desarrollo de habilidades —un objetivo es ayudar al equipo a dominar nuevas habilidades como adoptar las tecnologías de objetos—. En tales proyectos, el desarrollo de habilidades es un factor de gran peso para establecer las prioridades, que tiende a reorganizar las iteraciones en requisitos de menos riesgo o más simples en las primeras iteraciones, motivado por el aprendizaje en lugar de por objetivos de reducción del riesgo.

¿Qué priorizamos?

El UP está dirigido por casos de uso, que incluye la práctica de priorizar los casos de uso (y escenarios de casos de uso) para la implementación. También se expresan algunos requisitos como características de alto nivel no relacionadas con un caso de uso específico, normalmente porque abarcan muchos casos de uso o son un servicio general, como los servicios de registro. Estas funciones que no pertenecen a ningún caso de uso se recogerán en la Especificación Complementaria. Por tanto, incluya en una lista de priorización tanto los casos de uso como las características de alto nivel.

Requisito	Tipo	...
Procesar Venta	CU	...
Registrar	Característica	...
...	...	...

Métodos cualitativos de grupo para la priorización

En base a los anteriores criterios, se priorizan los requisitos, y los de alta prioridad se manejan en las primeras iteraciones. La priorización podría ser informal y cualitativa, generada en una reunión de grupo por miembros conscientes de estos criterios.

Sugerencia

Para priorizar informalmente los requisitos, tareas, o riesgos por medio de una reunión de grupo, utilice la “votación con puntos” de manera iterativa. Liste los ítems en una pizarra. Cada uno coge, por ejemplo, 20 puntos adhesivos. Como grupo, y en silencio (para reducir la influencia), todos se aproximan a la pizarra y aplican los puntos a los ítems, reflejando las prioridades del que vota. En cuanto se termine, se ordenan y discuten. Entonces haga una segunda vuelta de votación con puntos silenciosa para reflejar las percepciones actualizadas en base a la votación de la primera vuelta y a la discusión. Esta segunda vuelta proporciona la retroalimentación y adaptación según la cual se mejoran las decisiones.

La priorización de los requisitos o el riesgo se hará antes de la iteración 1, pero se repetirá de nuevo antes de la iteración 2, y así sucesivamente.

Métodos cuantitativos para la priorización

La discusión de grupo y algo como la votación con puntos para priorizar los requisitos o el riesgo probablemente son suficientes —un enfoque cualitativo difuso—. Para los de mente más cuantitativa, se han utilizado variaciones sobre lo siguiente. Los valores y pesos del ejemplo son sólo sugerencias; lo importante es que los valores numéricos y los pesos se pueden utilizar para razonar acerca de las prioridades.

Requisito	Tipo	SA	Riesgo	Naturaleza crítica	Suma P.
Procesar Venta	CU	3	2	3	15
Registrar	Caract	3	0	1	7
Gestionar Devoluciones	CU	1	0	0	2
...	...	...	...	...	...

	Peso	Rango
SA:Significativo para la Arquitectura	2	0-3
Riesgo: tecnol. complejo, nuevo,...	3	0-3
Naturaleza crítica: valor alto negocio inicial	1	0-3

En cualquier proyecto, los valores exactos no deberían tomarse con demasiada seriedad; en cuanto se termine, se pueden utilizar las puntuaciones numéricas para ayudar a agrupar los requisitos en conjuntos difusos de prioridad alta, media y baja. Claramente, parece importante trabajar en el caso de uso *Procesar Venta* en las primeras iteraciones.

Los números no lo dicen todo. Aunque registrar es una característica simple, de bajo riesgo, es significativo para la arquitectura porque tiene que integrarse completamente en el código desde el principio. Sería difícil y disminuye la integridad de la arquitectura el añadirlo a posteriori.

Priorización de los requisitos del PDV NuevaEra

En base a algunos métodos para priorizar, es posible establecer grupos difusos de requisitos. En términos de los artefactos del UP, esta clasificación se recoge en el Plan de Desarrollo del Software del UP.

Prioridad	Requisitos (caso de uso o característica)	Comentario
Alta	Procesar Venta Registrar ...	Puntuación alta en todos los criterios de clasificación. Generalizado. Difícil de añadir tarde. ...
Media	Gestionar Usuarios Autenticar Usuarios ...	Influye en el subdominio de seguridad. Proceso importante pero no demasiado difícil. ...
Baja	Cerrar Caja Desconectar ...	Fácil; efecto mínimo en la arquitectura. idem. ...

Los casos de uso “Poner en Marcha” y “Desconectar”

Prácticamente todos los sistemas tienen un caso de uso *Poner en Marcha*, implícito si no está explícito. Aunque su prioridad podría no ser alta de acuerdo a otros criterios, es necesario abordar por lo menos una visión simplificada de *Poner en Marcha* en la primera iteración de manera que se proporcione la incialización que asumen otros casos de uso. En cada iteración, se desarrolla incrementalmente el caso de uso *Poner en Marcha* para satisfacer las necesidades de comienzo de los otros casos de uso. De manera análoga, los sistemas a menudo tienen un caso de uso *Desconectar*. En algunos sistemas, es bastante complejo, como desconectar un conmutador de telecomunicaciones activo. En términos de la planificación, si son sencillos, estos casos de uso se pueden listar informalmente en el Plan de Iteración, como “implementar la puesta en marcha y desconexión como se necesite”. Obviamente, versiones más complejas necesitan que se cuiden más los requisitos y la planificación.

Advertencia: Planificación del proyecto vs. objetivos de aprendizaje

El objetivo del libro es ofrecer una ayuda para el aprendizaje de una introducción al análisis y diseño, en lugar de que funcione realmente el proyecto del PDV NuevaEra. Por tanto, se han tomado algunas licencias en la elección de lo que se aborda en las primeras iteraciones del caso de estudio, motivado por objetivos educativos en lugar de por objetivos del proyecto.

36.2. Priorización de los riesgos del proyecto

Un método útil para priorizar los riesgos del proyecto global es estimar su probabilidad e impacto (en coste, tiempo o esfuerzo). La estimación podría ser cuantitativa (que normalmente son muy especulativas) o simplemente cualitativas (por ejemplo, alto-medio-bajo, basado en discusiones y votaciones con puntos en grupo). Los peores riesgos son, naturalmente, aquellos tanto probables como de impacto alto. Por ejemplo:

Riesgo	Probabilidad	Impacto	Ideas para atenuarlo
Número insuficiente y calidad de desarrolladores orientados a objetos expertos.	A	A	Leer el libro. Contratar consultores temporales Educación en aulas & tutorías Diseño y programación por parejas
Demostración no preparada para la próxima convención POS-World en Hamburgo.	M	A	Contratar consultores temporales que sean especialistas en el desarrollo de sistemas de PDV en Java. Identificar requisitos “atractivos” que queden bien en una demostración y darles prioridad, sobre los otros. Maximizar el uso de componentes prefabricados.
...	...	...	...

En términos de los artefactos del UP, esto forma parte del Plan de Desarrollo de Software.

36.3. Planificación adaptable vs. predictiva

Una de las grandes ideas del desarrollo iterativo es la adaptación basada en la retroalimentación, en lugar de intentar predecir y planificar *en detalle* el proyecto completo. En consecuencia, en el UP, uno crea un Plan de Iteración sólo para la *siguiente* iteración. Más allá de la siguiente iteración se deja abierto el plan detallado, para que se ajuste de manera adaptable en el futuro (ver Figura 36.1). Además de fomentar el comportamiento flexible y oportunista, una razón sencilla para no planificar el proyecto completo en detalle es que en el desarrollo iterativo no todos los requisitos, detalles de diseño y, por tanto, las etapas, se conocen al comienzo del proyecto¹. Otra es la preferencia de confiar en el juicio de planificación del equipo conforme ellos proceden. Finalmente, suponga que hubiese un plan detallado de grano fino al comienzo del proyecto, y el equipo se “desvía” de él por adquirir una nueva percepción de cómo ejecutar el proyecto de la mejor manera. Desde el exterior, esto podría verse como un tipo de fallo, cuando de hecho es justo lo contrario.

¹ Tampoco se conocen realmente o de manera fiable en un proyecto “en cascada”, aunque se planifique de manera detallada el proyecto completo como si se conocieran.

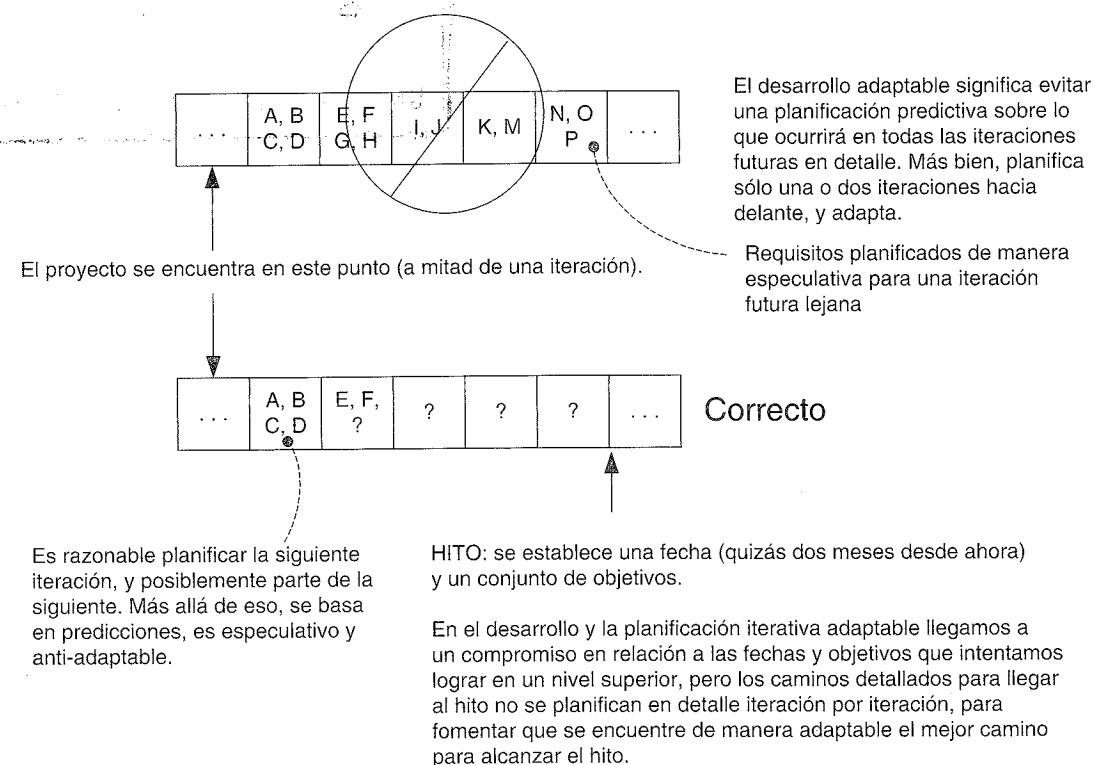


Figura 36.1. Los hitos son importantes, pero evite la planificación detallada que predice un futuro lejano.

Sin embargo, *existen* todavía objetivos e hitos; el desarrollo adaptable no significa que el equipo no sabe hacia dónde va, o las fechas de los hitos y objetivos. En el desarrollo iterativo, el equipo todavía llega a un compromiso en cuanto a las fechas y objetivos, pero el camino detallado para alcanzarlos es flexible. Por ejemplo, el equipo NuevaEra podría establecer un hito de que en tres meses se completarán los casos de uso *Procesar Venta*, *Gestionar Devoluciones* y *Autenticar Usuarios*, y las características de registrar y reglas conectables. Pero —y éste es el punto clave— no se define en detalle el plan o camino de grano fino de las iteraciones fijadas en dos semanas para ese hito. No se fija el orden de las etapas, o lo que se va a hacer en cada iteración a lo largo de los siguientes tres meses. Más bien, únicamente se planifica la siguiente iteración de dos semanas, y el equipo se adapta paso a paso, trabajando para cumplir los objetivos para la fecha de entrega. Por supuesto, las dependencias en componentes y recursos naturalmente restringen algo el orden del trabajo, pero no todas las actividades necesitan que se planifiquen en detalle de grano fino.

El personal involucrado externo ve un plan de macro-nivel (como al nivel de tres meses) al que el equipo se compromete. Pero la organización de micro-nivel se deja al mejor —y adaptable— juicio del equipo, cuando se beneficie de nuevas percepciones (ver Figura 36.1).

Finalmente, aunque en el UP se prefiere la planificación adaptable de grano fino, cada vez más es posible planificar con éxito hacia delante dos o tres iteraciones (con niveles crecientes de desconfianza) conforme los requisitos y la arquitectura se estabilizan, el equipo madura, y los datos se recogen a la velocidad del desarrollo.

36.4. Planes de Fase y de Iteración

A un macro-nivel, es posible establecer fechas y objetivos de los hitos; pero a micro-nivel, el plan para los hitos se deja flexible excepto para el futuro inmediato (por ejemplo, las siguientes cuatro semanas). Estos dos niveles se reflejan en el **Plan de Fase** y **Plan de Iteración** del UP, ambos forman parte del Plan de Desarrollo del Software compuesto. El Plan de Fase expone las fechas y objetivos de los hitos de macro-nivel, tales como los hitos del final de las fases y de las pruebas del sistema piloto a mitad de la fase. El Plan de Iteración define el trabajo para la iteración actual y la siguiente —no todas las iteraciones— (ver Figura 36.2).

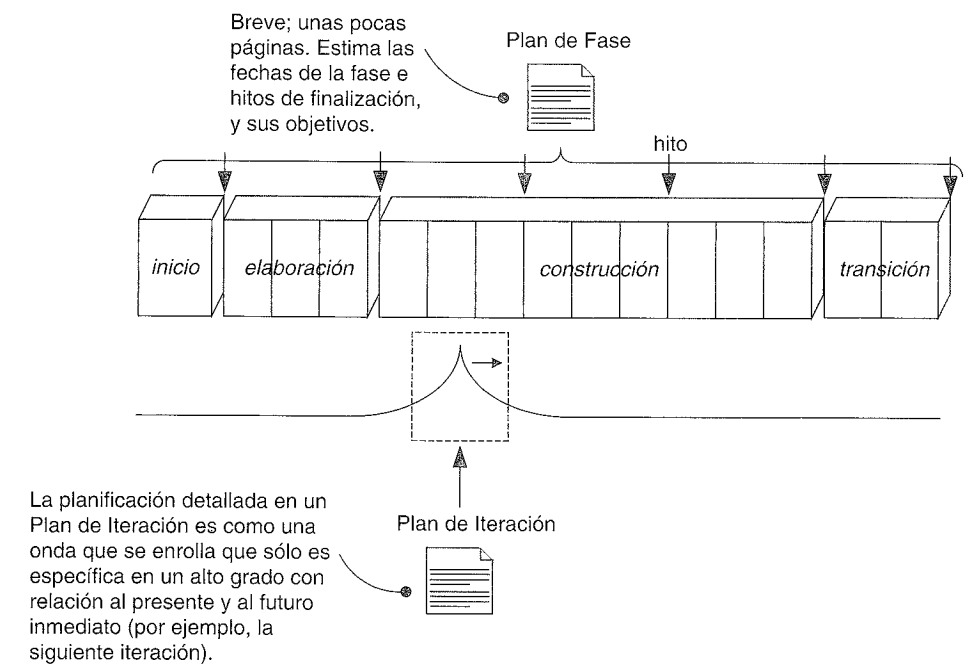


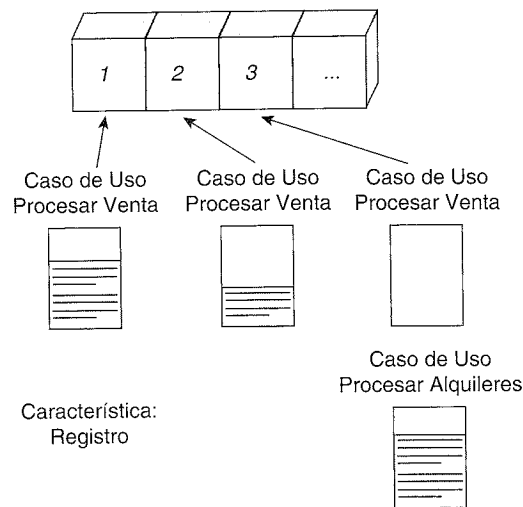
Figura 36.2. Planes de Fase y de Iteración.

Durante la fase de inicio, los hitos estimados en el Plan de Fase son vagos “estimados en base a conjeturas”. A medida que se progresa en la elaboración, las estimaciones se mejoran. Un objetivo de la fase de elaboración es tener, a su terminación, suficiente información realista para que el equipo llegue a un compromiso en relación con las fechas y objetivos de los hitos importantes para el final de la construcción y transición (esto es, la entrega del proyecto).

36.5. Plan de Iteración: ¿qué hacemos en la siguiente iteración?

El UP está dirigido por los casos de uso, lo que en parte implica que se organiza el trabajo alrededor de la finalización de los casos de uso. Es decir, se asigna una iteración para que implemente uno o más casos de uso, o escenarios de casos de uso cuando el

caso de uso completo sea demasiado complejo para una iteración. Y puesto que algunos requisitos no se expresan como casos de uso, sino como características, como registrar y reglas del negocio conectables, éstas también se deben asignar a una o más iteraciones (ver Figura 36.3).



Con frecuencia, un caso de uso o característica es demasiado complejo para que se complete en una iteración corta.

Por tanto, diferentes partes o escenarios se deben asignarse a diferentes iteraciones.

Figura 36.3. Trabajo asignado a una iteración.

Normalmente, se dedica la primera iteración de la elaboración a innumerables tareas generales como instalación y ajuste de herramientas y componentes, aclarar los requisitos, etcétera.

La priorización de los requisitos guía la elección del trabajo inicial. Por ejemplo, el caso de uso *Procesar Venta* es claramente importante. Por tanto, empezamos a abordarlo en la primera iteración. Aunque, no se implementan todos los escenarios de *Procesar Venta* en la primera iteración. Más bien, se escogen algunos escenarios simples, de camino feliz, como el de pago sólo en efectivo. Aunque el escenario es simple, su implementación comienza a desarrollar algunos elementos centrales del diseño.

Durante las iteraciones de la elaboración se abordarán requisitos diferentes, significativos para la arquitectura, relacionados con este caso de uso, forzando al equipo a tratar brevemente muchos aspectos de la arquitectura: las capas importantes, la base de datos, la interfaz de usuario, la interfaz entre los subsistemas importantes, etcétera. Esto nos lleva a la creación en las primeras etapas de una implementación “en anchura y superficial” a través de muchas partes del sistema —un objetivo común en la fase de elaboración—.

36.6. Trazo de los requisitos a través de las iteraciones

La tarea de crear el primer Plan de Iteración nos trae una cuestión relevante en el desarrollo iterativo, que se ilustra en la Figura 36.3.

Como se indicó en la última sección, no se implementarán todos los escenarios de *Procesar Venta* en la primera iteración. De hecho, este complejo caso de uso podría tar-

dar en completarse muchas iteraciones de dos semanas a lo largo de un periodo de seis meses. Cada iteración abordará nuevos escenarios o partes de escenarios.

Cuando no es posible llevar a cabo todos los escenarios de un caso de uso en una iteración, surge un problema en la traza de los requisitos. ¿Cómo recoge uno qué partes del caso de uso se han completado, en cuáles se está trabajando actualmente, o todavía no se han hecho? Una herramienta de requisitos construida para este trabajo proporciona una solución.

Un ejemplo es la herramienta de Rational, RequisitePro, y merece la pena dedicar un momento a su estudio para entender cómo trabajan estas herramientas para mantener la traza de los casos de uso completados parcialmente a lo largo de las iteraciones. No es que propongamos esta herramienta, sino que se ofrece la presentación para ilustrar una solución a este importante problema de mantener la traza.

Un ejemplo de herramienta de gestión de requisitos

RequisitePro está integrada con Microsoft Word de manera que uno podría introducir y editar los requisitos en Word, seleccionar una frase y definir la frase seleccionada como un requisito al que se le va a seguir la pista en RequisitePro.

Cada requisito puede tener una variedad de atributos, como el estado, riesgo, etcétera (ver Figuras 36.4 y 36.5). Con una herramienta como ésta, se puede manejar el problema de mantener la traza de la terminación parcial de los casos de uso a lo largo de las iteraciones.

Todas las sentencias de los escenarios principales de éxito y las extensiones se pueden representar individualmente como requisitos a los que se les va a seguir la pista, y

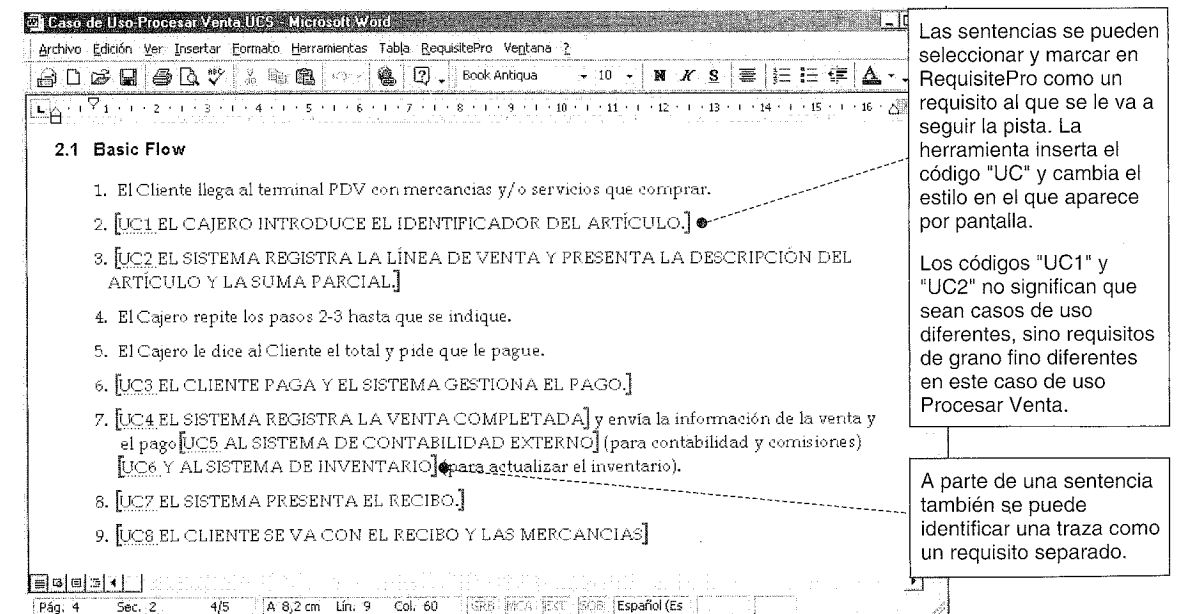


Figura 36.4. Etiquetado básico de frases de los casos de uso como requisitos.

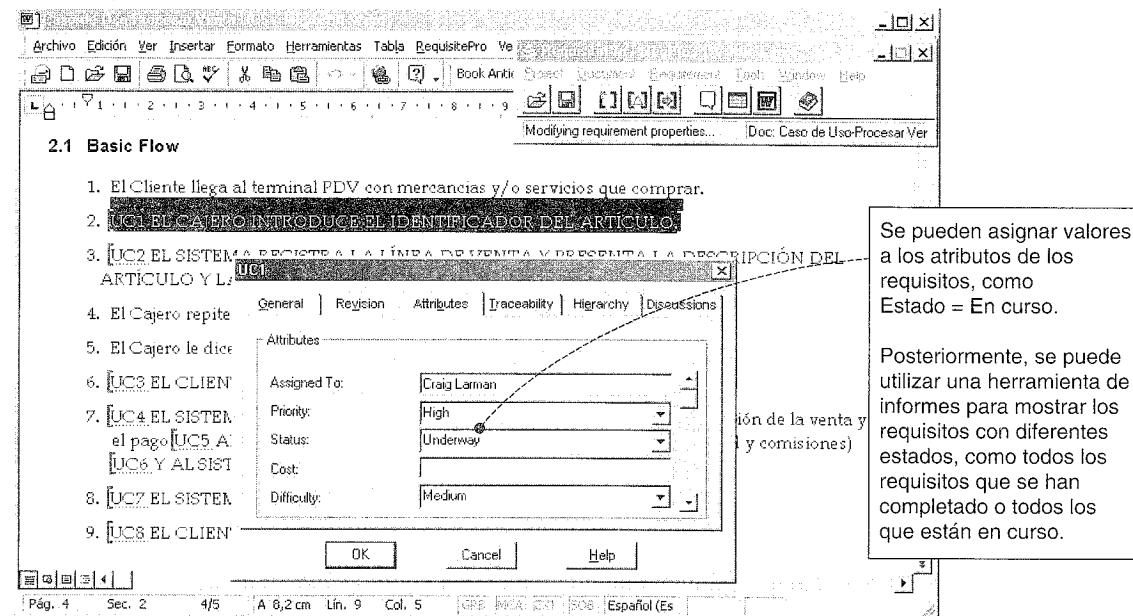


Figura 36.5. Cada requisito etiquetado tiene muchos atributos.

cada uno se puede identificar con varios valores de estado como *propuesto*, *aprobado*, etcétera.

36.7. La (in)validez de las primeras estimaciones

Basura entra, basura sale. Las estimaciones hechas con información poco fiable y difusa son poco fiables y difusas. En el UP se entiende que las estimaciones que se hacen durante la fase de inicio no son de fiar (esto se cumple para todos los métodos, pero el UP lo reconoce). Las estimaciones iniciales de la fase de inicio proporcionan una guía sobre si merece la pena que se haga un estudio real en la elaboración, para generar una buena estimación. Después de la primera iteración de la elaboración contamos con alguna información realista para producir una estimación aproximada. Después de la segunda iteración, la estimación comienza a cobrar credibilidad (ver Figura 36.6).

Las estimaciones útiles requieren de la investigación en algunas iteraciones de la elaboración.

Esto no significa que sea imposible o no merezca la pena intentar hacer estimaciones precisas al principio. Si es posible, muy bien. Sin embargo, la mayoría de las organizaciones no encuentran que éste sea el caso, por razones entre las que se encuentran la continua introducción de nuevas tecnologías, aplicaciones nuevas, y muchas otras complicaciones. Por tanto, el UP aboga por realizar un poco de trabajo realista en la elaboración antes de generar las estimaciones que se utilizan para la planificación del proyecto y la elaboración de los presupuestos.

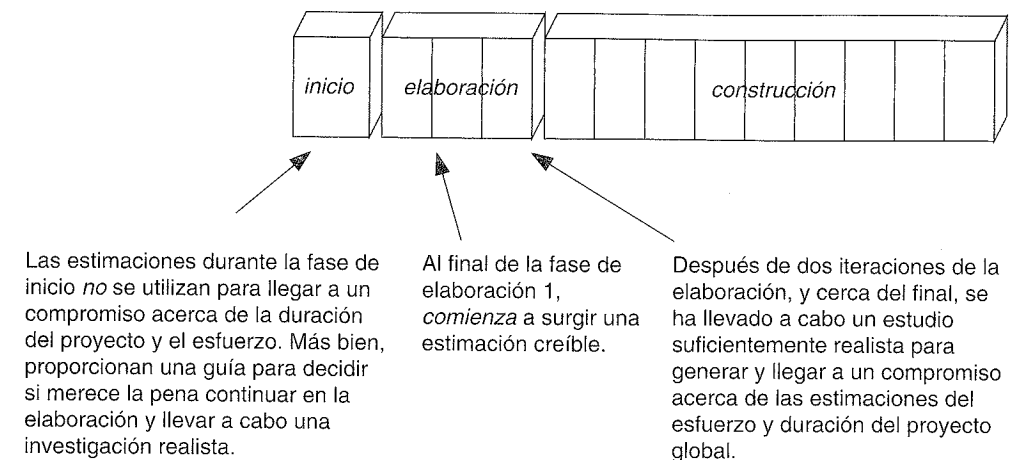


Figura 36.6. Estimación y fases del proyecto.

36.8. Organización de los artefactos del proyecto

El UP organiza los artefactos en función de las disciplinas. El Modelo de Casos de Uso y la Especificación Complementaria están en la disciplina de Requisitos. El Plan de Desarrollo del Software forma parte de la disciplina de Gestión del Proyecto, etcétera. Por tanto, organice carpetas en su control de versiones y sistema de directorios para reflejar las disciplinas, y coloque los artefactos de una disciplina en la carpeta de la disciplina relacionada (ver Figura 36.7).

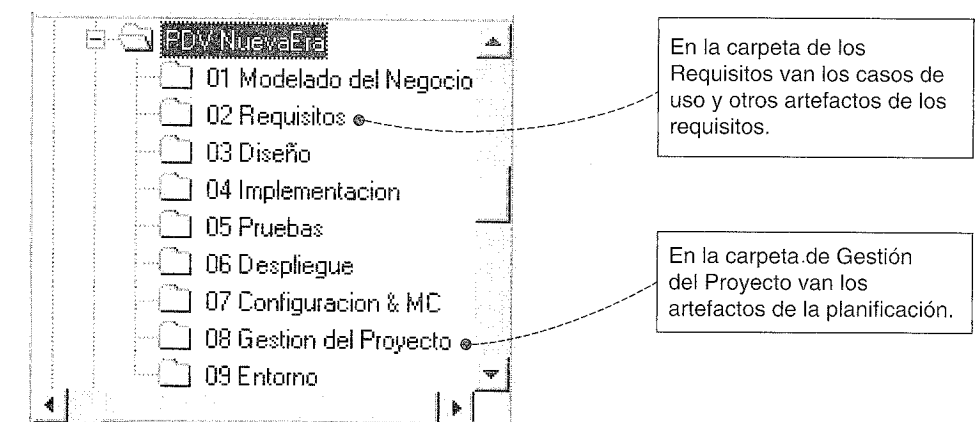


Figura 36.7. Organice los artefactos del UP en carpetas que se corresponden con sus disciplinas.

Esta organización funciona para la mayoría de los elementos que no son de la implementación. Algunos artefactos de la implementación, como la base de datos actual o los ficheros ejecutables, se encuentran comúnmente en diferentes ubicaciones debido a diversos motivos de la implementación.

Sugerencia

Después de cada iteración, utilice la herramienta de control de versiones para crear un punto de control etiquetado y congelado de todos los elementos en estas carpetas (incluyendo el código fuente). Habrá una versión "Elaboración-1", "Elaboración-2", y así sucesivamente, de cada artefacto. Para una estimación posterior de la velocidad del equipo (en éste o en otro proyecto), estos puntos de control proporcionan datos reales de la cantidad de trabajo que se hizo en cada iteración.

36.9. Algunas cuestiones de la planificación de la iteración del equipo

Equipos de desarrollo en paralelo

Un proyecto grande normalmente se divide en trabajos de desarrollo en paralelo, donde varios equipos trabajan en paralelo. Una manera de organizar los equipos es a lo largo de líneas de la arquitectura: por capas y subsistemas. Otra estructura organizacional es por conjuntos de características, que podrían muy bien corresponderse a la organización de la arquitectura.

Por ejemplo:

- Equipo de la capa del dominio (o equipo del subsistema del dominio)
- Equipo de la interfaz de usuario
- Equipo de internacionalización
- Equipo de servicios técnicos (equipo de persistencia, etc.)

Equipos en iteraciones de diferente longitud

Algunas veces, el desarrollo de un subsistema (como el servicio de persistencia) a cualquier nivel significativamente utilizable requiere mucho tiempo, especialmente durante sus primeras etapas. En lugar de alargar la longitud de la iteración global para todos los equipos, una alternativa es mantener la iteración corta (en general, un objetivo

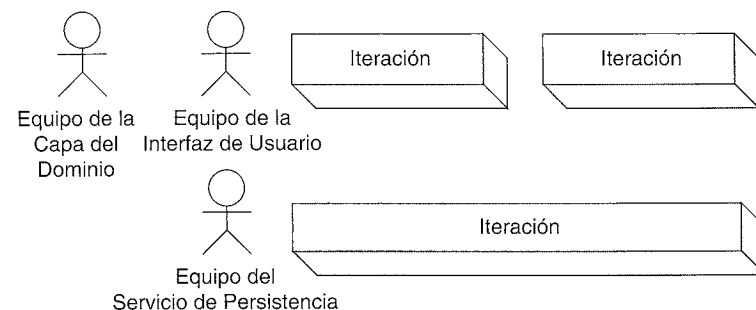


Figura 36.8. Diversas longitudes de las iteraciones.

respetable) para la mayoría de los equipos, y de longitud doble para el equipo "más lento" (ver Figura 36.8).

Velocidad del equipo y adopción incremental del proceso

Además de necesitar iteraciones más largas para los equipos muy grandes, otro motivo para alargar una iteración (por ejemplo, de tres semanas a cuatro), está relacionado con la velocidad y experiencia del equipo. Un equipo para el que son nuevas muchas de las prácticas y las tecnologías, naturalmente irá más despacio, o necesitará más tiempo para completar una iteración. Equipos con menos experiencia se beneficiarán de menos iteraciones *ligeramente* más largas que los equipos con más experiencia.

Nótese que el desarrollo iterativo proporciona un mecanismo para mejorar la estimación de la velocidad: el progreso real en las primeras iteraciones da a conocer las estimaciones para las iteraciones posteriores.

Relacionado con esto está la estrategia **adopción incremental del proceso**. En las primeras iteraciones, los equipos con menos experiencia asumen un conjunto pequeño de prácticas. A medida que los miembros del equipo los digieren y los dominan, añaden más —¡asumiendo que son útiles! Por ejemplo, en las primeras iteraciones el equipo podría construir y probar un sistema una vez al día. En iteraciones posteriores, podrían adoptar integraciones continuas y pruebas del sistema (que suceden muchas veces cada día) con una herramienta de integración continua como la de libre distribución Cruise Control (cruisecontrol.sourceforge.net).

36.10. No se entendió la planificación en el UP cuando...

- Todas las iteraciones se planifican de manera especulativa en detalle, prediciendo el trabajo y los objetivos de cada iteración.
- Se espera que las primeras estimaciones de la fase de inicio o de la primera iteración de la elaboración sean fiables, y se utilizan para llegar a compromisos del proyecto a largo plazo; generalizando, se esperan estimaciones fiables con investigaciones triviales o ligeras.
- Los problemas sencillos o las cuestiones de bajo riesgo se abordan en las primeras iteraciones.

Si la estimación de una organización y el proceso de planificación se parece algo al que se presenta a continuación, no se entendió la planificación que propone el UP:

1. Al comienzo de una fase de planificación anual, se identifican nuevos sistemas o características a un alto nivel; por ejemplo, "Sistemas Web para la gestión de la contabilidad".
2. A los encargados técnicos se les da poco tiempo para estimar de manera especulativa el esfuerzo y la duración de grandes proyectos, caros o arriesgados, que involucran con frecuencia nuevas tecnologías.

- 3. Se establece la planificación y presupuesto de los proyectos para el año.
- 4. Las personas involucradas se preocupan cuando los proyectos reales no se corresponden con las estimaciones originales. Ir al Paso 1.

Este enfoque carece de una estimación realista y refinada iterativamente basada en investigaciones serias como promueve el UP.

36.11. Lecturas adicionales

Software Project Management: A Unified Framework de Royce proporciona una perspectiva iterativa y del UP sobre la gestión y planificación de proyectos.

Surviving Object-Oriented Projects: A Manager's Guide de Cockburn contiene más información útil acerca de la planificación iterativa, y la transición a los proyectos iterativos que hacen uso de la tecnología de objetos.

The Rational Unified Process: An Introduction de Kruchten contiene capítulos útiles dedicados especialmente a la gestión y planificación de proyectos en el UP.

Una advertencia, hay algunos libros que se plantean discutir la planificación para el “desarrollo iterativo” o el “Proceso Unificado” que realmente esconden un enfoque de planificación en cascada o predictivo.

Rapid Development [McConnell96] ofrece una excelente visión global de muchas prácticas y cuestiones relacionadas con la gestión y planificación del proyecto, y riesgos del proyecto.

COMENTARIOS ACERCA
DEL DESARROLLO ITERATIVO
Y EL UP

Deberías utilizar el desarrollo iterativo sólo en los proyectos en los que quieras tener éxito.

Martin Fowler

Objetivos

- Introducir y ampliar algunos temas del UP.
- Introducir otras prácticas aplicables al desarrollo iterativo.
- Estudiar cómo el ciclo de vida iterativo puede ayudar a reducir algunos problemas del desarrollo.

37.1. Buenas prácticas y conceptos del UP adicionales

La idea más importante que hay que apreciar y poner en práctica en el UP es el desarrollo adaptable, con iteraciones cortas con una duración fija. Entre las buenas prácticas y conceptos claves adicionales del UP encontramos:

- **Abordar las cuestiones de alto riesgo y valor en las primeras iteraciones:** Por ejemplo, si el nuevo sistema es un servidor de aplicaciones que tiene que gestionar 2.000 clientes concurrentes con tiempo de respuesta de la transacción por debajo del segundo, no espere muchos meses (o años) para diseñar e implementar este requisito de alto riesgo. Más bien, céntrese rápidamente en diseñar, programar y probar los componentes esenciales del software y la arquitectura para esta cuestión arriesgada; deje el trabajo fácil hasta las iteraciones posteriores. La idea es disminuir los riesgos altos en las primeras iteraciones, de manera que el proyecto no “fracase tarde”, que es una característica de los proyectos en cascada que retrasan los temas difíciles y arriesgados hasta etapas posteriores del ciclo de vida. Es mejor “fracasar pronto”, si es que fracasa, haciendo primero las cosas difíciles. De

este modo, se dice que el UP está **dirigido por el riesgo**. Finalmente, obsérvese que el riesgo llega de muchas formas: falta de habilidades o recursos, desafíos técnicos, facilidad de uso, política, etcétera. Todas estas formas influyen en lo que se va a abordar en las primeras iteraciones.

- **Usuarios involucrados continuamente:** El desarrollo iterativo y el UP tratan de dar pequeños pasos rápidamente y obtener retroalimentación. Esto requiere una continua atención e implicación por parte de las personas involucradas del negocio y los expertos en la materia de estudio para clarificar y dirigir el proyecto. Al principio, el personal del negocio podría sentir que esto es una imposición. Sin embargo, existe una correlación entre la mayoría de los proyectos que han fracasado y la falta de implicación de los usuarios [Standish94], y este enfoque le proporciona al personal del negocio la capacidad de dar forma al software como realmente lo necesitan. En proyectos donde el “usuario” puede ser cualquiera, como un nuevo sitio web o producto del consumidor, grupos interesados podrían actuar como sustitutos.
- **Atención en las primeras etapas a construir una arquitectura básica cohesiva:** Es decir, el UP está **centrado en la arquitectura**. Esto está relacionado con abordar las cuestiones de alto riesgo en las primeras iteraciones, puesto que normalmente establecer los elementos básicos de la arquitectura es un elemento arriesgado o crítico. Las primeras iteraciones se centran en una implementación de la arquitectura “en anchura y superficial”, estableciendo las cuestiones de diseño importantes, y los subsistemas con sus interfaces y responsabilidades. El equipo “investigará” en áreas verticalmente profundas sobre requisitos específicos difíciles o arriesgados, como el requisito para transacciones por debajo del segundo con 2.000 clientes concurrentes.
- **Verificar continuamente la calidad, desde el principio y con frecuencia:** La calidad en este contexto abarca satisfacer o superar correctamente los requisitos en un proceso sostenible y repetible, con software de fácil mantenimiento y escalable. Una razón para una campaña temprana, continua e intensiva de pruebas, inspección y aseguramiento de la calidad es que el coste de un defecto tardío se incrementa de manera no lineal a través de las fases de un proyecto. Además, el desarrollo iterativo se basa en la retroalimentación y la adaptación; por tanto, las pruebas y evaluación realistas en las primeras etapas son actividades críticas para obtener retroalimentación significativa. Esto difiere del proyecto en cascada, donde la etapa de aseguramiento de la calidad se hace cerca del final del proyecto, cuando la respuesta es la más difícil y costosa. En el UP, la verificación de la calidad se integra de manera continua desde el principio, de manera que no hay grandes sorpresas cerca del final del proyecto. Observe que en el UP, la verificación de la calidad también se refiere a la calidad del proceso —cada iteración, evaluando lo bien que lo está haciendo el equipo—.
- **Aplicar casos de uso:** Informalmente, los casos de uso son historias escritas del uso de un sistema. Son mecanismos para explorar y recoger los requisitos funcionales, a diferencia del estilo más antiguo de la lista de funciones o la lista “El sistema deberá hacer...”. El UP recomienda que se apliquen los casos de uso como la forma principal para la captura de los requisitos, y como una fuerza que dirige la planificación, diseño, pruebas y la escritura de la documentación del usuario final.

- **Modelar el software visualmente:** Un porcentaje muy importante del cerebro de las personas está implicado en el procesamiento visual, que es una de las motivaciones que hay detrás de la presentación visual o gráfica de la información [Tufte92]. Por tanto, es conveniente emplear no sólo lenguajes textuales (como texto o código), sino también lenguajes simbólicos, basados en diagramas, visuales con orientación espacial, como UML, porque esto se beneficia de las ventajas naturales del cerebro¹. Además, la *abstracción* es una práctica útil para razonar sobre los diseños del software y comunicarlos, porque esto nos permite centrarnos en los aspectos importantes, mientras ocultamos o ignoramos los detalles que producen ruido. Un lenguaje visual como UML nos permite visualizar y razonar sobre los modelos abstractos del software, pasando rápidamente al diseño con esquemas en forma de diagramas de las grandes ideas. Pero como se estudiará más tarde, existe un “punto dulce en UML” entre realizar pocos o demasiados diagramas.
- **Gestión de requisitos cuidadosa:** Esto no significa hacer uso del estilo en cascada de definir completamente y congelar los requisitos en la primera fase del proyecto. Más bien, implica no ser descuidado; es decir, ser habilidoso en la elicitación, escritura, asignación de las prioridades, establecimiento de las trazas y mantenimiento de la pista a lo largo del ciclo de vida de los requisitos, normalmente con el soporte de una herramienta. Esto parece obvio, pero da la impresión de que rara vez se pone en práctica bien. La gestión de requisitos pobre es un factor común en proyectos que tienen problemas [Standish94].
- **Control de cambios:** Esta práctica comprende varias ideas: Primero, gestión de peticiones de cambios. Aunque un proyecto UP iterativo acepta los cambios, no acepta el caos. Cuando surge la solicitud de un nuevo requisito durante las iteraciones, en lugar de un jovial “¡Por supuesto, sin problemas!” hay una evaluación racional del esfuerzo y el impacto, y si se acepta, se modifica la planificación. También incluye la idea de mantener la pista del ciclo de vida de todas las peticiones de cambio (solicitado, en curso,...). Segundo, gestión de configuraciones. Se utilizan herramientas de gestión de configuraciones y construcción para dar soporte a la frecuente (idealmente, por lo menos diariamente) integración y pruebas del sistema, desarrollo en paralelo, áreas de trabajo de desarrolladores y configuraciones separadas, y control de versiones —desde el comienzo del proyecto—. En el UP, todos los bienes del proyecto (no sólo el código) deberían estar bajo el control de configuraciones y versiones.

37.2. Las fases de construcción y transición

Construcción

La elaboración termina cuando se han resuelto las cuestiones de alto riesgo, se ha completado el núcleo central o esqueleto de la arquitectura y se han entendido “la mayoría” de los requisitos. Al final de la elaboración, es posible estimar de manera más realista el esfuerzo y duración restante del proyecto.

¹ Éste también es un motivo para utilizar colores en los diagramas (a no ser que algún miembro del equipo sea daltónico). Por ejemplo, véase [CDL99].

Le sigue la **fase de construcción**, cuyo propósito es esencialmente terminar de construir la aplicación, realizar pruebas alpha, preparar las pruebas beta (en la fase de transición) y preparar el despliegue, mediante actividades tales como la escritura de las guías de usuario y la ayuda on-line. A veces, se resume como poner la “carne sobre el esqueleto” creado en la elaboración. Mientras que la elaboración se puede caracterizar como que construye el núcleo del sistema arriesgado y significativo para la arquitectura, la construcción se puede describir como que construye el resto. Como antes, el desarrollo avanza por medio de una serie de iteraciones en las que se fija la duración. En cuanto al personal, se recomienda que se utilice un equipo pequeño y cohesivo durante la elaboración, y luego ampliar el tamaño del equipo durante la construcción; además probablemente habrá más equipos en paralelo desarrollando durante esta fase.

Transición

La construcción termina cuando se considera que el sistema está preparado para el despliegue operacional, y se han completado todos los materiales de soporte, como las guías de usuario, materiales de aprendizaje, etcétera. Le sigue la **fase de transición**, cuyo propósito es poner el sistema en producción. Esto podría incluir actividades como pruebas beta, reaccionar a la retroalimentación de las pruebas beta, pequeños ajustes, conversión de datos, cursos de entrenamiento, marketing para el lanzamiento del producto, funcionamiento en paralelo del antiguo y el nuevo sistema, y cosas por el estilo.

37.3. Otras prácticas interesantes

Ésta no es una lista exhaustiva, pero entre las prácticas interesantes —no documentadas explícitamente en el UP— que han sido de valor en los proyectos iterativos se encuentran:

- El patrón de proceso **SCRUM** [BDSS00]; véase también www.controlchaos.com. El más concreto es una reunión de SCRUM diaria de pie de “15 minutos”. El jefe del proyecto pregunta a cada persona: 1) las cosas hechas desde la última reunión; 2) los objetivos para el día siguiente; y 3) los obstáculos para que el jefe los elimine. Yo también he preguntado a cada miembro las percepciones relevantes que quiere compartir con el equipo. La reunión fomenta el comportamiento del equipo adaptable que emerge, medidas del progreso de grano fino, comunicación de alta densidad y la socialización de los proyectos. Otras ideas claves son: el equipo está libre de todas las distracciones externas, no se le añade ningún trabajo adicional (desde fuera del equipo) durante una iteración, y el trabajo de gestión es eliminar todos los obstáculos y distracciones, de manera que el equipo pueda centrarse.
- Algunas prácticas de la **Programación Extrema** (XP, *Extreme Programming*) [Beck00], como **programar probando primero**: escribir una prueba de unidad *antes* del código que se va a probar, y escribir las pruebas para casi todas las clases. Si estamos trabajando con Java, **JUnit** (www.junit.org) es un framework de pruebas conocido de libre de distribución. Escriba un poco de las pruebas, escriba un poco de código, haga que pase las pruebas, repita. Es esencial que se escriban las pruebas *en primer lugar* para experimentar el valor de este enfoque.

- **Integración continua**, otra práctica de XP; véase [Beck00] para obtener una introducción y www.martinfowler.com para los detalles. El UP incluye la buena práctica de integrar el sistema completo al menos una vez en cada iteración. Esto se abrevia con frecuencia como la práctica de la **construcción diaria**. Integración *continua* lo acorta todavía más, integrando todo el nuevo código que se registra (al menos) cada pocas horas. Aunque esto se puede hacer manualmente, una alternativa es utilizar un entorno que automatice la integración continua y las pruebas en una máquina para la construcción rápida que ejecuta un proceso demonio. Periódicamente se despierta (como cada dos minutos) y busca nuevo código para registrar, que dispara la ejecución de un *script* de prueba y reconstrucción. Está disponible libremente un sistema de integración continua para los proyectos Java denominado **Cruise Control** de libre distribución en SourceForge (cruisecontrol.sourceforge.net).

37.4. Motivos para fijar la duración de una iteración

Hay por lo menos cuatro motivos para fijar la duración de una iteración.

Primero, la **ley de Parkinson**. Parkinson con decepción observó que “El trabajo se expande de manera que rellena el tiempo disponible para su terminación” [Parkinson58]. Fechas de finalización distantes o difusas (por ejemplo, dentro de seis meses), agravan este efecto. Cerca del comienzo del proyecto, puede parecer que hay mucho tiempo para proceder sin prisa. Pero si la fecha de finalización para la siguiente iteración es sólo dentro de *dos semanas*, y se debe colocar un sistema parcial ejecutable y probado para esta fecha, el equipo tiene que centrarse, tomar decisiones y moverse.

Segundo, **asignar prioridades y decisión**. Las iteraciones cortas con la duración fija fuerzan al equipo de desarrollo a tomar decisiones teniendo en cuenta la prioridad del trabajo y los riesgos, identificar cuáles tienen el valor más alto para el negocio o técnico, y estimar algo de trabajo. Por ejemplo, si está embarcado en la primera iteración, que se ha elegido que dure exactamente cuatro semanas, no hay mucha libertad para ser vago —se deben tomar decisiones concretas sobre lo que se hará realmente en las cuatro primeras semanas—.

Tercero, **satisfacción del equipo**. Las iteraciones cortas con la duración fija nos llevan a una rápida y repetida sensación de terminación, competencia y finalización. En ciclos ordinarios de dos o cuatro semanas, el equipo tiene la experiencia de terminar algo, en lugar de trabajar lentamente durante meses sin terminar nada. Estos factores psicológicos son importantes para la satisfacción del trabajo individual, y para generar confianza en el equipo.

Cuarto, **confianza del personal involucrado**. Cuando un equipo llega a un compromiso público de producir algo ejecutable y estable en un periodo corto de tiempo, en una fecha concreta, como dentro de dos semanas, y lo hace, el personal del negocio y otras personas implicadas incrementan su confianza en el equipo y en el proyecto.

37.5. El ciclo de vida secuencial en “cascada”

A diferencia del ciclo de vida iterativo del UP, una antigua alternativa es el ciclo de vida secuencial, lineal, o “en cascada” [Royce70], asociado a procesos grandes y predictivos.

En la práctica habitual, un ciclo de vida en cascada define etapas parecidas a las siguientes:

1. Clarificar, recoger y llegar a un compromiso de un conjunto de requisitos finales.
2. Diseñar un sistema en base a estos requisitos.
3. Implementar, basado en el diseño.
4. Integrar módulos dispares.
5. Evaluar y probar para conseguir corrección y calidad.

Un proceso de desarrollo basado en el ciclo de vida en cascada se asocia con estos comportamientos o aptitudes:

- Definición completa y cuidadosa de un artefacto (por ejemplo, los requisitos o el diseño) antes de pasar a la siguiente etapa.
- Comprometerse a un conjunto congelado de requisitos detallados.
- La desviación de los requisitos o el diseño durante las etapas posteriores indican un fallo porque no se ha sido lo suficientemente habilidoso o exhaustivo. ¡La próxima vez, intente con más empeño hacerlo bien!

Un proceso en cascada es similar al enfoque de la ingeniería según el cual se construyen los edificios o los puentes. Su adopción hace que el desarrollo del software parezca más estructurado y análogo a la ingeniería en algún otro campo. Durante algún tiempo, el proceso en cascada fue el enfoque que aprendieron más desarrolladores de software, directores, autores y profesores cuando eran estudiantes (y después repetían), sin investigar de manera crítica su aplicabilidad al desarrollo del software.

Algunas cosas se deberían construir como los edificios —tales como, bien... edificios— pero, normalmente, no el software.

Como se mencionó en el capítulo preliminar acerca del UP, el estudio de dos años presentado en el *MIT Sloan Management Review*, acerca de los proyectos software exitosos identificó cuatro factores comunes para el éxito: el desarrollo iterativo, en lugar del ciclo de vida en cascada, era el primero de la lista [MacCormack01].

Algunos problemas con el ciclo de vida en cascada

La metáfora de la construcción ha sobrevivido a su utilidad. Es el momento de cambiar de nuevo. Si, como creo, las estructuras conceptuales que construimos hoy son demasiado complicadas para que se especifiquen con precisión por adelantado, y demasiado complejas para que se construyan sin errores, entonces debemos utilizar un enfoque radicalmente diferente (desarrollo iterativo, incremental).

—Frederick Brooks, “No Silver Bullet”, *The Mythical Man-Month*

En una cierta escala de tiempo, realizar parte de los requisitos antes del diseño, y parte del diseño antes de la implementación, es inevitable y sensato. Para un proyecto corto de dos meses, se puede utilizar un ciclo de vida secuencial. Y una iteración individual en el desarrollo iterativo es como un proyecto en cascada corto.

Sin embargo, comienzan a aparecer las dificultades cuando la escala de tiempo se alarga. La complejidad pasa a ser alta, las decisiones especulativas se incrementan y complican, no existe retroalimentación, y en general las cuestiones de alto riesgo no se están abordando lo suficientemente pronto. Por definición, uno intenta todo o la mayoría del trabajo de los requisitos para el sistema completo antes de avanzar, y la mayoría del diseño antes de pasar a lo siguiente.

Se abordan etapas largas en las que se toman muchas decisiones sin el beneficio de una retroalimentación concreta a partir de implementaciones y pruebas reales. En la escala de un mini-proyecto de dos semanas (es decir, una iteración), se puede utilizar una secuencia lineal requisitos-diseño-implementación; el grado de compromisos especulativos de algunos requisitos y del diseño no se encuentra en la zona peligrosa. Sin embargo, a medida que se amplía la escala, así lo hace la especulación y el riesgo.

Entre los problemas relacionados con un proceso en cascada a la escala del proyecto completo encontramos:

- Mitigación del riesgo atrasada; abordando tarde problemas de alto riesgo o difíciles.
- Especulación e inflexibilidad de los requisitos y el diseño
- Alta complejidad
- Baja adaptabilidad

Mitigación de algunos problemas con el ciclo de vida iterativo

El desarrollo iterativo no es una bala mágica para el desafío del desarrollo del software. Pero, ayuda a reducir algunos problemas agravados por un ciclo de vida en cascada lineal.

Problema: Mitigación del riesgo atrasada

El riesgo aparece de muchas formas: el diseño incorrecto, el conjunto de requisitos equivocado, un entorno político extraño, falta de habilidades o recursos, facilidad de uso, etcétera.

En un ciclo de vida en cascada, *no* existe un intento activo de identificar y mitigar en primer lugar las cuestiones de más riesgo. Por ejemplo, la arquitectura incorrecta para un sitio web de alta carga y alta disponibilidad puede causar retrasos costosos, o cosas peores. En un proceso en cascada, la validación de la adecuación de la arquitectura tiene lugar *mucho tiempo* después de que se especifiquen (inevitablemente de manera imperfecta) todos los requisitos y todo el diseño, durante la importante etapa posterior de la implementación. Esto podría ser muchos meses e incluso años después de la fase de inicio del proyecto (ver Figura 37.1). Y no hay escasez de historias donde equipos separados hayan construido subsistemas a lo largo de un largo periodo, y entonces intentaran integrar éstos y comenzar las pruebas del sistema global cerca del final del proyecto —previsiblemente con resultados dolorosos—.

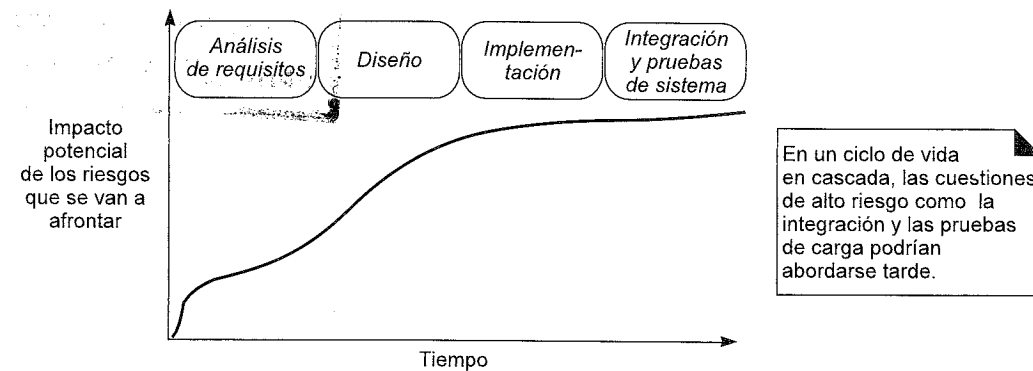


Figura 37.1. Ciclo de vida en cascada y los riesgos.

Mitigación

En cambio, en el desarrollo iterativo el objetivo es identificar y mitigar las cuestiones de más riesgo cuanto antes. Los riesgos altos podrían encontrarse en el diseño del núcleo de la arquitectura, la facilidad de uso de las interfaces, personal involucrado que no se implica. Sea lo que sea, se abordan en primer lugar. Como se ilustra en la Figura 37.2, las primeras iteraciones se centran en disminuir el riesgo. Continuando con el ejemplo anterior del sitio web con carga alta, en un enfoque iterativo, antes de realizar mucho estudio en otros requisitos o trabajo de diseño, el equipo, en primer lugar, diseña, implementa y prueba de manera realista lo suficiente del núcleo de la arquitectura para demostrar que se encuentran en el camino adecuado con respecto a la carga y a la disponibilidad. Si las pruebas demuestran que están equivocados, adaptan el diseño básico en las primeras etapas del proyecto, en lugar de cerca del final.

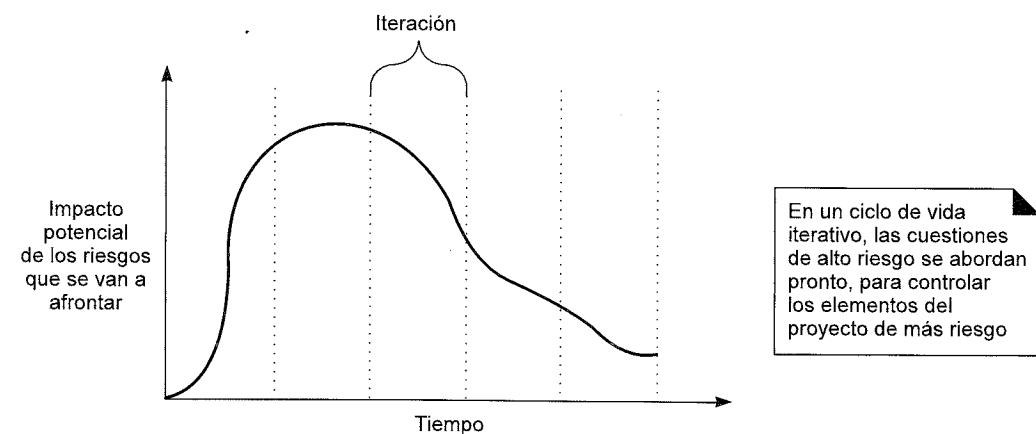


Figura 37.2. Ciclo de vida iterativo y los riesgos.

Problema: Especulación e inflexibilidad de los requisitos

Una suposición fundamental en un proceso en cascada es que los requisitos se pueden especificar por completo y después se congelan en la primera fase de un proyecto. En ta-

les proyectos, se hace un esfuerzo por realizar en primer lugar el análisis de requisitos completo, culminando en un conjunto de artefactos de requisitos que se revisan y se "dan por concluidos".

Normalmente resulta ser una suposición que tiene algún fallo. El esfuerzo para obtener todos los requisitos definidos y dados por concluidos antes del trabajo de diseño e implementación es probable, irónicamente, que incremente las dificultades del proyecto en lugar de mejorarlas. También lo hace difícil responder más tarde en un proyecto a una nueva oportunidad del negocio por medio de un cambio en el software.

Garantizado, hay algunos proyectos que necesitan que se realice en primer lugar un esfuerzo para especificar completamente y con precisión los requisitos. Esto es especialmente cierto cuando el software se acopla con la construcción de componentes físicos. Entre los ejemplos encontramos los dispositivos de aviación y médicos. Pero nótese que incluso en este caso, el desarrollo iterativo puede todavía aplicarse con provecho al proceso de diseño e implementación.

La investigación que suscita mayor interés en la crítica del mito de ser capaz de definir en primer lugar, con éxito, todos los requisitos procede de [Jones97]. Como se ilustra en la Figura 37.3, en este amplio estudio de 6.700 proyectos, los requisitos arrastrados —aquellos que no se anticiparon cerca del comienzo— son un hecho significativo de la vida del desarrollo del software, variando desde alrededor del 25% en proyectos de tamaño medio, hasta el 50% en los más amplios; Boehm y Papaccio presentan conclusiones basadas en investigaciones análogas en [BP88]. Las actitudes en cascada, que luchan en contra de (o simplemente niegan) este hecho, asumiendo que los requisitos y los diseños se pueden especificar y congelar, son incongruentes con la mayoría de las realidades de los proyectos.

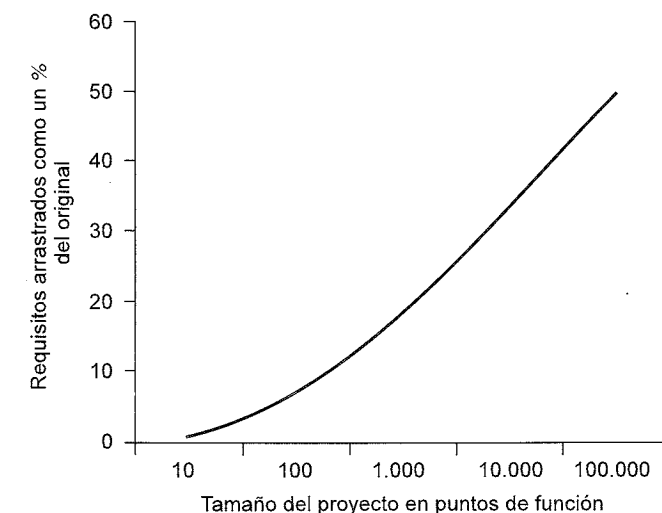


Figura 37.3. Los requisitos que cambian son una fuerza inevitable del desarrollo².

² Los puntos de función describen la complejidad del sistema con una métrica independiente del lenguaje de programación (véase www.ifpug.org).

Por tanto, “la única constante es el cambio”, normalmente porque:

- Las personas involucradas cambian de idea o no pueden imaginarse lo que quieren hasta que no ven un sistema concreto³.
- El mercado cambia.
- La especificación validada correctamente, detallada, y precisa es un desafío psicológico y organizativo para la mayoría de las personas involucradas [Kruchten00].

Y así, existen problemas previsibles y que se ve con frecuencia que surgen en los proyectos en cascada. Puesto que en realidad el cambio significativo es inevitable, éstos incluyen:

- Como se describió anteriormente, descubrimiento y mitigación de altos riesgos retrasado.
- Un sentimiento negativo entre los miembros del equipo de “vivir una ficción” o fracaso del proyecto, ya que la realidad de los cambios no se corresponde con el ideal.
- La realización de una amplia (y costosa) inversión en el diseño e implementación incorrectos (puesto que se basa en requisitos incorrectos).
- Falta de sensibilidad ante los deseos de los usuarios, o las oportunidades de mercado, que cambian.

Mitigación

En el desarrollo iterativo, no se especifican todos los requisitos antes del diseño y la implementación, y no se estabilizan los requisitos hasta después de, al menos, varias iteraciones. Por ejemplo:

Primero, se define un subconjunto de los requisitos básicos, por ejemplo, en un taller de requisitos de dos días. Entonces, el equipo selecciona un subconjunto de éstos para el diseño y la implementación (normalmente en base al riesgo o valor de negocio más alto). Después de una iteración de cuatro semanas, el personal involucrado se reúne en un segundo taller de requisitos de uno o dos días, revisan de manera intensiva el sistema parcial y aclaran y modifican sus solicitudes. Tras una segunda iteración (más corta) de dos semanas en la que se implementa incrementalmente el sistema, las personas involucradas se reúnen en un tercer taller de requisitos, y refinan de nuevo. A estas alturas, los requisitos comienzan a estabilizarse y representar el verdadero alcance y las intenciones clarificadas de las personas involucradas. A estas alturas, es posible determinar un plan algo realista y una estimación del trabajo restante. Estas iteraciones podrían caracterizarse como parte de la fase de elaboración del UP.

Todavía se admiten cambios posteriores en los requisitos. Sin embargo, la interacción en las primeras iteraciones del trabajo de implementación paralelo y el análisis de requisitos que obtiene retroalimentación a partir de la implementación parcial, nos lleva a una mejor definición de los requisitos en la fase de elaboración.

³ Barry Boehm ha llamado a esto el efecto “Lo sabré cuando lo vea”.

Problema: Especulación e inflexibilidad del diseño

Otra idea central del ciclo de vida en cascada es que la arquitectura y la mayor parte del diseño pueden y deberían especificarse por completo en la segunda fase importante de un proyecto, una vez que se han aclarado los requisitos. En tales proyectos, se hace un esfuerzo por describir a fondo la arquitectura completa, los diseños de los objetos, la interfaz de usuario, el esquema de base de datos, etcétera, antes del que comience la implementación. Algunos problemas asociados con esta suposición:

1. Puesto que los requisitos cambiarán, el diseño original podría no ser fiable.
2. Herramientas, componentes y entornos inmaduros o mal entendidos, hacen las decisiones de diseño especulativas y arriesgadas; podría demostrarse que son incorrectas sobre la implementación porque “no se suponía que el servidor de aplicación fuera a hacer eso...”.
3. En general, la falta de retroalimentación para probar o desaprobar el diseño, hasta mucho tiempo después de que se tomaran las decisiones de diseño.

Mitigación

Estos problemas se mitigan en el desarrollo iterativo construyendo rápidamente parte del sistema y validando el diseño y los componentes de terceras partes mediante las pruebas.

37.6. Ingeniería de usabilidad y diseño de interfaces de usuario

No existe probablemente ninguna técnica con mayor disparidad entre su importancia para el éxito en el desarrollo de software y la falta de una atención y educación formal que la **ingeniería de usabilidad** y el diseño de las interfaces de usuario (UI). Aunque fuera del alcance de esta introducción al A/DOO y el UP, nótese que el UP no incluye el reconocimiento de esta actividad; modelos de usabilidad y UI forman parte de la disciplina de Requisitos. En terminología del UP, los **guiones de los casos de uso** se pueden utilizar para describir de manera abstracta los elementos de la interfaz, y la navegación entre ellos, ya que se relacionan con los escenarios de los casos de uso.

Entre los libros útiles encontramos *Software for Use* de Constantine y Lockwood, *The Usability Engineering Lifecycle* de Mayhew, y *GUI Bloopers* de Johnson.

37.7. El Modelo de Análisis del UP

El UP contiene un artefacto denominado **Modelo de Análisis**; no es necesario y pocos lo crean. Quizás al **Modelo de Análisis** no se le ha asignado el nombre ideal, ya que es realmente un tipo de modelo de diseño. En el uso convencional (por ejemplo, véase [Coleman+94, MO95, Fowler96]), un modelo de análisis sugiere esencialmente un modelo de objetos del dominio —un estudio y descripción de los conceptos del dominio—. Pero el “Modelo de Análisis” del UP es una primera versión del Modelo de Diseño del UP describiendo objetos del software que colaboran, con responsabilidades. Citando textualmente, “El modelo de análisis

es una abstracción, o generalización, del diseño” [Kruchten00]. Y, “Un modelo de análisis puede verse como un primer corte de un modelo de diseño” [JBR99].

El equipo del producto del RUP destaca que es opcional y que su valor no es frecuente, y no fomenta que se cree habitualmente —ya que es otro conjunto más de diagramas que hay que crear antes de la implementación, y los metodologistas y arquitectos expertos rara vez lo utilizan—.

37.8. El producto del RUP

El producto del RUP es un conjunto de documentación basada en el web cohesivo y bien diseñado (páginas HTML) vendido por Rational Software que describe el Proceso Unificado de Rational, un refinamiento actualizado y detallado del UP, que es más general. Describe todos los artefactos, actividades y roles, proporciona guías e incluye plantillas para la mayoría de los artefactos (ver Figura 37.4).

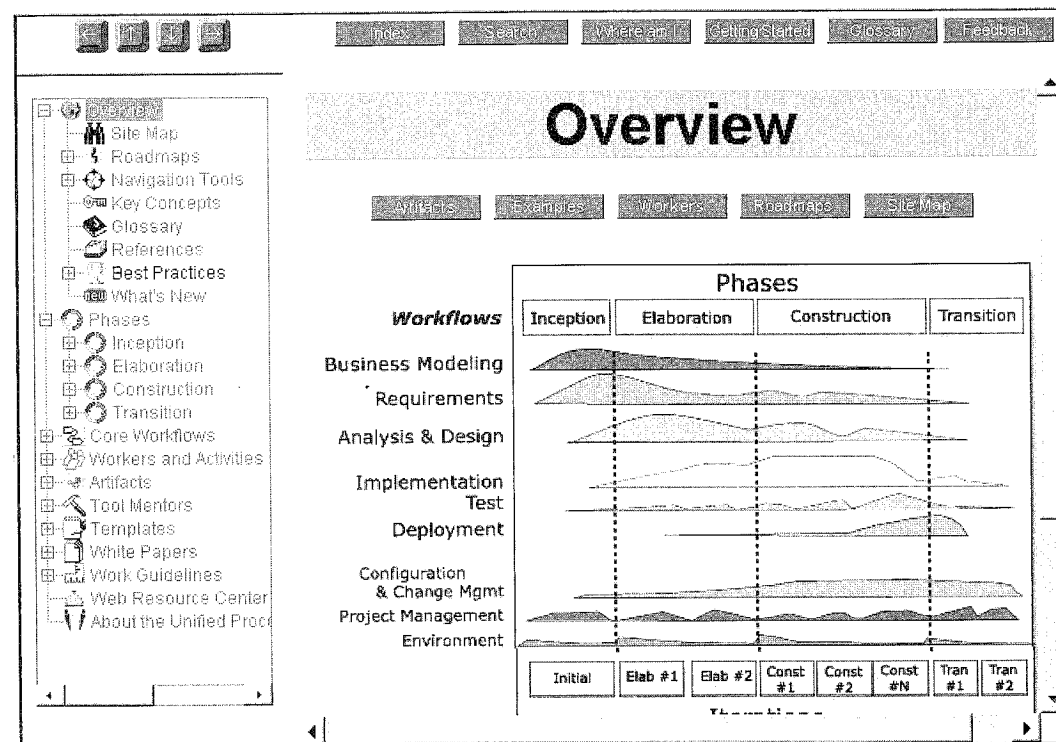


Figura 37.4. El producto del RUP.

El UP se puede aplicar o adoptar con la ayuda de tutores del proceso y libros; las ideas básicas, como el desarrollo iterativo, se describen en éste y en otros libros. En consecuencia, no es necesario poseer el producto del RUP. No obstante, algunas organizaciones consideran que colocar este producto basado en el web (y sus plantillas) en su intranet (respetando la licencia) en una localización visible es un mecanismo simple y efectivo para extender gradualmente su adopción. Que una organización pase a un nue-

vo proceso de desarrollo, más allá de un nivel superficial, requiere que se le apoye de varios modos. Además de tutores para el proceso, proyectos pilotos y seminarios, la documentación y las plantillas basadas en el web proporcionadas por el producto del RUP, sin duda son ayudas útiles que merece la pena que se evalúen.

37.9. Los desafíos y mitos de la reutilización

El UP está desarrollado con los proyectos de la tecnología de objetos (TO) en mente, y con frecuencia se promueve la adopción de la TO para conseguir la **reutilización** del software. Conseguir reutilizar de manera significativa es un objetivo loable, pero difícil. Es una función de mucho más que adoptar la TO y escribir clases; la TO no es más que una tecnología que la posibilita mediante un conjunto de cambios técnicos, organizativos y sociales que tienen que ocurrir para poder apreciar una reutilización significativa. Ciertamente, las librerías de clases para los servicios técnicos, como las librerías de la tecnología Java, proporcionan un gran ejemplo de reutilización, pero me estoy refiriendo a la dificultad de reutilizar código creado en una organización, no a las librerías básicas.

En una encuesta a organizaciones que han adoptado la TO, se les preguntó el valor real de su adopción. Curiosamente, la reutilización estaba *al final* de la lista [Cutter97]. Entre los expertos y organizaciones con experiencia en la TO, no es una sorpresa: saben que la popular descripción de la prensa de la TO para reutilizar es, en cierto modo, un mito; la mayoría de las organizaciones lo ven poco. Esto no implica que no sea un objetivo valioso, o que no haya reutilización —merece la pena, y ha habido algo—. Pero no los altos niveles de reutilización que sugieren muchos artículos y libros. Y muchos desarrolladores con experiencia en la TO puede contarle una historia de guerra sobre el intento desorientado a gran escala de una organización para crear las grandes “librerías reutilizables” o servicios para la compañía, gastando un año y millones de dólares, y terminando con un proyecto fracasado, o uno que perdió el rumbo. La reutilización es difícil, y se puede sostener que es una función de cuestiones más sociales y organizativas que técnicas.

¿Significa esto que la TO no tiene valor? En absoluto, pero su valor se ha asociado incorrectamente sobre todo con la reutilización, en lugar de al modo en el que ayuda de manera más perceptible en la práctica: flexibilidad, facilidad de cambio y manejo de la complejidad. La misma encuesta [Cutter97] lista los valores sobresalientes que realmente se experimentan al adoptar la TO: mantenimiento de la aplicación más simple y ahorro de costes. Los sistemas de objetos —si se diseñan bien— son relativamente más fáciles o más rápidos de modificar y extender, que si se utilizan tecnologías no orientadas a objetos. Esto es importante; muchas organizaciones encuentran que la mayoría de los costes globales a largo plazo de una aplicación tienen que ver con la revisión y el mantenimiento, no con el desarrollo original y, por tanto, son importantes las estrategias que permitan reducir los costes de revisión. Aunque es racional querer reducir los costes de desarrollo de nuevos sistemas, es irónico que pocas de las personas involucradas pregunten lo que sigue, “¿Cómo podemos reducir los costes de revisión y mantenimiento?” cuando con frecuencia es lo más caro. Es aquí donde puede contribuir la TO, además de su fuerza y elegancia al abordar sistemas complejos.

38.1. Notación general

Especificaciones de estereotipos y propiedades con etiquetas

Los estereotipos se utilizan en UML para clasificar un elemento (ver Figura 38.1).

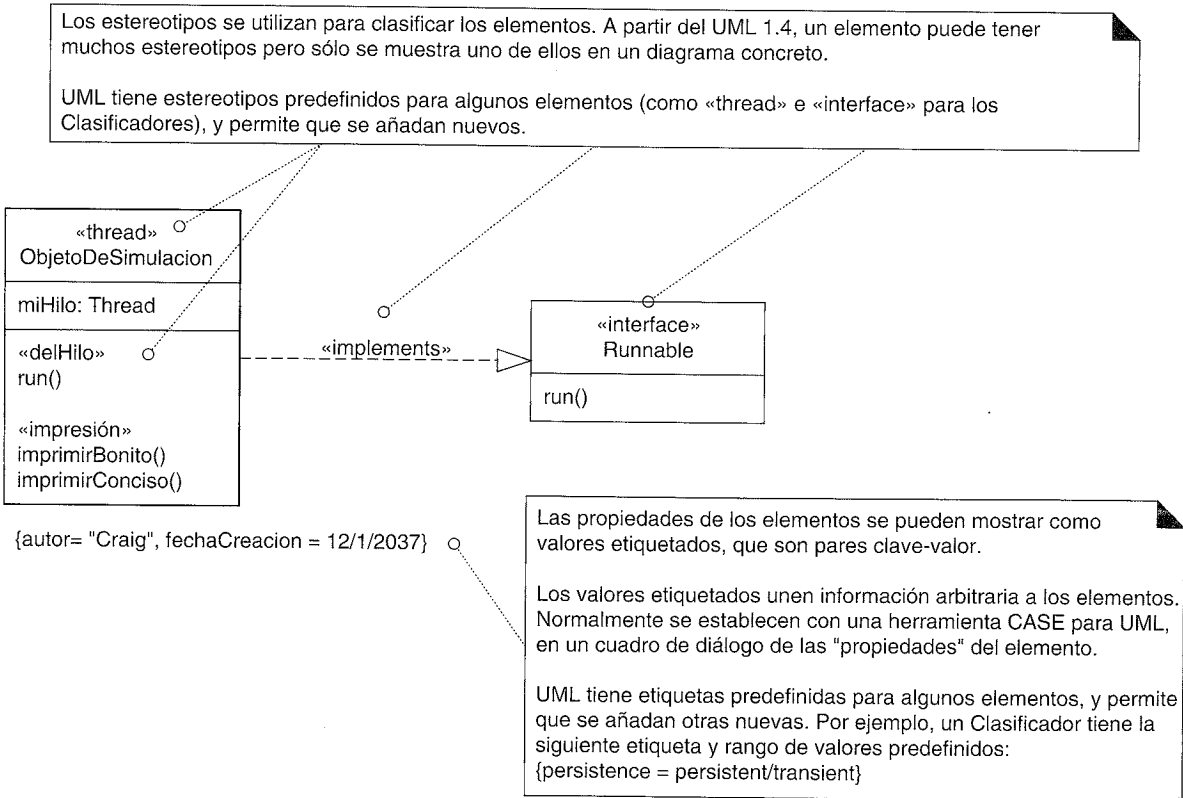


Figura 38.1. Estereotipos y propiedades.

Interfaces de paquetes

Se puede representar un paquete que implementa una interfaz (ver Figura 38.2).

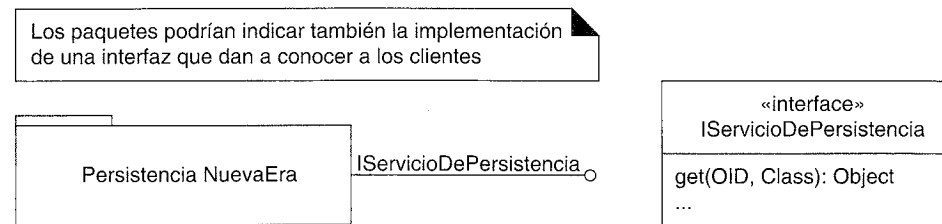


Figura 38.2. Interfaz de un paquete.

Dependencia

Pueden existir dependencias entre elementos cualesquiera, pero probablemente se utilizan con más frecuencia entre en los diagramas de paquetes de UML para ilustrar las dependencias de los paquetes (ver Figura 38.3).

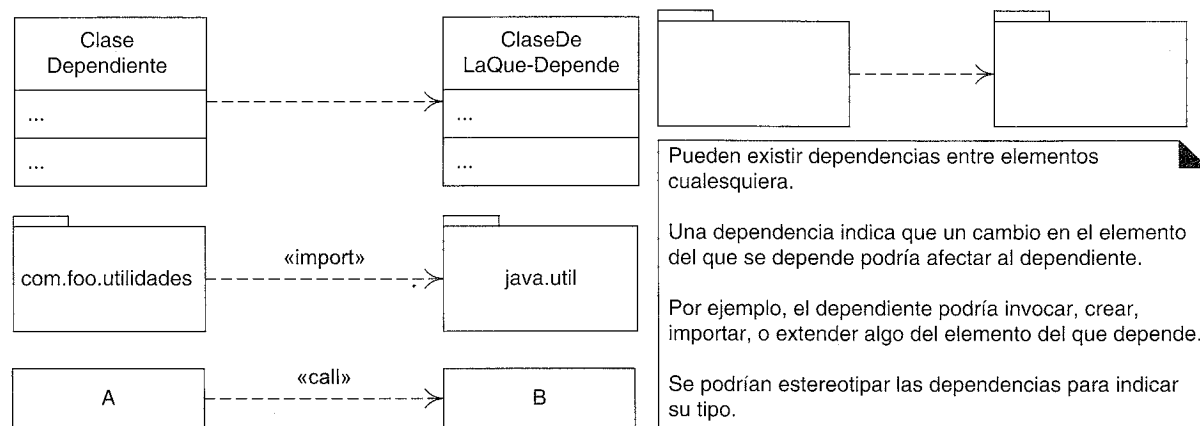


Figura 38.3. Dependencias.

38.2. Diagramas de implementación

UML define varios diagramas que se pueden utilizar para ilustrar los detalles de implementación. El que se utiliza más comúnmente es un diagrama de despliegue, para ilustrar el despliegue de los componentes y procesos en los nodos de proceso.

Diagramas de componentes

Citando textualmente: Un **componente** representa una parte de un sistema modular, desplegable, y reemplazable, que encapsula la implementación y expone un conjunto de interfaces [OMG01]. Podría ser, por ejemplo, código fuente, binario o ejecutable. Entre los

ejemplos encontramos navegadores o servidores HTTP, una base de datos, una DLL, o un fichero JAR (como para un Enterprise Java Bean). Los componentes en UML se representan en los diagramas de despliegue, en lugar de independientemente. La Figura 38.4 ilustra algo de la notación común.

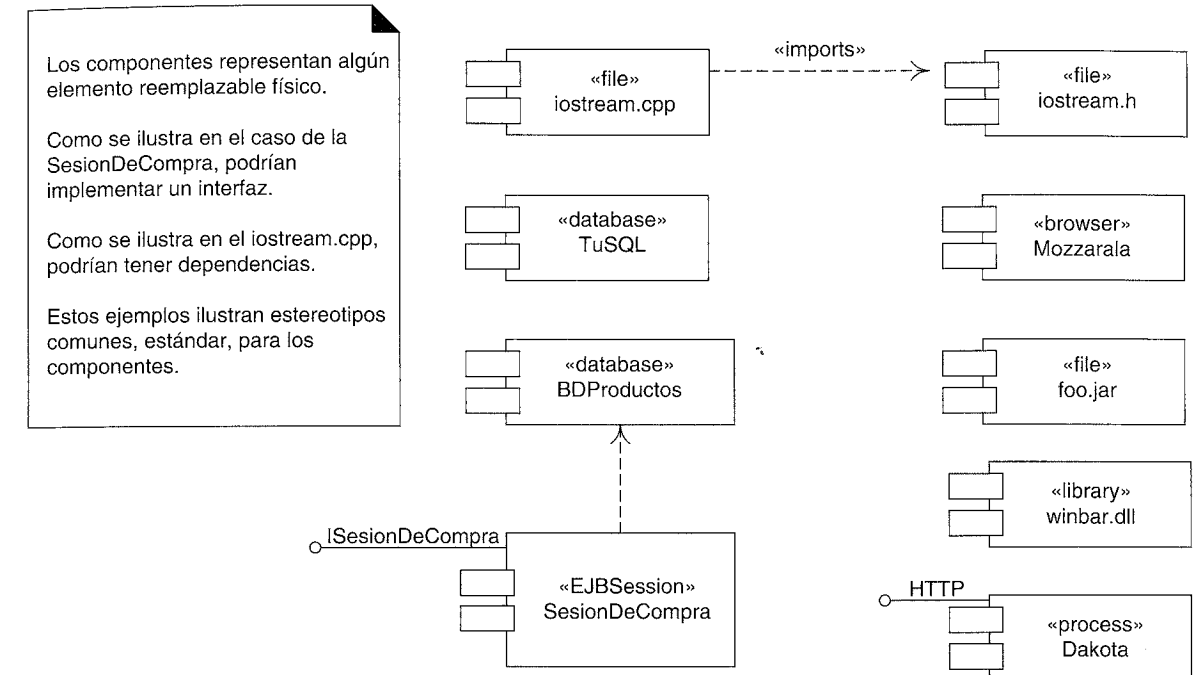


Figura 38.4. Componentes en UML.

Diagramas de despliegue

Un diagrama de despliegue muestra cómo se configuran las instancias de los componentes y los procesos para la ejecución run-time en las instancias de los **nodos** de proceso (algo con memoria y servicios de proceso; ver Figura 38.5).

38.3. Clase plantilla (parametrizada, genérica)

En la Figura 38.6 se muestran las clases plantilla y su instanciación.

Algunos lenguajes, como C++, soportan las clases plantilla, genéricas o parametrizadas. Además, esta característica se añadirá al lenguaje Java. Por ejemplo, en C++, *tabla<String, Persona>* declara la instanciación de la clase plantilla con claves de tipo *String*, y valores de tipos *Persona*.

38.4. Diagramas de actividades

Un **diagrama de actividades** de UML ofrece una notación rica para representar una secuencia de actividades. Podría aplicarse a cualquier propósito (como para mostrar los pasos de un algoritmo), pero se considera especialmente útil para visualizar los flujos de

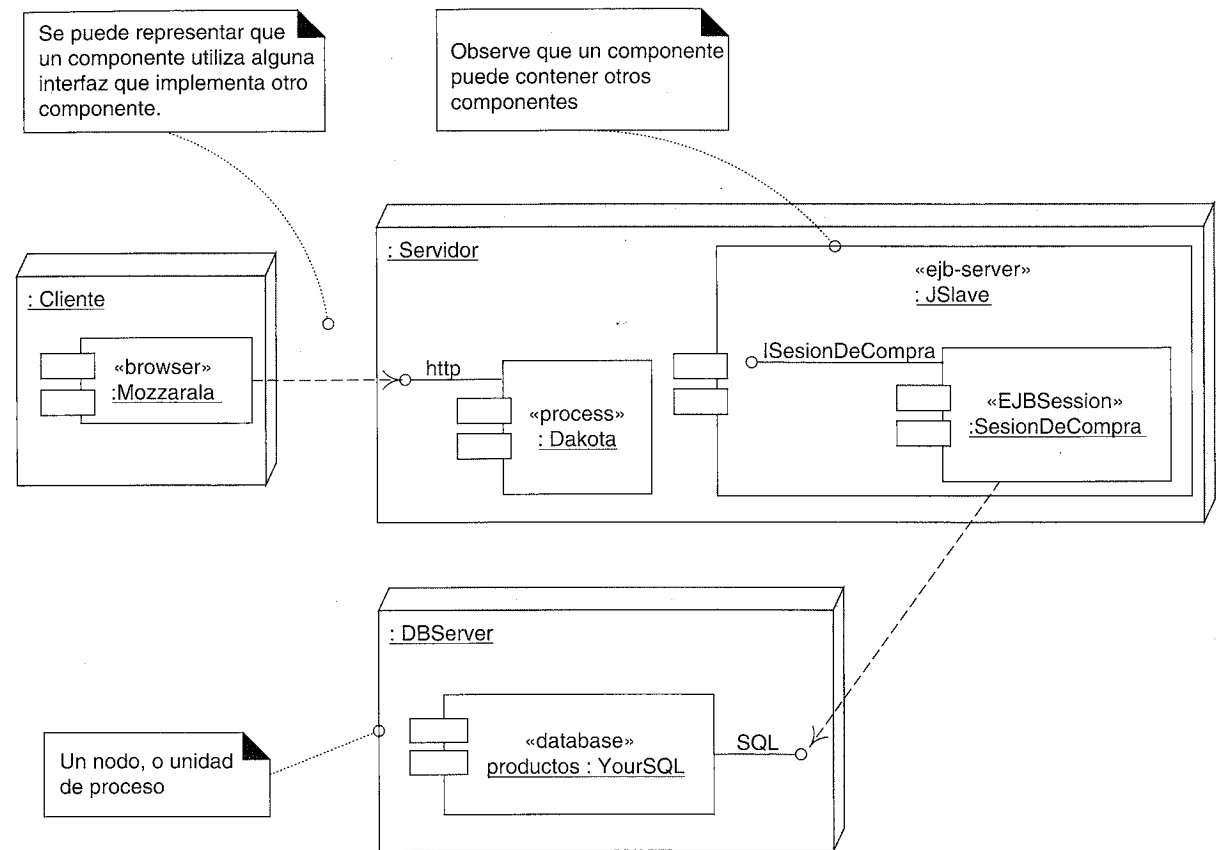


Figura 38.5. Un diagrama de despliegue.

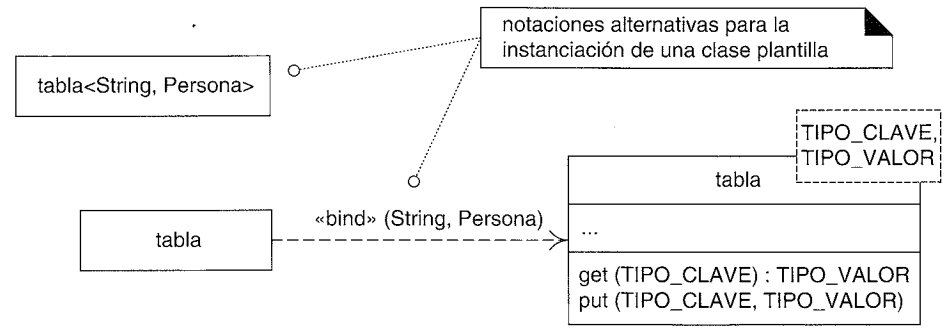


Figura 38.6. Clases plantilla.

trabajo y los procesos del negocio, o casos de uso. Uno de los flujos de trabajo (disciplinas) del UP es el **Modelado del Negocio**; su propósito es entender y comunicar “la estructura y la dinámica de la organización en el que se va a desplegar un sistema” [RUP]. Un artefacto clave de la disciplina del Modelado del Negocio es el **Modelo de Objetos del Negocio** (un superconjunto del Modelo del Dominio del UP), que visualiza esencialmente cómo funciona un negocio utilizando diagramas de clases, secuencia y actividades de UML. De esta manera, los diagramas de actividades se aplican especialmente dentro de la disciplina del Modelado del Negocio del UP.

Entre la notación destacada encontramos actividades concurrentes, calles y relaciones de flujo acción-objeto, como se ilustra en la Figura 38.7 (adaptado a partir de [OMG01, FS00]). Formalmente, un diagrama de actividad se considera un tipo especial de diagrama de estados de UML en el que los estados son acciones, y las transiciones de los eventos se disparan automáticamente al completarse la acción.

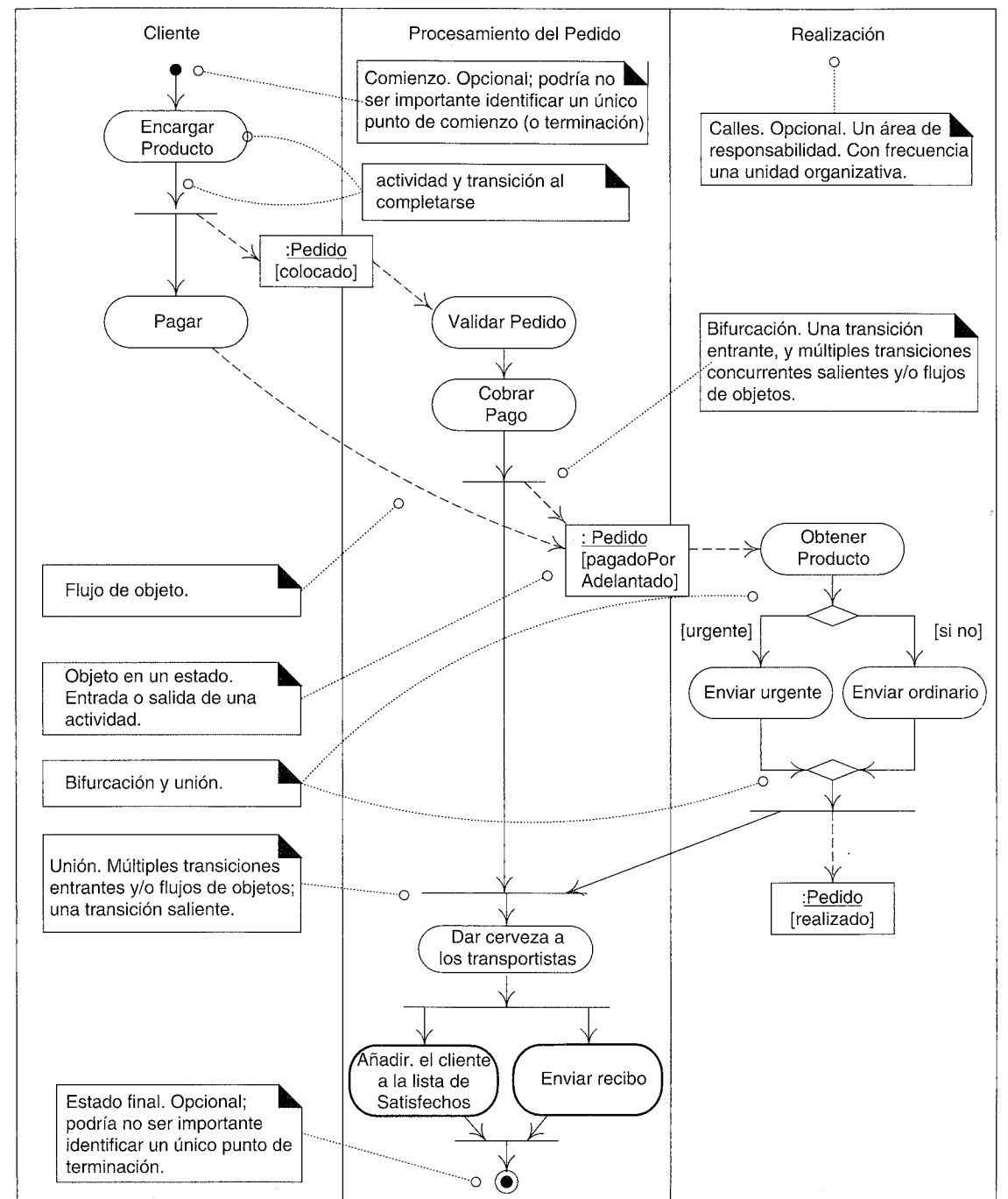


Figura 38.7. Diagrama de actividades.